AMBA® CHI Chip-to-Chip (C2C)

Architecture Specification

Document number	ARM IHI 0098
Document quality	Released
Document version	Issue B
Document confidentiality	Non-confidential
Date of issue	23 Mar 2026

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.

AMBA® CHI Chip-to-Chip (C2C) Architecture Specification

Release information

Date	Version	Changes
2026/Mar/23	B	• Third public release
2026/Feb/03	A.b	• Second public release
2024/Feb/07	A	• First public release

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Non-Confidential Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm's trademark usage guidelines at http://www.arm.com/company/policies/trademarks. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-21451 version 3

AMBA SPECIFICATION LICENCE

THIS END USER LICENCE AGREEMENT ("LICENCE") IS A LEGAL AGREEMENT BETWEEN YOU (EITHER A SINGLE INDIVIDUAL, OR SINGLE LEGAL ENTITY) AND ARM LIMITED ("ARM") FOR THE USE OF ARM'S INTELLECTUAL PROPERTY (INCLUDING, WITHOUT LIMITATION, ANY COPYRIGHT) IN THE RELEVANT AMBA SPECIFICATION ACCOMPANYING THIS LICENCE. ARM LICENSES THE RELEVANT AMBA SPECIFICATION TO YOU ON CONDITION THAT YOU ACCEPT ALL OF THE TERMS IN THIS LICENCE. BY CLICKING "I AGREE" OR OTHERWISE USING OR COPYING THE RELEVANT AMBA SPECIFICATION YOU INDICATE THAT YOU AGREE TO BE BOUND BY ALL THE TERMS OF THIS LICENCE.

"LICENSEE" means You and your Subsidiaries. "Subsidiary" means, if You are a single entity, any company the majority of whose voting shares is now or hereafter owned or controlled, directly or indirectly, by You. A company shall be a Subsidiary only for the period during which such control exists.

1. Subject to the provisions of Clauses 2, 3 and 4, Arm hereby grants to LICENSEE a perpetual, non-exclusive, non-transferable, royalty free, worldwide licence to:
   (i) use and copy the relevant AMBA Specification for the purpose of developing and having developed products that comply with the relevant AMBA Specification;
   (ii) manufacture and have manufactured products which either: (a) have been created by or for LICENSEE under the licence granted in Clause 1(i); or (b) incorporate a product(s) which has been created by a third party(s) under a licence granted by Arm in Clause 1(i) of such third party's AMBA Specification Licence; and
   (iii) offer to sell, sell, supply or otherwise distribute products which have either been (a) created by or for LICENSEE under the licence granted in Clause 1(i); or (b) manufactured by or for LICENSEE under the licence granted in Clause 1(ii).
2. LICENSEE hereby agrees that the licence granted in Clause 1 is subject to the following restrictions:
   (i) where a product created under Clause 1(i) is an integrated circuit which includes a CPU then either: (a) such CPU shall only be manufactured under licence from Arm; or (b) such CPU is neither substantially compliant with nor marketed as being compliant with the Arm instruction sets licensed by Arm from time to time;
   (ii) the licences granted in Clause 1(iii) shall not extend to any portion or function of a product that is not itself compliant with part of the relevant AMBA Specification; and
   (iii) no right is granted to LICENSEE to sublicense the rights granted to LICENSEE under this Agreement.
3. Except as specifically licensed in accordance with Clause 1, LICENSEE acquires no right, title or interest in any Arm technology or any intellectual property embodied therein. In no event shall the licences granted in accordance with Clause 1 be construed as granting LICENSEE, expressly or by implication, estoppel or otherwise, a licence to use any Arm technology except the relevant AMBA Specification.
4. THE RELEVANT AMBA SPECIFICATION IS PROVIDED "AS IS" WITH NO REPRESENTATION OR WARRANTIES EXPRESS, IMPLIED OR STATUTORY, INCLUDING BUT NOT LIMITED TO ANY WARRANTY OF SATISFACTORY QUALITY, MERCHANTABILITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE, OR THAT ANY USE OR IMPLEMENTATION OF SUCH ARM TECHNOLOGY WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADE SECRETS OR OTHER INTELLECTUAL PROPERTY RIGHTS.
5. NOTWITHSTANDING ANYTHING TO THE CONTRARY CONTAINED IN THIS AGREEMENT, TO THE FULLEST EXTENT PETMITTED BY LAW, THE MAXIMUM LIABILITY OF ARM IN AGGREGATE FOR ALL CLAIMS MADE AGAINST ARM, IN CONTRACT, TORT OR OTHERWISE, IN CONNECTION WITH THE SUBJECT MATTER OF THIS AGREEMENT (INCLUDING WITHOUT LIMITATION (I) LICENSEE'S USE OF THE ARM TECHNOLOGY; AND (II) THE IMPLEMENTATION OF THE ARM TECHNOLOGY IN ANY PRODUCT CREATED BY LICENSEE UNDER THIS AGREEMENT) SHALL NOT EXCEED THE FEES PAID (IF ANY) BY LICENSEE TO ARM UNDER THIS AGREEMENT. THE EXISTENCE OF MORE THAN ONE CLAIM OR SUIT WILL NOT ENLARGE OR EXTEND THE LIMIT. LICENSEE RELEASES ARM FROM ALL OBLIGATIONS, LIABILITY, CLAIMS OR DEMANDS IN EXCESS OF THIS LIMITATION.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

6. No licence, express, implied or otherwise, is granted to LICENSEE, under the provisions of Clause 1, to use the Arm tradename, or AMBA trademark in connection with the relevant AMBA Specification or any products based thereon. Nothing in Clause 1 shall be construed as authority for LICENSEE to make any representations on behalf of Arm in respect of the relevant AMBA Specification.
7. This Licence shall remain in force until terminated by you or by Arm. Without prejudice to any of its other rights if LICENSEE is in breach of any of the terms and conditions of this Licence then Arm may terminate this Licence immediately upon giving written notice to You. You may terminate this Licence at any time. Upon expiry or termination of this Licence by You or by Arm LICENSEE shall stop using the relevant AMBA Specification and destroy all copies of the relevant AMBA Specification in your possession together with all documentation and related materials. Upon expiry or termination of this Licence, the provisions of clauses 6 and 7 shall survive.
8. The validity, construction and performance of this Agreement shall be governed by English Law.

PRE-21451 version 3

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Product Status

The information in this document is final, that is for a developed product.

Web Address

http://www.arm.com

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Contents

AMBA® CHI Chip-to-Chip (C2C) Architecture Specification

AMBA® CHI Chip-to-Chip (C2C) Architecture Specification . . . ii
Release information . . . ii
Non-Confidential Proprietary Notice . . . iii
AMBA SPECIFICATION LICENCE . . . iv
Confidentiality Status . . . v
Product Status . . . v
Web Address . . . v

Part A Preface

About this specification

Intended audience . . . xiii

Using this specification

Conventions . . . xv
Typographical conventions . . . xv
Signals . . . xv
Numbers . . . xv
Additional reading . . . xvi
Feedback . . . xvii
Feedback on this specification . . . xvii
Inclusive terminology commitment . . . xvii

Part B Specification

Chapter B1 Introduction

B1.1 Overview . . . 20
B1.1.1 Multi-chip Symmetric Multi-Processor (SMP) configurations . . . 20
B1.1.2 Multi-chip coherent accelerator attach configuration . . . 21
B1.1.3 Hub-and-spoke topology . . . 21
B1.2 Terminology . . . 22
B1.3 Representation of transactions . . . 24

Chapter B2 Interface structures

B2.1 Interface layers . . . 26
B2.1.1 Protocol layer . . . 26
B2.1.2 Packetization layer . . . 26
B2.1.3 Link layer . . . 26
B2.1.4 Physical layer . . . 27
B2.2 Message classes . . . 28
B2.3 Channel connection . . . 29
B2.4 Multiple interfaces . . . 30
B2.4.1 Hash function example . . . 30

Chapter B3 Packetization

B3.1 Containers . . . 33
B3.2 Protocol header . . . 35

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B3.2.1 ChipTgtID . . . 35
B3.2.2 NumHops . . . 35
B3.2.3 G0IDECtrl . . . 35
B3.2.4 MsgStart . . . 36
B3.2.5 ContCredit . . . 36
B3.2.6 ContFormat . . . 37
B3.2.7 MsgCredit . . . 37
B3.2.8 ChipSrcID . . . 37
B3.2.9 ContRollover . . . 37
B3.3 Message packing . . . 39
B3.3.1 Rules and restrictions . . . 39
B3.3.2 Container rollover limit . . . 41

Chapter B4 Multi-request
B4.1 Introduction . . . 43
B4.2 Container granule savings for Multi-request . . . 45
B4.2.1 Non-WritePush . . . 45
B4.2.2 WritePush . . . 45
B4.3 Flow control . . . 46

Chapter B5 Message structures
B5.1 Transactions and transaction flows . . . 48
B5.1.1 Unsupported transactions . . . 48
B5.1.2 Redundant fields . . . 48
B5.1.3 Field mapping in requests to Subordinate Nodes . . . 50
B5.2 C2C-specific message fields . . . 51
B5.2.1 Message type, MsgType . . . 51
B5.2.2 Resource plane, ResPlane . . . 52
B5.2.3 Shared credit, SharedCrdt . . . 52
B5.2.4 Chunk valid, ChunkValid . . . 52
B5.2.5 RsvdZero . . . 52
B5.2.6 Opcode2 . . . 52
B5.2.7 Opcode3 . . . 53
B5.2.8 Union1 . . . 53
B5.2.9 Union2 . . . 54
B5.2.10 Union3 . . . 54
B5.3 Request message fields . . . 55
B5.4 Response message fields . . . 58
B5.4.1 Resp message type . . . 58
B5.4.2 Resp2 message type . . . 58
B5.5 Data message fields . . . 61
B5.6 WritePush message . . . 63
B5.6.1 Transaction flow . . . 63
B5.6.2 Message fields . . . 64
B5.7 WritePush data extension for Multi-request, WrDataExt . . . 67
B5.8 Snoop message fields . . . 68
B5.9 Uncredited Miscellaneous message fields . . . 69
B5.9.1 Activation message format . . . 69
B5.9.2 Connect message format . . . 71
B5.9.3 CrdtGrant message format . . . 72
B5.9.4 Properties message format . . . 73
B5.9.5 LinkStatus message format . . . 73
B5.9.6 Event message format . . . 75
B5.9.7 IDEMsg message format . . . 77

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B6 **Node ID management**
B6.1 Introduction . . . 84
B6.2 ChipID . . . 85
B6.3 AgentID . . . 86
B6.3.1 Request Node enhanced NodeID assignment . . . 86

Chapter B7 **Message routing**
B7.1 Introduction . . . 88
B7.2 Message routing structures . . . 89
B7.2.1 Chip address decoder . . . 89
B7.2.2 Port Aggregation Group address decoder . . . 90
B7.2.3 ID Route Table . . . 91
B7.3 Request and response routing . . . 92
B7.3.1 Single-hop system, no port aggregation . . . 92
B7.3.2 Single-hop system with port aggregation . . . 92
B7.4 Snoop request routing . . . 94
B7.4.1 Single-hop system, no port aggregation . . . 94
B7.4.2 Single-hop system with port aggregation . . . 94
B7.5 Multihop routing . . . 96
B7.5.1 Unpack Multihop routing . . . 96
B7.5.2 Pass-through Multihop routing . . . 97
B7.5.3 Unpack and Pass-through combination . . . 98
B7.5.4 MultihopSupport capability . . . 98
B7.5.5 Request and response Unpack Multihop routing with port aggregation . 99
B7.5.6 Snoop Unpack Multihop routing with port aggregation . . . 100
B7.5.7 Pass-through Multihop routing . . . 102

Chapter B8 **Flow control**
B8.1 Introduction . . . 105
B8.2 Message credits . . . 106
B8.2.1 Credit grant using a Protocol header . . . 107
B8.2.2 Credit grant using a MiscU message . . . 108
B8.3 Resource Planes . . . 109
B8.3.1 Request Resource Plane mapping . . . 110
B8.3.2 Request resource plane attributes . . . 110
B8.4 Data credit pools . . . 111
B8.4.1 Credits for DataS and DataL messages . . . 111
B8.4.2 Credits for WrReqData messages . . . 112
B8.5 Multi-request flow control . . . 113
B8.6 Ordered interface . . . 114
B8.7 DVM flow control . . . 115
B8.8 Multihop flow control . . . 116
B8.8.1 Unpack Multihop routing . . . 116
B8.8.2 Pass-through Multihop routing . . . 116

Chapter B9 **DVM transactions**
B9.1 Introduction . . . 118
B9.2 DVM Node, DN . . . 119
B9.2.1 Multiple DVM Nodes per chip . . . 119
B9.2.2 DVM Node assignment . . . 119
B9.3 DVM transaction flows . . . 120
B9.3.1 DVMReq . . . 120
B9.3.2 DVMSync . . . 121
B9.4 DVM transaction responses . . . 123
B9.5 DVM with Multihop routing . . . 124

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B10 RME-DA and RME-CDA support
B10.1 Introduction . . . 126
B10.2 Fields supporting RME-DA and RME-CDA . . . 128
B10.3 Host and device responsibilities . . . 129
B10.4 Request Nodes per device . . . 130

Chapter B11 Integrity and Data Encryption
B11.1 Introduction . . . 132
B11.1.1 Limitations . . . 133
B11.2 Format Z and IDE . . . 134
B11.2.1 Encryption and integrity protection to container fields . . . 134
B11.2.2 IDE message rollover rules . . . 134
B11.2.3 Packer reservation of granule G0 for MAC insertion . . . 135
B11.3 Multihop routing with IDE . . . 137

Chapter B12 Interface state management
B12.1 Introduction . . . 139
B12.2 Interface activation and deactivation . . . 140
B12.2.1 Activity states . . . 140
B12.2.2 DeactivateHint usage constraints . . . 140
B12.2.3 Messages . . . 141
B12.2.4 Activation and Deactivation flow . . . 141
B12.2.5 Activation trigger, ActTrig . . . 142
B12.2.6 Activation state, ActState . . . 143
B12.3 Coherency connect and disconnect . . . 144
B12.3.1 Coherency connect states . . . 144
B12.3.2 Messages . . . 145
B12.3.3 Coherency Trigger, CohTrig . . . 146
B12.3.4 Coherency states, CohStateRN and CohStateHN . . . 147
B12.4 DVM connect and disconnect . . . 148
B12.4.1 DVM connect states . . . 148
B12.4.2 Messages . . . 148
B12.4.3 DVM connect and disconnect flow . . . 148
B12.4.4 DVM Trigger, DVMTrig . . . 149
B12.4.5 DVM State, DVMState . . . 150

Chapter B13 Interface initialization
B13.1 Introduction . . . 152
B13.2 Initialization flow . . . 153
B13.3 PortID exchange . . . 155

Chapter B14 Protocol layer property exchange
B14.1 Introduction . . . 157
B14.2 On-chip payload properties . . . 160
B14.2.1 Uniform properties . . . 160
B14.2.2 Receiver and Transmitter properties . . . 160
B14.2.3 Expectations when exchanged on-chip payload properties differ . . . 162
B14.3 On-chip opcode and flow properties . . . 164
B14.3.1 Uniform properties . . . 164
B14.3.2 Receiver and Transmitter properties . . . 164
B14.3.3 Expectations when on-chip opcode and flow properties differ . . . 173
B14.4 C2C-specific properties . . . 176
B14.4.1 Uniform properties . . . 176
B14.4.2 Receiver and Transmitter properties . . . 178
B14.4.3 Expectations when exchanged C2C specific properties differ . . . 182

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B14.5 Property packing in MiscU.Properties messages . . . 187
B14.6 Property packing into registers . . . 190

Chapter B15 Software-visible registers
B15.1 Introduction . . . 192
B15.1.1 Registers access restrictions . . . 193
B15.1.2 Commit and lock-on-commit functionality . . . 194
B15.1.3 Error behavior . . . 194
B15.2 Chip offset register block . . . 195
B15.2.1 C2C_ChipHdr . . . 196
B15.2.2 C2C_ChipOffset1 . . . 197
B15.2.3 C2C_ChipOffset2 . . . 198
B15.2.4 C2C_ChipOffset3 . . . 199
B15.3 Chip properties register block . . . 200
B15.3.1 C2C_ChipPropHdr . . . 201
B15.3.2 C2C_ChipCap . . . 202
B15.3.3 C2C_ChipCtrlStatus . . . 204
B15.3.4 C2C_ChipIDCtrl . . . 205
B15.4 Chip address decoders register block . . . 206
B15.4.1 C2C_AddrDecHdr . . . 208
B15.4.2 C2C_AddrDecCommonCap . . . 209
B15.4.3 C2C_AddrDecCommonCtrl . . . 210
B15.4.4 C2C_AddrDecCap . . . 211
B15.4.5 C2C_AddrDecCtrlStatus . . . 212
B15.4.6 C2C_AddrDecBaseAddr . . . 214
B15.4.7 C2C_AddrDecEndAddr . . . 215
B15.4.8 C2C_AddrDecHashMask . . . 216
B15.4.9 C2C_AddrDecIwToTgtMap1 . . . 217
B15.4.10 C2C_AddrDecIwToTgtMap2 . . . 218
B15.4.11 C2C_AddrDecIwToTgtMap3 . . . 219
B15.4.12 C2C_AddrDecIwToTgtMap4 . . . 220
B15.5 Chip Port Aggregation Group address decoders register block . . . 221
B15.5.1 C2C_PortAggrHdr . . . 223
B15.5.2 C2C_PortAggrCommonCap . . . 224
B15.5.3 C2C_PortAggrCommonCtrl . . . 225
B15.5.4 C2C_PAGCap . . . 226
B15.5.5 C2C_PAGCtrlStatus . . . 227
B15.5.6 C2C_PAGHashMask . . . 229
B15.5.7 C2C_PAGIwToPortMap1 . . . 230
B15.5.8 C2C_PAGIwToPortMap2 . . . 231
B15.5.9 C2C_PAGIwToPortMap3 . . . 232
B15.5.10 C2C_PAGIwToPortMap4 . . . 233
B15.6 Chip ID Route Table register block . . . 234
B15.6.1 C2C_ChipIDRouteTableHdr . . . 236
B15.6.2 C2C_ChipIDRouteTableCap . . . 237
B15.6.3 C2C_ChipIDRouteTableCtrlStatus . . . 238
B15.6.4 C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1] . . . 240
B15.7 Port register locator register block . . . 241
B15.7.1 C2C_PortRegLocHdr . . . 242
B15.7.2 C2C_PortRegLocCtrlStatus . . . 243
B15.7.3 C2C_Port[0]RegPtr to C2C_Port[N-1]RegPtr . . . 244
B15.8 Port offset register block . . . 245
B15.8.1 C2C_PortRegOffsetHdr . . . 246
B15.8.2 C2C_PortRegOffset1 . . . 247
B15.8.3 C2C_PortRegOffset2 . . . 248

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.9 Port capabilities and control register block 249
B15.9.1 C2C_PortCapCtrlHdr 250
B15.9.2 C2C_PortCap1 251
B15.9.3 C2C_PortCap2 253
B15.9.4 C2C_PortCtrlStatus1 254
B15.9.5 C2C_PortCtrlStatus2 256
B15.10 Port common Logical Links register block 257
B15.10.1 C2C_LogLinkCommonHdr 258
B15.10.2 C2C_LogLinkCommonCap 259
B15.10.3 C2C_LogLinkOffset1 260
B15.10.4 C2C_LogLinkOffset2 261
B15.11 Port individual Logical Links register block 262
B15.11.1 C2C_LogLinkHdr 264
B15.11.2 C2C_LogLinkCap 265
B15.11.3 C2C_LogLinkCtrlStatus1 266
B15.11.4 C2C_LogLinkCtrlStatus2 270
B15.11.5 C2C_LogLinkRPMapTable 272
B15.11.6 C2C_LogLinkRPAttrTable 273
B15.11.7 Uniform property registers 275
B15.11.8 Receiver property registers 277
B15.11.9 Transmitter property registers 280
B15.11.10 C2C_LogLinkCrdt1 282
B15.11.11 C2C_LogLinkCrdt2 284
B15.11.12 C2C_LogLinkCrdt3 285
B15.11.13 C2C_LogLinkCrdt4 286
B15.11.14 C2C_LogLinkCrdt5 287

Part C Appendicies

Chapter C1 Revisions

C1.1 Changes between Issue A and Issue A.b 290
C1.2 Changes between Issue A.b and Issue B 291

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Part A
Preface

About this specification

This specification describes the AMBA® CHI *Chip-to-Chip* (C2C) architecture.

Intended audience

This specification is written for engineers who want to become familiar with the C2C architecture compatible with the CHI architecture.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Using this specification

The information in this specification is organized into parts, as described in this section:

Chapter B1 *Introduction*

Read this for an introduction to the CHI C2C architecture and the terminology in this specification.

Chapter B2 *Interface structures*

Read this for an overview of the functional layers between on-chip CHI and CHI C2C.

Chapter B3 *Packetization*

Read this for a description of packetization.

Chapter B4 *Multi-request*

Read this for a description of how the multi-request feature is supported in CHI C2C.

Chapter B5 *Message structures*

Read this for a description of the size and fields in each message type.

Chapter B6 *Node ID management*

Read this for a description of node identifier management for each C2C agent.

Chapter B7 *Message routing*

Read this for a description of message routing and supporting software-visible structures.

Chapter B8 *Flow control*

Read this for a description of the credit exchange used to flow control messages.

Chapter B9 *DVM transactions*

Read this for a description of the DVM transaction format and how they are propagated.

Chapter B10 *RME-DA and RME-CDA support*

Read this for an overview of RME-*Device Assignment* (RME-DA) and RME-*Coherent Device Assignment* (RME-CDA).

Chapter B11 *Integrity and Data Encryption*

Read this for a description of mechanisms to support the transport of CHI C2C containers across an IDE-protected link.

Chapter B12 *Interface state management*

Read this for a description of how to manage the connectivity state of the interface.

Chapter B13 *Interface initialization*

Read this for an overview of how to initialize C2C interface.

Chapter B14 *Protocol layer property exchange*

Read this for a description of interface properties and how these properties are communicated.

Chapter B15 *Software-visible registers*

Read this for a description of the software-visible registers.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Conventions

Typographical conventions

The typographical conventions are:

italic Highlights important notes, introduces special terminology, and denotes internal cross-references and citations.

bold Denotes signal names, and is used for terms in descriptive lists, where appropriate.

`monospace` Used for assembler syntax descriptions, pseudocode, and source code examples. Also used in the main text for instruction mnemonics and for references to other items appearing in assembler syntax descriptions, pseudocode, and source code examples.

SMALL CAPITALS Used for a few terms that have specific technical meanings.

Signals

The signal conventions are:

Signal level The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means:

- HIGH for active-HIGH signals.
- LOW for active-LOW signals.

Lowercase n At the start or end of a signal name denotes an active-LOW signal.

Numbers

Numbers are normally written in decimal. Binary numbers are preceded by `0b`, and hexadecimal numbers by `0x`. Both are written in a `monospace` font.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Additional reading

This section lists publications by Arm and by third parties.

See Arm Developer, http://developer.arm.com for access to Arm documentation.

[1] *AMBA® CHI Architecture Specification*. (ARM IHI 0051) Arm Ltd.

[2] *Universal Chiplet Interconnect Express (UCIe)*. Universal Chiplet Interconnect Express Consortium.

[3] *Arm® Architecture Reference Manual for A-profile architecture*. (ARM DDI 0487) Arm Ltd.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Feedback

Arm welcomes feedback on its documentation.

Feedback on this specification

If you have any comments or queries about our documentation, create a ticket at https://support.developer.arm.com.

As part of the ticket, please include:

- The title (AMBA® CHI Chip-to-Chip (C2C) Architecture Specification).
- The number (ARM IHI 0098 Issue B).
- The section name to which your comments refer.
- The page number(s) to which your comments refer.
- A concise explanation of your comments.

Arm also welcomes general suggestions for additions and improvements.

Inclusive terminology commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used terms that can be offensive.

Arm strives to lead the industry and create change.

This specification does not contain such terms. If you find offensive terms in this document, please contact terms@arm.com.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Part B
Specification

Chapter B1
Introduction

This specification is a Chip-to-Chip (C2C) extension of the *AMBA® CHI Architecture Specification* [1]. It contains the following sections:

- B1.1 *Overview*
- B1.2 *Terminology*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B1.1 Overview

The C2C extension enables building a system with multiple CPUs, accelerators, or other devices. CHI C2C focuses on the packetization of CHI messages, making them suitable to be transported over a chip-to-chip link. The packetization format is optimized for link utilization and latency, while avoiding complex packing and unpacking schemes.

The primary use cases are:

- *Multi-chip Symmetric Multi-Processor* (SMP). See B1.1.1 *Multi-chip Symmetric Multi-Processor (SMP) configurations*.
- Multi-chip coherent accelerator attach. See B1.1.2 *Multi-chip coherent accelerator attach configuration*.
- Hub-and-spoke. See B1.1.3 *Hub-and-spoke topology*.

The C2C extension can also be referred to as Chip(let)-to-Chip(let), as it covers both the board-level connection of multiple chips and the package level connection of multiple dies or chiplets.

B1.1.1 Multi-chip Symmetric Multi-Processor (SMP) configurations

The SMP configurations of interest are a small number of chips that are similar in their functionality and tightly connected together. Each chip includes *System-on-Chip* (SoC) components with a large number of processing cores. Each chip is expected to have attached memory that is coherently shared by processing cores on all the chips.

Figure B1.1 shows an example of a two-chip topology.

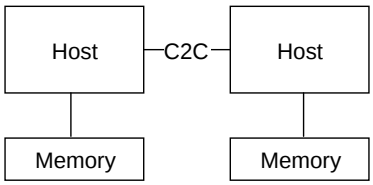


Figure B1.1: Example of a two-chip topology

Figure B1.2 shows an example of four chips in a fully connected topology. The attached memories will exist but are not shown in the figure.

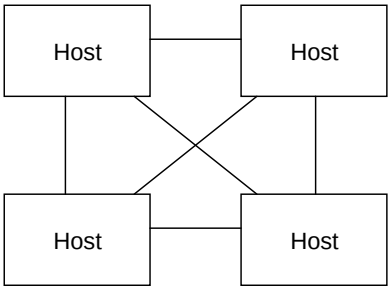


Figure B1.2: Example of four chips in a fully connected topology

Figure B1.3 shows an example of a four-chip mesh topology.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

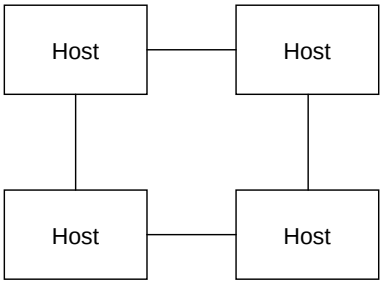


Figure B1.3: Example of a four-chip mesh topology

B1.1.2 Multi-chip coherent accelerator attach configuration

The accelerator attach configurations of interest are one or more coherent accelerators connected to a host chip.

The accelerators can be fully coherent or IO coherent.

Figure B1.4 shows an example of a topology of coherent accelerators.

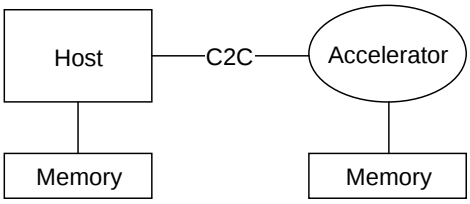


Figure B1.4: Example of a topology with coherent accelerators

B1.1.3 Hub-and-spoke topology

In a hub-and-spoke topology, multiple Hosts and multiple coherent accelerators can be attached to a central Hub, which may also have memory attached.

Figure B1.5 shows the hub-and-spoke topology.

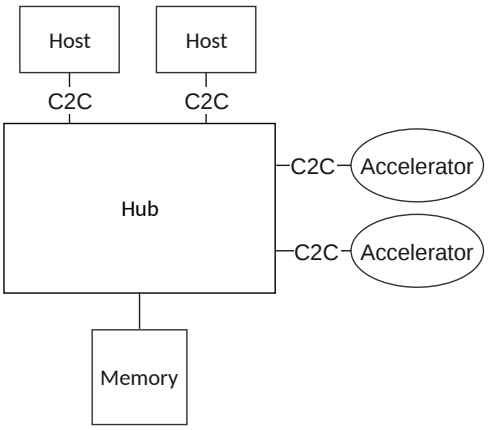


Figure B1.5: Example of a hub-and-spoke connected topology

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B1.2 Terminology

The following terms have a specific meaning within this specification:

Container

A container is the smallest fixed size unit in which messages are transferred across an interface.

Container rollover

Occurs when the last message of a container is a multi-granule message which spans across to the next container.

DN

DVM Node (DN) is a node located within the chip that processes DVM operations to and from remote chips.

Flit

Flow control unit. In this specification, this term is used to refer to the Link Layer flit.

IwP

The Interleave way to Port (IwP) table is a component of the PAG address decoder routing structure which associates a hashed version of an address to a particular port and Logical Link within a set of aggregated ports.

IwT

The Interleave way to Target (IwT) table is a component of the address decoder routing structure which associates a hashed version of an address to a particular target that is either a local target, a port, or a PAG address decoder.

Lock-on-commit

Allows software to request that when committing a change to a particular group of registers, the access is changed to Read-Only (RO). This is implemented at the granularity of a register block, unless specified otherwise.

Logical Link

A logical connection between a Transmitter interface of a chip or chiplet and a Receiver interface of a different chip or chiplet, where C2C containers are exchanged and routed unmodified. A Logical Link can cover a single hop or multiple hops.

MISC

A C2C-specific message class that groups all messages not directly related to on-chip CHI protocol messages. The class is further sub-divided in MiscU (Uncredited) and MiscC (Credited) messages.

Multi-request

A single request message targeting multiple contiguous cache lines.

PAG Address Decoder

A software-visible routing structure that distributes messages across a set of aggregated ports based on address.

Receiver

An interface component that receives messages from the Link layer.

RP

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Resource Planes are a collection of link resources that enable groups of messages within the same message class to be independently transported over the link.

Transmitter

An interface component that forwards messages to the Link layer.

WritePull

The semantics for a write transaction that relies on a three-step flow to send a write request, receive a Data Buffer Identifier (DBID) response, and send write data.

WritePush

The semantics for a write transaction that relies on a single-step flow to send a combined write request and write data message.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B1.3 Representation of transactions

Table B1.1 shows the representation of transactions.

Table B1.1: Representation of transactions

Specification use	Represents collectively
WrReqData*	WrReqDataS and WrReqDataL.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B2
Interface structures

This chapter describes the structure of the C2C interface. It contains the following sections:

- B2.1 *Interface layers*
- B2.2 *Message classes*
- B2.3 *Channel connection*
- B2.4 *Multiple interfaces*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B2.1 Interface layers

The functional logic between on-chip CHI interface and the chip pins that a message traverses through is divided into multiple functional layers [1]. These layers are referred to as:

- Protocol layer
- Packetization layer
- Link layer
- Physical layer

Link layer and Physical layer presence is expected, but not required, for connecting two chips together.

Figure B2.1 shows the C2C interface structure.

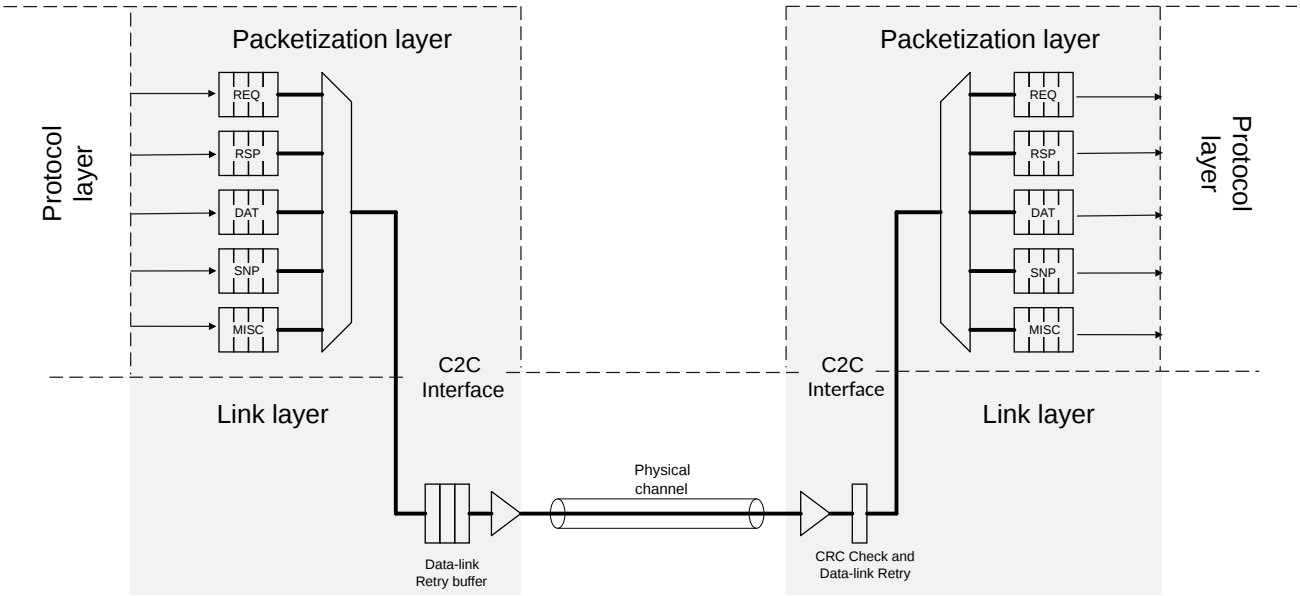


Figure B2.1: C2C interface structure

B2.1.1 Protocol layer

The Protocol layer handles protocol conversion between on-chip CHI [1] and CHI C2C.

The logic required for the conversion is minimal.

B2.1.2 Packetization layer

When the Link layer is present, the Packetization layer is between the Protocol layer and the Link layer.

If the Link layer is not present, the Packetization layer connects directly to the external signals. If required, the Packetization layer packs outgoing messages into a fixed size container and unpacks incoming containers into individual messages. The incoming messages are forwarded to the Protocol layer or other functional blocks within the Packetization layer.

B2.1.3 Link layer

The Link layer is responsible for handling message transport at a *FLow control unIT* (Flit) granularity.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

The Link layer is also responsible for optional data integrity and transport error detection and recovery. Typically, this is done by adding a generated *Cyclic Redundancy Check* (CRC) checksum to a flit being transmitted and checking the CRC checksum on the received flit.

When an error occurs, the Link layer can use a flit retry mechanism to recover from the error.

There must be a space in a container for the Link layer to fill the following transport information:

- Flit header
- Link reliability

B2.1.4 Physical layer

The Physical layer is responsible for providing reliable electrical connectivity between the two chips.

Some of the capabilities of the Physical layer include the logic to overcome latency skew among different signals and link retraining to maintain signal integrity.

The details of the Link layer and Physical layer are outside the scope of this specification.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B2.2 Message classes

This specification defines a message class as the group of messages of a single CHI message type. Message classes correspond to on-chip CHI physical channels [1].

Messages are grouped into the following five message classes:

- Request, REQ
- Response, RSP. The RSP message class does not include data.
- Snoop, SNP
- Data, DAT
- Miscellaneous, MISC

MISC is the C2C-specific message class that is defined to group all messages that do not directly relate to on-chip CHI protocol messages [1]. Example uses of MISC messages are interface initialization and message credit grant.

For further MISC message details, see B5.9 *Uncredited Miscellaneous message fields*.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B2.3 Channel connection

There are two different approaches to connect the different message class channels between chips:

1. Shared channel connect
2. Independent channel connect

The common usage for a C2C interface is expected to be a shared channel connection. All on-chip channels share the same connection between chips.

A fixed size container is used as the unit of exchange across the C2C interface. The messages to be sent are placed in a container. The container can include multiple messages from the same message class and messages from different message classes.

The use of a container with messages included from any message class instead of using independent physical channels for each message class typically results in a better link efficiency.

Figure B2.2 shows a shared channel connection.

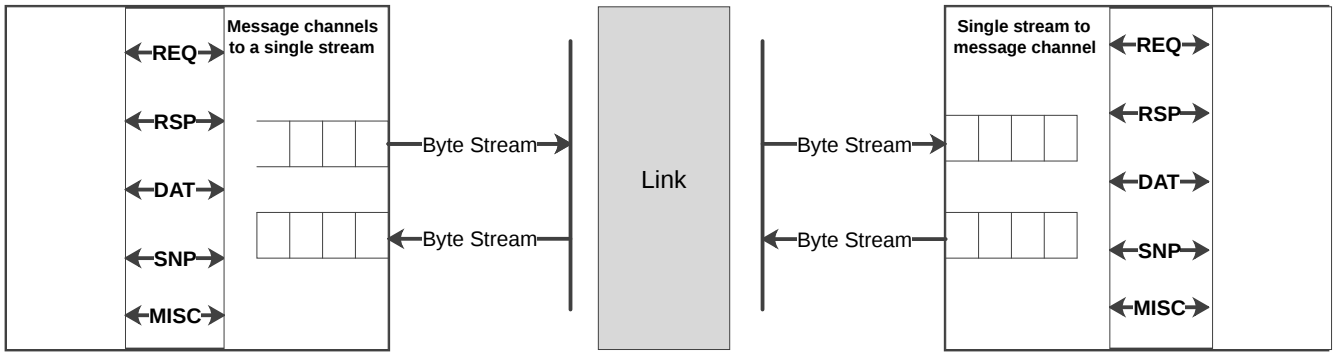


Figure B2.2: Shared channel connect

Alternatively, an independent channel connect can be used, where a separate physical set of wires is used for each message class. This type of connection provides implementation ease due to not requiring packetization logic. However, this is at the cost of potential underutilization of the physical channels due to lack of messages to fully utilize all available channels.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B2.4 Multiple interfaces

It is expected that a single C2C interface is used to connect two chips.

It is permitted that multiple C2C interfaces are used between two chips to meet greater throughput requirements.

When using multiple interfaces, both the chips must follow message association requirements.

Within transaction For a Request or Snoop, all associated Data or Response messages must use the same interface.

Between transaction All allocating requests and snoops to the same address from the same Requester agent must use the same interface.
All requests that require same-agent ordering to the same address range must use the same interface.
DVMReq and DVMSync transactions from a given source must use the same interface. See B9.3 *DVM transaction flows*.
MISC messages to the same target must use the same interface.

A Completer can ensure that a response is sent on the same interface as the corresponding memory or snoop request by tracking the interface used by the request.

A Home can ensure snoop requests are sent on the same interface as requests to that address by either tracking the interface the request was received on or by using a common address hash.

When using an address hash:

- A chip is required to use a single address hash for requests to a given target chip.
- Requests to different target chips can use different address hash functions.
- It is permitted to use an approach where both the request and snoop sides are able to be programmed with a hash function. An example hash function is provided in B2.4.1 *Hash function example*.

The number of interfaces permitted for aggregating traffic between two components is 1 to 16.

B2.4.1 Hash function example

Figure B2.3 shows how the interface number is determined from an address hash.

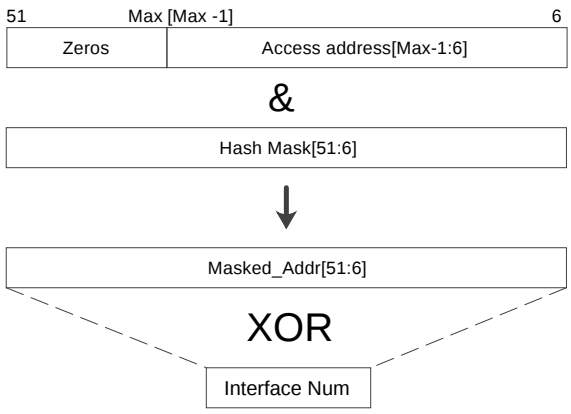


Figure B2.3: Interface determination from address hash

The following steps are used to determine the interface on which to send a request or a snoop:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

1. The cache line aligned input address is first filtered through a Hash Mask. The most significant address bits that are not used must be 0 before filtering the address. In this example, the result of the filtering is labeled as Masked_Addr.
2. The individual bits of the Masked_Addr are combined with a logical XOR to obtain the target interface in a manner defined by the number of interfaces between the two chips.

The following are examples of determining the interface number for a given address when the number of interfaces is:

- A power-of-two

  For 2 interfaces Interface Num[0] = Masked_Addr[n-1] XOR Masked_Addr[n-2]...Masked_Addr[7] XOR Masked_Addr[6]

  For 4 interfaces Interface Num[1:0] = Masked_Addr[n-1:n-2] XOR Masked_Addr[n-3:n-4]...Masked_Addr[9:8] XOR Masked_Addr[7:6]

  For 8 interfaces Interface Num[2:0] = Masked_Addr[n-1:n-3] XOR Masked_Addr[n-4:n-6]...Masked_Addr[11:9] XOR Masked_Addr[8:6]

- Not a power-of-two.
  The first step is to perform the hash using a power-of-two number of interfaces which is larger than the available number of interfaces. The obtained intermediate interface number is mapped to the interface number by using a table or by some other implementation means. The table is heuristically set up to distribute, as evenly as possible, the hashed numbers across the available number of interfaces. The larger the intermediate interface number relative to the actual number of interfaces, the greater than the uniformity of message distribution across the interfaces.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B3
Packetization

This chapter describes the process of packetization within the C2C interface. It contains the following sections:

- B3.1 *Containers*
- B3.2 *Protocol header*
- B3.3 *Message packing*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B3.1 Containers

A container is 256 bytes and comprises Link header bytes, Protocol header bytes and message bytes organized in granules.

Table B3.1 shows the container format definitions.

Table B3.1: Container formats

Format type	Payload (bytes)	Link header (bytes)	Protocol header (bytes)	Reserved (bytes)	Link layer compatibility
Format X	240, split into: – Twelve 20-byte granules	6	10	-	UCIe 256-byte Latency Optimized Mode with optional bytes.
Format Y	226, split into: – Ten 20-byte granules – One 16-byte granule. – One 10-byte granule.	20	10	-	CXL 256-byte latency optimized flit format.
Format Z	228, split into: – Eleven 20-byte granules – One 8-byte granule.	16	10	2[a]	UCIe Non-latency Optimized flit format for CXL.cachemem or Streaming protocol and CXL 256-byte non-latency optimized flit format.

[a] In Format Z, bytes 126 and 127 are Reserved and must not be used by the Protocol layer

Figure B3.1 shows the container byte layouts for Format X, Format Y, and Format Z. The granules are distributed so they are aligned across 64-byte and 128-byte parts of the container.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

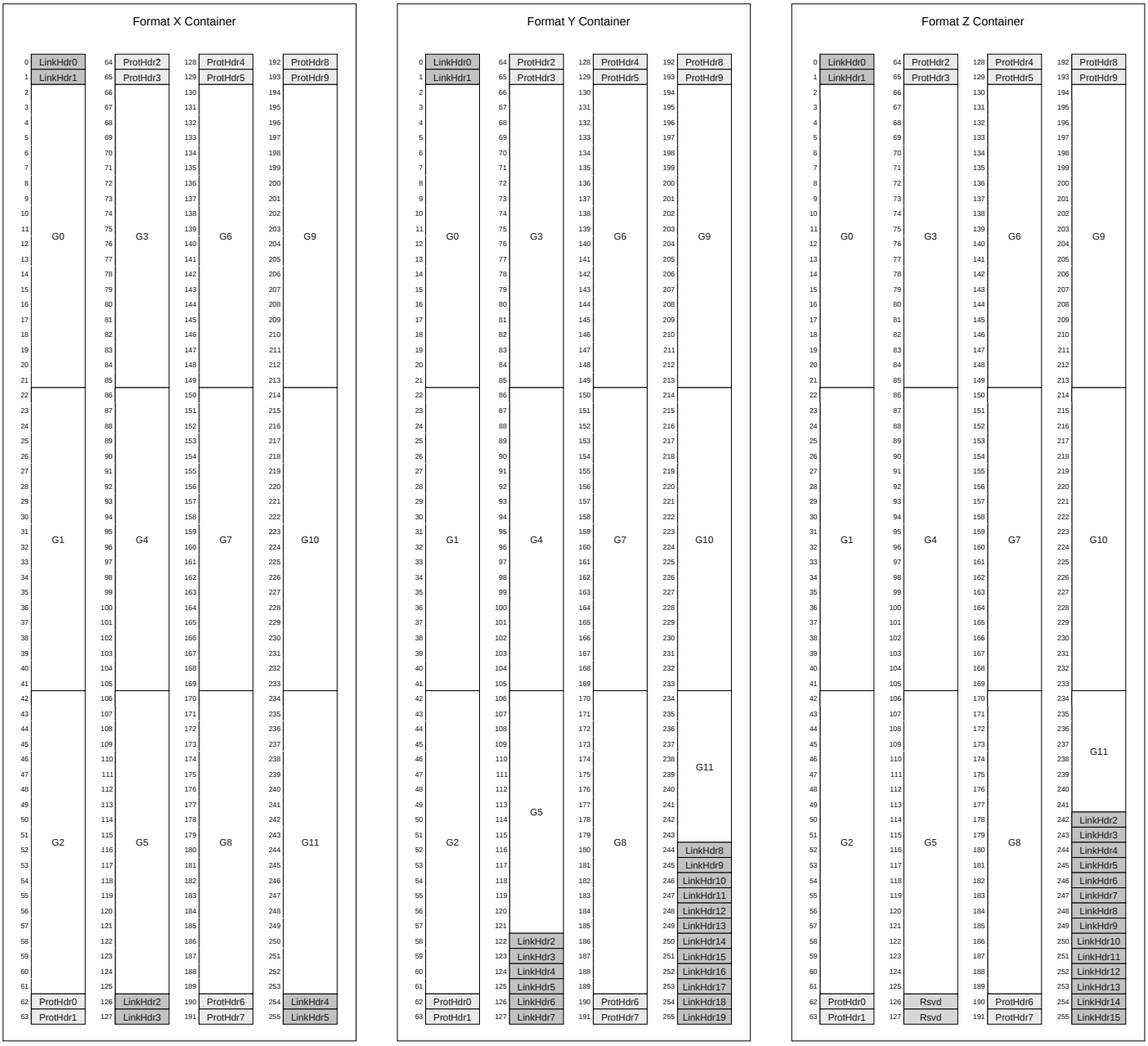


Figure B3.1: Container layout format

For information on ProtHdr bits, see B3.2 *Protocol header*.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B3.2 Protocol header

The bytes that are not used for message granules are either the Link header or the Protocol header.

The description of Link header bytes are outside the scope of this specification. Typical usage for these bits are for adding a flit header, a CRC, and an FEC.

The Protocol layer header bits are allocated as shown in Figure B3.2. Reserved header bits must be set to 0 by the Transmitter and ignored by the Receiver.

ProtHdr0	0	Rsvd
	1	Rsvd
	2	ChipTgtID[0]
	3	ChipTgtID[1]
	4	ChipTgtID[2]
	5	ChipTgtID[3]
	6	ChipTgtID[4]
	7	ChipTgtID[5]
ProtHdr1	0	NumHops[0]
	1	NumHops[1]
	2	NumHops[2]
	3	Rsvd
	4	G0IDECtrl
	5	MsgStart[0]
	6	MsgStart[1]
	7	MsgStart[2]

ProtHdr2	0	ContCrdt[0]
	1	ContCrdt[1]
	2	ContCrdt[2]
	3	ContCrdt[3]
	4	ContFormat[0]
	5	ContFormat[1]
	6	Rsvd
	7	Rsvd
ProtHdr3	0	Rsvd
	1	Rsvd
	2	Rsvd
	3	Rsvd
	4	Rsvd
	5	MsgStart[3]
	6	MsgStart[4]
	7	MsgStart[5]

ProtHdr4	0	MsgCredit[0]
	1	MsgCredit[1]
	2	MsgCredit[2]
	3	MsgCredit[3]
	4	MsgCredit[4]
	5	MsgCredit[5]
	6	MsgCredit[6]
	7	MsgCredit[7]
ProtHdr5	0	MsgCredit[8]
	1	MsgCredit[9]
	2	MsgCredit[10]
	3	MsgCredit[11]
	4	MsgCredit[12]
	5	MsgCredit[13]
	6	MsgCredit[14]
	7	MsgCredit[15]

ProtHdr6	0	ChipSrcID[0]
	1	ChipSrcID[1]
	2	ChipSrcID[2]
	3	ChipSrcID[3]
	4	ChipSrcID[4]
	5	ChipSrcID[5]
	6	Rsvd
	7	Rsvd
ProtHdr7	0	Rsvd
	1	Rsvd
	2	Rsvd
	3	Rsvd
	4	Rsvd
	5	MsgStart[6]
	6	MsgStart[7]
	7	MsgStart[8]

ProtHdr8	0	Rsvd
	1	Rsvd
	2	Rsvd
	3	Rsvd
	4	Rsvd
	5	Rsvd
	6	Rsvd
	7	Rsvd
ProtHdr9	0	Rsvd
	1	Rsvd
	2	Rsvd
	3	Rsvd
	4	ContRollover
	5	MsgStart[9]
	6	MsgStart[10]
	7	MsgStart[11]

Figure B3.2: Protocol header field layout

B3.2.1 ChipTgtID

The ChipTgtID field identifies the target ID of the chip to which the container needs to be routed and unpacked.

The ChipTgtID field has a width of 6 bits.

B3.2.2 NumHops

The NumHops field is set by the packer to indicate the number of hops required for the container to reach the ingress port where it will be unpacked.

The NumHops field is decremented as the container progresses towards the target.

The NumHops field is encoded to represent a minimum value of 1 and a maximum value of 8 hops. That is, when NumHops is 0, this indicates 1 hop.

The NumHops field has a width of 3 bits.

> **Note**
>
> This definition of the NumHops field is from the Transmitter perspective populating the field. From the Receiver perspective, when the container is received at the ingress port, one additional hop will already have been traversed.

B3.2.3 G0IDECtrl

The G0IDECtrl field is used with MsgStart[0] to provide information about the content of granule G0. See B11.2.3 *Packer reservation of granule G0 for MAC insertion* for more information.

The G0IDECtrl field is inapplicable and must be 0 in container formats other than Format Z.

The G0IDECtrl field is only applicable when the IDE_G0_Reservation property has negotiated to True.

The G0IDECtrl field has a width of 1 bit.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B3.2.4 MsgStart

The MsgStart[i] field indicates if a message starts in granule i. There is a MsgStart bit for each of the granules.

The MsgStart field has a width of 12 bits.

Table B3.2 shows the MsgStart[i] field encodings.

Table B3.2: MsgStart[i] field encodings

MsgStart[i]	Description
`0b0`	A message does not start in the granule i. The granule might be empty or be part of a message that started in an earlier granule.
`0b1`	A message starts in the granule i. That is, there is a MsgType field in the first bits of that granule.

B3.2.5 ContCredit

The ContCredit field is used for Pass-through Multihop routing to grant credits for container exchange with the connected C2C interface. Container credits guarantee that the connected C2C interface has buffer space to accept the container. One container credit corresponds to buffering space for one full container.

The ContCredit field has a width of 4 bits.

> **Note**
>
> For container credits, there is only a single set of credits to track and exchange. Container credits are only exchanged between adjacent chips and not between opposite ends of a Multihop Logical Link.

Table B3.3 shows the ContCredit field encodings.

Table B3.3: ContCredit field encodings

ContCredit	Description
`0b0000`	0 container credits.
`0b0001`	1 container credit.
`0b0010`	2 container credits.
`0b0011`	3 container credits.
`0b0100`	4 container credits.
`0b0101`	5 container credits.
`0b0110`	6 container credits.
`0b0111`	7 container credits.
`0b1000`	8 container credits.
`0b1001`	16 container credits.
`0b1010`	24 container credits.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B3.3 – *Continued from previous page*

ContCredit	Description
`0b1011`	32 container credits.
Others	Reserved.

B3.2.6 ContFormat

The ContFormat field indicates the container format used for container packing.

The ContFormat field has a width of 2 bits.

> **Note**
>
> When connecting to a Transmitter interface compatible with Issue A, a Receiver interface compatible with Issue B must not rely on the ContFormat container header field to determine the container format. Instead, it must rely on the result of the link property negotiation for the Container_Format Uniform property.

Table B3.4 shows the ContFormat field encodings.

Table B3.4: ContFormat field encodings

ContFormat	Description
`0b00`	Format X is in use.
`0b01`	Format Y is in use.
`0b10`	Format Z is in use.
Others	Reserved.

B3.2.7 MsgCredit

The MsgCredit field indicates the message credits granted by the Receiver. See B8.2 *Message credits*.

The MsgCredit field has a width of 16 bits.

B3.2.8 ChipSrcID

The ChipSrcID field identifies the source ID of the chip where the container was packed.

The ChipSrcID field is also required in a container to identify the chip corresponding to the credit pool used by the messages in the container and to associate any transmitted message class credits to the correct Logical Link.

The ChipSrcID field has a width of 6 bits.

B3.2.9 ContRollover

The ContRollover field indicates if the last message in the container spans across to the next container.

The ContRollover field has a width of 1 bit.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Note

When connecting to a Transmitter interface compatible with Issue A, a Receiver interface compatible with Issue B must not rely on the ContRollover container header field to determine whether to expect a container rollover. Instead, it must rely on the message information included in each granule.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B3.3 Message packing

Messages are packed into a container using granules. Messages are only permitted to begin at the start of a granule. This limits the number of positions each message type can occupy within a container to minimize decode logic.

The MsgStart vector in the Protocol Header determines if a new message starts at a particular granule. Each message begins with a MsgType field that defines the type of the message.

B3.3.1 Rules and restrictions

The restrictions on message placement within a container are:

- In a Format X container, a message can be placed in any granule.
- In a Format Y container:
  - A message, except RSP and MISC messages, can be placed in any granule except G5 and G11.
  - A RSP message can be placed in any granule.
  - A MISC message with a total message size:
    - Of 10 bytes or less can be placed in any granule.
    - Of 16 bytes or less can be placed in any granule except G11.
    - Greater than 16 bytes can be placed in any granule except G5 and G11.
- In a Format Z container:
  - A message, except MISC messages, can be placed in any granule except G11.
  - A MISC message with a total message size:
    - Of 10 bytes or less can be placed in any granule. When the message size is larger than 8 bytes, and it is placed in G11, the two most significant bytes are not transmitted and are implicitly set to 0.
    - Greater than 10 bytes can be placed in any granule except G11.
  - For information on IDE restrictions, see B11.2.2 *IDE message rollover rules*.
- Multi-granule messages:
  - Can start in any granule, with the following exceptions:
    - G5 and G11 in Format Y.
    - G11 in Format Z.
  - Must be contiguous, except when skipping G5 or G11 in Format Y, or G11 in Format Z. Multi-granule messages in Format Y skip over G5 and G11. Multi-granule messages in Format Z skip over G11. When a multi-granule message skips over a granule, the Transmitter must ensure that no ordering dependencies exist between the multi-granule message and the content of the skipped granule.
  - Can span across consecutive containers. See B3.3.2 *Container rollover limit* for more information. When spanning multiple containers, the message must continue in G0 of the next container, except when programmed to skip G0 for IDE purposes. See B11.2.2 *IDE message rollover rules* for more information.

Additionally, the container packing rules are:

- Inclusion of a message in a granule is optional. A container with no messages is permitted.
- The LSB of a message is placed in the LSB of the granule.
- Unused bits must be 0, including:
  - Granules that do not include a message.
  - Any bits between the end of a message and the end of the granule.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- 32-byte data is placed in the half of the 64-byte Data field, corresponding to the request address bit[5] value.
- Granules are organized into groups: G0 to G2, G3 to G5, G6 to G8, and G9 to G11.
  - Each group is permitted to include:
    - A maximum of one MiscU message.
    - A maximum of four response messages, where Resp2 counts as two response messages.
    - No granules populated.
    - The lowest numbered granule populated.
    - The two lowest numbered granules populated.
    - All granules populated.

Note

For example, if G6 is not populated, G7 and G8 must also be empty.

- A MiscU.LinkStatus message can only use G0.

Figure B3.3 shows an example of Format X container packing.

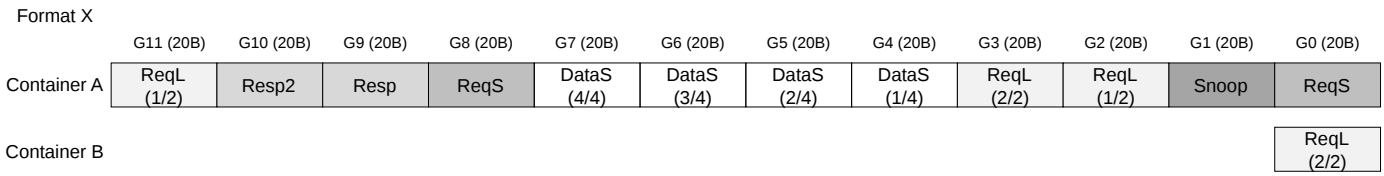


Figure B3.3: Example of Format X container packing

The two containers in Figure B3.3 include:

- Two ReqS and two ReqL messages, where one ReqL message spans over two containers.
- One Resp and one Resp2 message.
- One Snoop message.
- One DataS message.
- Empty granules in Container B.

Figure B3.4 shows an example of Format Y container packing.

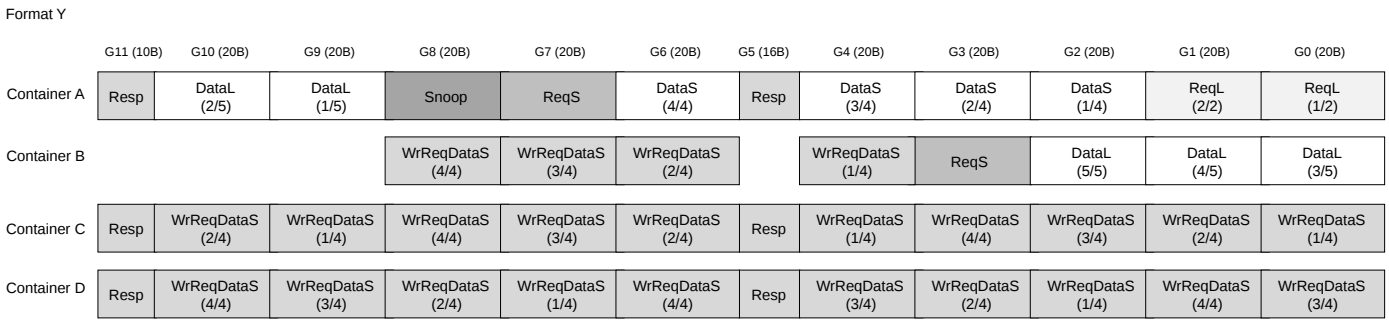


Figure B3.4: Example of Format Y container packing

The four containers in Figure B3.4 include:

- Two ReqS messages and one ReqL.
- Six Resp messages.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- One Snoop message.
- One DataS message and one DataL message.
  - The DataS message in Container A skips over G5.
  - The DataL message starts in Container A, skips over G11, and rolls over to the beginning of Container B.
- Six WrReqDataS messages.
  - One WrReqDataS message in Container B, skips over G5.
  - One WrReqDataS message starts in Container C G9, skips over G11, and rolls over to the beginning of Container D.
- Empty granules in Container B, as part of the second Group and the fourth Group.

Figure B3.5 shows an example of Format Z container packing.

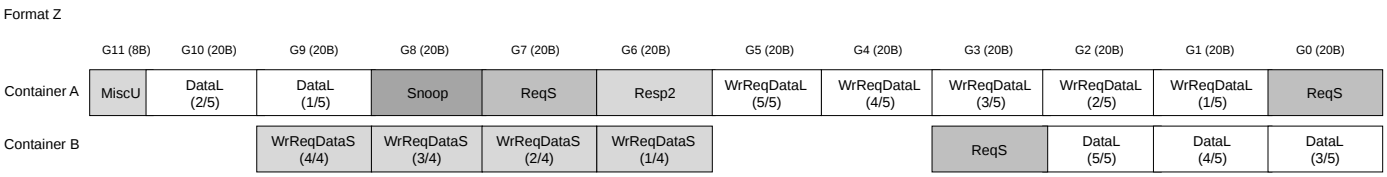


Figure B3.5: Example of Format Z container packing

The two containers in Figure B3.5 include:

- Three ReqS messages.
- One Resp2 message.
- One Snoop message.
- One DataL message in Container A, skipping over G11 and rolling over to Container B.
- One MiscU message in Container A, placed in G11.
- One WrReqDataS message and one WrReqDataL message.
- Empty granules in Container B, as part of the second Group and the fourth Group.

B3.3.2 Container rollover limit

Multi-granule messages are permitted to rollover across a container boundary.

A ContRollover bit in the protocol header is used to indicate that the last message in the container rolls over into the next container.

A single message can span at most two containers. A rollover sequence of multiple containers can be created where the first and last message of a container span across multiple containers.

The maximum permitted length of such a container rollover sequence is 4, where the first three containers have the ContRollover field set to 1.

The maximum container rollover limit that a port supports is indicated by the C2C_PortCap1.ContRolloverLimit register field. For each Logical Link, software is permitted to reduce the maximum container rollover limit to a lower value by programming the C2C_LogLinkCtrlStatus2.ContRolloverLimit register field for the container packer.

Interleaving of container rollover sequences is not permitted on a C2C link.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B4
Multi-request

This chapter describes the Multi-request feature. It contains the following sections:

- B4.1 *Introduction*
- B4.2 *Container granule savings for Multi-request*
- B4.3 *Flow control*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B4.1 Introduction

The CHI protocol describes a multi-request feature to permit multiple 64-byte requests with the same attributes to a contiguous address region to be combined into a single multi-request [1]. The multi-request usage rules and constraints are defined in the *AMBA® CHI Architecture Specification Issue H* protocol [1].

C2C permits multi-requests to be transmitted across the link as a mechanism to optimize link bandwidth utilization. See B4.2 *Container granule savings for Multi-request* for more information on the number of container granules saved with a multi-request.

C2C additionally supports WritePush multi-requests. A multi-request WritePush transaction of length N consists of a single WrReqDataL message, followed by ((N - 1) * (WrDataExt)) messages. The WrDataExt messages transport multiple 64-byte data quantities in incrementing address order and must be issued in order. See B5.7 *WritePush data extension for Multi-request, WrDataExt* for more detail on the WrDataExt message format and associated packing rules.

Support for the multi-request feature across the C2C link is controlled by the following Receiver and Transmitter properties:

- MultiReq_WritePull_Support
- MultiReq_WritePush_Support
- MultiReq_NonWrite_Support
- CacheLineID_Accurate.

CacheLineID_Accurate Receiver and Transmitter properties enable a chip that does not fully support the Multi-request feature to participate in multi-request transaction flows.

For the purposes of properties defined in this section, CHI transactions that are permitted to use the Multi-request feature are classified into groups:

- WritePull, including WriteNoSnpFull, WriteNoSnpPtl, WriteNoSnpZero, WriteUniqueFull, WriteUniquePtl and WriteUniqueZero.
- WritePush, including WriteNoSnpFull, WriteNoSnpPtl, WriteUniqueFull, and WriteUniquePtl.
- NonWrite, including ReadNoSnp, ReadNoSnpSep, ReadOnce, ReadOnceCleanInvalid, and ReadOnceMakeInvalid.

Note

WriteNoSnpZero and WriteUniqueZero are included in the WritePull group, but the associated CHI transaction flows are simpler than the other transactions as write data is not required to be transmitted [1].

For more information on the multi-request properties, see Table B14.5 and Table B14.4.

Table B4.1 shows the Order field encodings permitted to be used with the Multi-request feature.

The following key is used:

Y Yes, permitted.

- No, not permitted.

Table B4.1: Order field encodings permitted to be used with the Multi-request feature

Order[1:0]	Description	Permitted
`0b00`	No ordering required.	Y
`0b01`	Request accepted.	-[a]

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B4.1 – *Continued from previous page*

Order[1:0]	Description	Permitted
`0b10`	Request Order.	Y
	Ordered Write Observation (OWO)	Y
`0b11`	Endpoint Order.	-

[a] This Order field encoding is not supported as Request Retry flows are not supported on the C2C link.

For any multi-request that uses a non-zero value of Order, the hazarding guarantees are the same as if the multi-request was issued using separate requests.

Note

Implementations may choose to over address hazard to assist with logic complexity. A non-exhaustive list of how this could be implemented includes hazarding at:

- Twice the overall transaction size, to assist with any address un-alignment compared to the natural multi-request boundary.
- A granular 64-byte level up to a certain multi-request size and then a full 4-kilobyte range afterwards.
- A full 4-kilobyte range for any multi-request.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B4.2 Container granule savings for Multi-request

This section describes the container granule requirements for multi-request transactions.

B4.2.1 Non-WritePush

For a multi-request message of length N, the number of container granules saved compared to sending N individual ReqS requests is as follows:

- When N is between 1 and 8, and the used multi-request message is ReqS, the saving is (N - 1).
- When N is greater than 8, and the used multi-request message is ReqL, the saving is (N - 2).

Note

With the ReqS request, the message format includes 3 bits of NumReq, that is, NumReq[2:0]. This limits a single multi-request message to target up to 8 cache lines. When more than 8 cache lines are targeted, the largest granule saving is achieved using a single multi-request ReqL message that includes all 6 bits of NumReq.

For a multi-request message of length N, the number of container granules saved compared to sending N individual ReqL requests is (2*(N - 1)).

B4.2.2 WritePush

Table B4.2 shows the comparison of granules needed for a selective number of transaction sizes, depending on the message type and the use of Multi-request.

Table B4.2: Number of 20-byte granules required for certain data sizes, depending on MsgType

Cache lines (Value of N)	Total Data (Bytes)	Not using Multi-Request		Using Multi-Request
		WrReqDataS	WrReqDataL	WrReqDataL + ((N - 1) * WrDataExt)
2	128	8	10	9
4	256	16	20	17
8	512	32	40	33
16	1024	64	80	65
32	2048	128	160	129
64	4096	256	320	257
Formula	-	(4 * N)	(5 * N)	5 + ((N - 1)*4)

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B4.3 Flow control

For information on Multi-request flow control, see B8.5 *Multi-request flow control*.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B5
Message structures

This chapter describes the extent of support for on-chip CHI transaction flows, the on-chip CHI fields in messages that are optimized out as they are not required and resulting message formats with list of fields in each message class [1]. It contains the following sections:

- B5.1 *Transactions and transaction flows*
- B5.2 *C2C-specific message fields*
- B5.3 *Request message fields*
- B5.4 *Response message fields*
- B5.5 *Data message fields*
- B5.6 *WritePush message*
- B5.7 *WritePush data extension for Multi-request, WrDataExt*
- B5.8 *Snoop message fields*
- B5.9 *Uncredited Miscellaneous message fields*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.1 Transactions and transaction flows

This section lists the CHI transactions and fields that are not supported [1]. This section also describes the resulting mapping of fields in a request received by a Subordinate Node to account for the unsupported transactions.

B5.1.1 Unsupported transactions

The following transactions are not supported:

- Forwarding snoops
- Exclusive access to Non-cacheable and Device memory
- Request Retry flows
- *Direct Memory Transfer* (DMT). The ReturnNID and ReturnTxnID fields used for DMT are not included in a Request message.
- *Direct Cache Transfer* (DCT). The FwdNID and FwdTxnID fields used exclusively for DCT are not included in a Snoop message.
- *Direct Write Transfer* (DWT). The ReturnNID field used for DWT is not included in a Request message.
- When Tag Match is enabled, direct TagMatch response to Request Node from Subordinate Node is not supported.
- In Persist *Cache Maintenance Operation* (CMO) transaction flow, direct Persist response to Request Node from Subordinate Node is not supported.

Note

For a Read transaction, the chip which includes the Home Node and Subordinate Node can use DMT internally to that chip. This allows data to be forwarded directly to the C2C port from the Subordinate Node.

However, because DMT is not supported across the C2C interface, the data response only has a single SrcID field, rather than having both SrcID and HomeNID fields. At the C2C port, the SrcID field in the data response must be populated with the Node ID of the Home. This ensures that any subsequent message in the transaction, such as CompAck, is correctly sent to the Home.

For Write transactions and Snoop transactions, the SrcID field of the data response will contain the correct Node ID and does not need to be replaced with the Home ID.

At the Requester chip interface, the SrcID and HomeNID in the responses to the Requester must be populated by duplicating the common field value.

DCT support can be partially supported at the Home chip egress by keeping track of the snoop type before converting the Forwarding snoop into a Non-forwarding snoop. When the snoop response with data is received, the interface logic can split the response where one is sent to the Requester and the other is sent to the Home.

B5.1.2 Redundant fields

Certain CHI fields [1] are not required in C2C messages because of one or more of the following reasons:

- The transaction flow that uses the field is not supported.
- The field is used in messages in the Home Node to Subordinate Node interface only as part of flow optimizations not supported in C2C.
- Message class routing is based on address decode rather than ID.

Table B5.1 shows the CHI fields [1] not required in C2C messages.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.1: CHI message fields not included in C2C messages

Message Class	Field name	Bits	Description
Request	TgtID	11	This field is removed as requests are routed using address decode rather than using TgtID.
	ReturnNID ReturnTxnID DoDWT	11 12 1	These fields are removed as they are only used in requests from Home Node to Subordinate Node to provide an optimized direct path response communication between a Request Node and a Subordinate Node. Direct responses between these nodes are not supported.
	AllowRetry PCrdType	1 4	These fields are removed as they are only used in a Request Retry transaction flow.
	PAS[2]	1	This field is not transported as *Granular Data Isolation* (GDI) is not supported. See *AMBA® CHI Architecture Specification Issue H* [1] for more details.
Response	FwdState	3	This field is removed as it is only used in DCT transaction flows.
	PCrdType	4	This field is removed as it is only used in Request Retry transaction flows.
Data	FwdState	3	This field is removed as it is only used in DCT transaction flows.
	DataID	2	This field is removed because data is always sent across a C2C interface in a single 64-byte block.
	Poison	8	This field is removed as the poison indication is per the cache line and is encoded in RespErr field. The RespErr encoding of `0b10` is used to indicate Poison or Data Error.
	DataCheck	64	This field is removed as the Link layer may use its own proprietary mechanism, such as CRC and *Forward Error Correction* (FEC), to support transport reliability. This makes the DataCheck field redundant.
	NumDat Replicate	2 1	These fields are removed as they are used to support limited data elision. A cache line data is always transported as a 64B granule.
Snoop	FwdNID FwdTxnID	11 12	These fields are removed as they are only used in Forwarding snoops.
	VMIDExt	8	This field is removed as it is only used in DVM snoops. DVM snoops are sent across the C2C interface as DVM requests, therefore this field is not required in snoops.
	PAS[2]	1	This field is not transported as GDI is not supported. See *AMBA® CHI Architecture Specification Issue H* [1] for more details.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.1.3 Field mapping in requests to Subordinate Nodes

An interface receiving requests targeting Subordinate Node must map the fields in the received request to the on-chip CHI request message in the following manner:

- Duplicate SrcID onto ReturnNID
- Duplicate TxnID onto ReturnTxnID
- Set DoDWT to 0

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.2 C2C-specific message fields

The following fields are added to C2C messages and are not present in on-chip CHI messages [1]:

- MsgType
- ResPlane
- SharedCrdt
- ChunkValid
- RsvdZero
- Opcode2
- Opcode3
- Union1
- Union2
- Union3

B5.2.1 Message type, MsgType

This field identifies the C2C message type in terms of message class and size, where:

- Request, Data, and WrReqData* messages can be one of two types, dependent upon message length. See B5.3 *Request message fields*, B5.5 *Data message fields*, and B5.6 *WritePush message* for more details.
- Response messages can be one or two per granule of 20 bytes. See B5.4 *Response message fields*.
- Miscellaneous messages can be credited or uncredited.
  - See B5.9 *Uncredited Miscellaneous message fields* for details on uncredited MISC messages.
  - MiscC messages are not defined in this specification.

Table B5.2 shows the encoding of the MsgType field.

Table B5.2: MsgType field encodings

MsgType	Type	CHI channel
0b0000	MiscU	-
0b0001	MiscC	-
0b0010	ReqS	REQ
0b0011	ReqL	REQ
0b0100	Resp	RSP
0b0101	Resp2	RSP
0b0110	Snoop	SNP
0b0111	DataS	DAT
0b1000	DataL	DAT
0b1001	WrReqDataS	REQ and DAT
0b1010	WrReqDataL	REQ and DAT

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.2 – *Continued from previous page*

MsgType	Type	CHI channel
0b1011	WrDataExt	DAT
0b1100 - 0b1111	Reserved	

B5.2.2 Resource plane, ResPlane

This field is used to support multiple resource planes in Request and WritePush messages. See B8.3 *Resource Planes* and B8.4 *Data credit pools*.

B5.2.3 Shared credit, SharedCrdt

This field is used to support multiple resource planes and data credit pools in Request and Data message class messages. See B8.3 *Resource Planes* and B8.4 *Data credit pools*.

B5.2.4 Chunk valid, ChunkValid

This field is present in DataS, DataL, and WrReqDataL messages. The ChunkValid field value indicates if one or both of the 32 byte data chunks have valid data.

The ChunkValid field must be consistent with the associated Request Size field.

Table B5.3 shows the ChunkValid field encodings.

Table B5.3: ChunkValid field encoding

ChunkValid	Valid chunk
0b00	Reserved
0b01	Lower 32 bytes
0b10	Upper 32 bytes
0b11	All 64 bytes

B5.2.5 RsvdZero

This field indicates that the use of the bits is Reserved and a Transmitter must set the field value to 0, a Receiver must ignore the field and an intermediate agent must forward them unchanged. The inclusion of RsvdZero field in a message may be for starting the subsequent field at a byte or granule boundary or to fill the bits beyond useful information in the last granule.

B5.2.6 Opcode2

The Opcode2 field is a 2-bit field indicating the write request opcode for an individual cache line in a WrReqDataS message.

Table B5.4 shows the Opcode2 field in a WrReqDataS message.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.4: Opcode2 field encoding in WrReqDataS message

Opcode2	WrReqDataS command	Comments
`0b00`	WriteNoSnpFull.	
`0b01`	WriteUniqueFull.	
`0b10`	WriteNoSnpPtl.	All 64 BE bits set.
`0b11`	WriteUniquePtl.	All 64 BE bits set.

B5.2.7 Opcode3

The Opcode3 field is a 2-bit field indicating the DAT Opcode for an individual cache line being transferred using the WrDataExt message as part of a multi-request WritePush transaction flow.

> **Note**
>
> The inclusion of the Opcode3 field allows implementations to commit to a WritePush transaction of a particular length without needing all of that data available upfront. For example, this would allow WriteDataCancel to be used to complete a multi-request WriteUniquePtl transaction.

Table B5.5 shows the encoding of Opcode3.

Table B5.5: Opcode3 field encoding

Opcode 3	Encoding
`0b00`	NonCopyBackWriteData.
`0b01`	WriteDataCancel.
`0b10`	Reserved.
`0b11`	Reserved.

B5.2.8 Union1

The Union1 field is a 34-bit field providing a flexible way to encode MPAM, MECID or StreamID, and RSVDC fields. When transported, MECID and StreamID share a common 16-bit field, similarly to the Request message format. See Table B5.6.

Figure B5.1 shows how the Union1 field is divided across the MPAM, MECID or StreamID, and RSVDC fields.

The lower order bits of the Union1 field determine how the other bits of the field are used for MPAM, MECID or StreamID, and RSVDC values.

<table>
<tr><td>33</td><td>-</td><td>26</td><td>25</td><td>-</td><td>18</td><td>17</td><td>-</td><td>14</td><td>13</td><td>-</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td colspan="11">RsvdZero</td><td colspan="2"></td><td>0</td><td>0</td></tr>
<tr><td colspan="5">MECID[15:0] or StreamID[15:0]</td><td colspan="6">MPAM[14:0]</td><td>0</td><td>0</td><td>1</td></tr>
<tr><td colspan="5">RSVDC[15:0]</td><td colspan="6">MPAM[14:0]</td><td>1</td><td>0</td><td>1</td></tr>
<tr><td colspan="5">RSVDC[15:0]</td><td colspan="6">MECID[15:0] or StreamID[15:0]</td><td colspan="2"></td><td>1</td><td>0</td></tr>
<tr><td colspan="2">RSVDC[7:0]</td><td colspan="4">MECID[11:0] or StreamID[11:0]</td><td colspan="5">MPAM[11:0]</td><td colspan="2"></td><td>1</td><td>1</td></tr>
</table>

Figure B5.1: Union1 field use for MPAM, MECID, and RSVDC

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

When the MECID or StreamID field is 12 bits, the MECID[15:12] or StreamID[15:12] is implicitly set to 0.

The most significant bit of the MPAM field encodes PMG. The two least significant bits encode MPAMSP[1:0]. The remaining bits of the MPAM field encode PartID and are zero-extended, if required. When the MPAM field is 12-bits wide, PartID[11:9] is assumed to be 0.

B5.2.9 Union2

The Union2 field is a 49-bit field that is used in the WrDataExt message as part of a multi-request WritePush transaction flow to optionally transfer the Tag, TU, TagOp, RSVDC[23:16], and RSVDC[15:0] fields.

Figure B5.2 shows the packing of the aforementioned fields into the Union2 field.

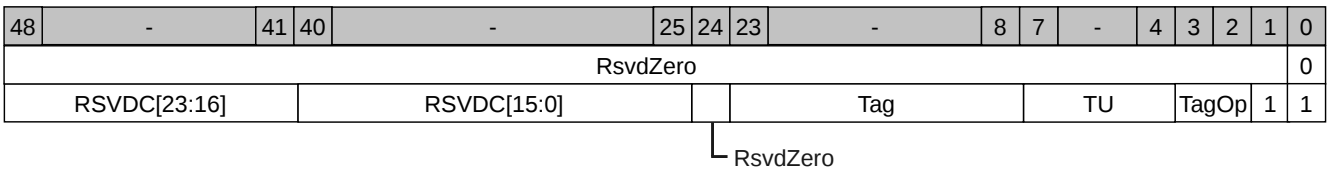


Figure B5.2: Union2 field use for Tag, TU, TagOp, and RSVDC

B5.2.10 Union3

The Union3 field is a 66-bit field that is used in the WrDataExt message as part of a WritePush transaction flow to optionally transfer the BE field.

Figure B5.3 shows the packing of the BE field into the Union3 field.

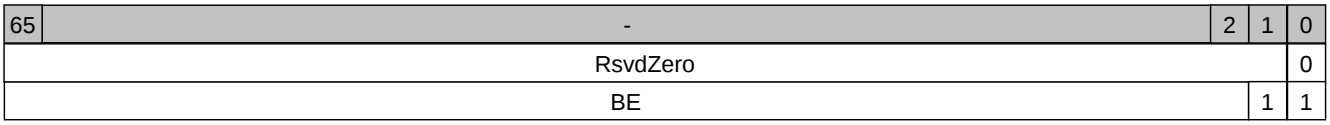


Figure B5.3: Union3 field use for BE

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.3 Request message fields

The fields in the Request message types are described in this section:

- ReqS, for short-sized formats
- ReqL, for long-sized formats

A Request message uses a ReqL message format when any of the following fields are present in the request and have a non-zero value:

- Addr[3:0] in any request. Addr[5:0] bits in Dataless requests of 64 bytes size to normal memory are permitted to be set to 0.
- LPID in an exclusive transaction.
- GroupID in a Persist CMO, a StashOnceSep, or a Write with TagOp value of *Match*.
- StashNID[10:1]
- StashLPIDValid
- PBHA
- LikelyShared
- DataTarget[6:1]
- RSVDC[N-1:16], where N is the negotiated width of the RSVDC field.
- NumReq[5:3], indicating a multi-request of length greater than 8.

When the use of ReqS is allowed, a Transmitter is permitted to use ReqL messages in place of ReqS. The use of ReqL instead of ReqS may lead to reduced container packing efficiency.

Table B5.6 lists Request message fields and their placement into ReqS and ReqL.

The columns in the table are defined as follows:

Common Fields marked with the same 'x' in 'Cx' share a common position in the message.

ReqS The request includes all mandatory fields and most commonly used optional fields.

ReqL Request size is extended to include all supported Request fields.

Table B5.6: Request message fields

Common	Field name	Width (bits)	
		ReqS	**ReqL**
	MsgType	4	4
	SharedCrdt	1	1
	ResPlane	3	3
	QoS	4	4
	SrcID	11	11
	TxnID	12	12
	PAS[1:0]	2	2
	SecSID1	1	1
	Order	2	2

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.6 – *Continued from previous page*

Common	Field name	Width (bits)	
		ReqS	**ReqL**
	MemAttr	4	4
	ExpCompAck	1	1
	TraceTag	1	1
	Addr[51:6]	46	46
	Addr[5:4]	2	2
	SnpAttr	1	1
	MPAM[a]	15	15
C0	MECID StreamID	16	16
	RSVDC[15:0]	16	16
C11	Size NumReq[2:0]	3	3
	Opcode	7	7
	TagOp	2	2
C1	StashNIDValid Endian Deep PrefetchTgtHint	1	1
C2	Excl SnoopMe CAH	1	1
C3	DataTarget[0] StashNID[0]	1	1
	MultiReq	1	1
	RsvdZero	2	2
	PBHA	0	4
	Addr[3:0]	0	4
	StashLPIDValid	0	1
	StashLPID	0	5
C4	{`4'b0`, DataTarget[6:1]} StashNID[10:1]	0	10
C5	{`3'b0`, LPID} PGroupID StashGroupID TagGroupID	0	8
	RSVDC[23:16]	0	8

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.6 – *Continued from previous page*

Common	**Field name**	**Width (bits)**	
		ReqS	**ReqL**
	RSVDC[31:24]	0	8
	LikelyShared	0	1
	NumReq[5:3]	0	3
	RsvdZero	0	108
Total (bits)[b]		158	210

[a] The most significant bit of the MPAM field encodes PMG. The two least significant bits encode MPAMSP[1:0]. The remaining bits encode PartID and are zero-extended, if required.

[b] The total bit count does not include the RsvdZero bits.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.4 Response message fields

The fields in the Response message types are described in this section:

- Resp
- Resp2

B5.4.1 Resp message type

Table B5.7 shows the fields in a Resp message type, where one response is packed into a single granule.

Table B5.7: Resp response message fields

Common	Field name	Width (bits)
	MsgType	4
	QoS	4
	TgtID	11
	SrcID	11
	TxnID	12
	Opcode	5
	RespErr	2
	Resp	3
	DataPull	1
	CBusy	3
	TagOp	2
	TraceTag	1
C6	DBID {4'b0, PGroupID} {4'b0, StashGroupID} {4'b0, TagGroupID}	12
	CacheLineID	6
	RsvdZero	3
Total (bits)[a]		77

[a] The total bit count does not include the RsvdZero bits.

B5.4.2 Resp2 message type

Table B5.8 shows the fields in a Resp2 message type, where two responses are packed into a single granule.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.8: Resp2 response message fields

Common	Field name	Width (bits)
	MsgType	4
	QoS	4
	TgtID	11
	SrcID	11
	TxnID	12
	Opcode	5
	RespErr	2
	Resp	3
	DataPull	1
	CBusy	3
	TagOp	2
	TraceTag	1
C6	DBID {4'b0, PGroupID} {4'b0, StashGroupID} {4'b0, TagGroupID}	12
	CacheLineID	6
	RsvdZero	3
	RsvdZero	4
	QoS	4
	TgtID	11
	SrcID	11
	TxnID	12
	Opcode	5
	RespErr	2
	Resp	3
	DataPull	1
	CBusy	3
	TagOp	2
	TraceTag	1
C6	DBID {4'b0, PGroupID} {4'b0, StashGroupID} {4'b0, TagGroupID}	12
	CacheLineID	6

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.8 – *Continued from previous page*

Common	Field name	Width (bits)
	RsvdZero	3
Total (bits)[a]		150

[a] The total bit count does not include the RsvdZero bits.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.5 Data message fields

DataS and DataL message formats are used for read data, snoop response data, and non-WritePush data messages.

The characteristics of DataS and DataL message format include:

- SrcID and HomeNID share a single 11-bit field. This field is used by:
  - HomeNID in CompData and DataSepResp
  - SrcID in SnpRespData and WriteData
- Data poison is a single bit per data transfer and is indicated by a RespErr value of DERR.
- The ChunkValid field is used to indicate if the corresponding 32 bytes includes valid data. See Table B5.3.
- The Data field starts at a byte boundary. This requires a RsvdZero bit padding before the Data field.
- Fields exclusively in DataL message start at a granule boundary.
- Invalid bytes in the Data field in any data message, indicated by ChunkValid, and BE in write data and snoop response data, must be 0.

The DataL message format must be used instead of DataS when any of the following conditions are true:

- The BE field in a data response to a partial write request has one or more bits with zero value.
- The following fields are present in the message and have a non-zero value:
  - QoS
  - PBHA
  - RSVDC[N-1:16]

When the use of DataS is allowed, a Transmitter is permitted to use DataL message in place of DataS. The use of DataL instead of DataS may lead to reduced container packing efficiency.

Table B5.9 shows the message fields for DataS and DataL.

Table B5.9: DataS and DataL message fields

Common	Field name	DataS	DataL
	MsgType	4	4
	SharedCrdt	1	1
	RsvdZero	1	1
	ChunkValid	2	2
	TgtID	11	11
C8	SrcID HomeNID	11	11
	TxnID	12	12
	Opcode	4	4
	RespErr	2	2
	Resp	3	3
	DataSource	8	8
	DataPull	1	1

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.9 – *Continued from previous page*

Common	Field name	DataS	DataL
	CBusy	3	3
	CCID	2	2
	TagOp	2	2
	Tag	16	16
	TU	4	4
	TraceTag	1	1
	CAH	1	1
C9	{4'b0, DBID} MECID	16	16
	RSVDC[15:0]	16	16
	CacheLineID	6	6
	RsvdZero	1	1
	Data	512	0
	RsvdZero	0	32
	RSVDC[31:16]	0	16
	QoS	0	4
	PBHA	0	4
	MismatchedMECID	0	1
	RsvdZero	0	39
	BE	0	64
	Data	0	512
Total Non-data (bits)[a]		126	215
Total (bits)[a]		638	727

[a] The total bit count does not include the RsvdZero bits.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.6 WritePush message

Immediate Writes, Combined Immediate Writes with Cache Maintenance Operations or Persist Cache Maintenance Operations, Atomics, and DVMReq transactions with NonCopyBackWriteData data are permitted, but not required, to optimize their transaction flow by using WritePush semantics. A WritePush flow combines the request and data sent from the Transmitter to the Receiver into a single message, resulting in eliminating the need for a DBIDResp response.

The following interface properties are defined to support WritePush transactions:

WritePush_Support This property indicates if the interface supports the use of WritePush for Immediate Writes, their combined CMO variants, and Atomic transactions. See B14.4.2 *Receiver and Transmitter properties*.

DVMPush_Support This property indicates if the interface supports the use of WritePush for DVM transactions. See B14.4.2 *Receiver and Transmitter properties*.

See B5.6.2 *Message fields* for message format details.

B5.6.1 Transaction flow

Figure B5.4 shows the transaction flow for the different transaction types that can use WritePush semantics.

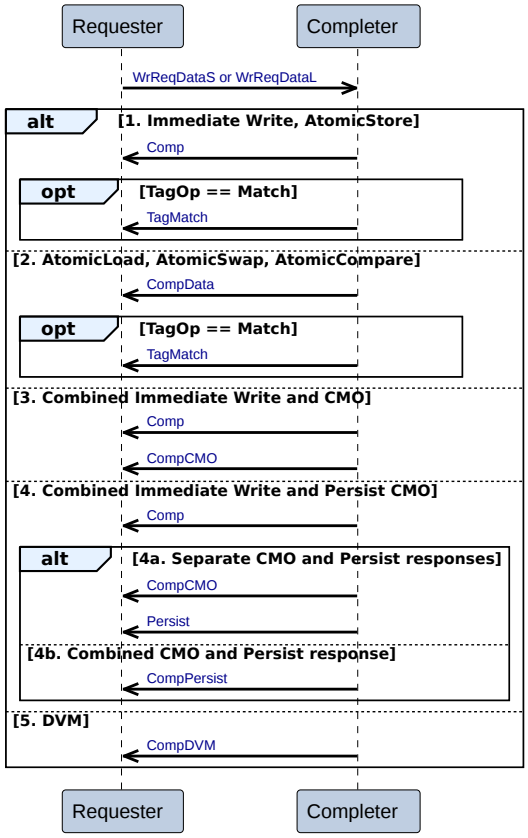


Figure B5.4: WritePush transaction flow

The Receiver of WrReqDataS and WrReqDataL is permitted, but not required, to send Comp before receiving the complete message. That is, the Receiver can process the request as soon as the message header is received and before data, including complete or none, is received.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.6.2 Message fields

Table B5.4 lists the request opcodes that are permitted to use a WrReqDataSmessage. In these requests, the BE field values must all be 1. All other push transactions, including those that require fields which are not included in a WrReqDataSmessage, must use a WrReqDataLmessage format.

In WrReqData* messages, invalid bytes in the Data field must be set to 0.

> **Note**
>
> The interface can use ChunkValid to determine which bytes to set to 0.

Table B5.10 shows the message fields in a WrReqDataS and WrReqDataLmessage.

The OWO field in Table B5.10 indicates whether the write is part of a group of writes that require OWO guarantees.

When OWO = 0 OWO guarantees are not required.

When OWO = 1 OWO guarantees are required.

When the rules permit the use of WrReqDataS, a WritePush message is permitted to use WrReqDataLinstead of WrReqDataS. The use of WrReqDataLmay lead to reduced container packing efficiency.

Table B5.10: WrReqDataS and WrReqDataL message fields

Common	Field name	Width (bits)	
		WrReqDataS	**WrReqDataL**
	MsgType	4	4
	SharedCrdt	1	1
	ResPlane	3	3
	QoS	4	4
	SrcID	11	11
	TxnID	12	12
	PAS[1:0]	2	2
	SecSID1	1	1
	Order	2	2
	MemAttr	4	4
	OWO	1	1
	TraceTag	1	1
	Addr[51:6]	46	46
	Opcode2	2	0
	Union1[a]	34	0
	Data	512	0
	Addr[5:4]	0	2
	SnpAttr[b]	0	1

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.10 – *Continued from previous page*

Common	Field name	Width (bits)	
		WrReqDataS	**WrReqDataL**
	MPAM[c]	0	15
	MECID	0	16
	RSVDC[15:0]	0	16
C11	Size[d] NumReq[2:0]	0	3
	Opcode	0	7
	TagOp[e]	0	2
C1	StashNIDValid Endian Deep PrefetchTgtHint	0	1
C2	Excl SnoopMe CAH	0	1
C3	DataTarget[0] StashNID[0]	0	1
	MultiReq	0	1
	RsvdZero	0	2
	PBHA	0	4
C12	Addr[3:0] {`1'b0`, NumReq[5:3]}	0	4
	StashLPIDValid	0	1
	StashLPID	0	5
C4	{4'`b0`, DataTarget[6:1]} StashNID[10:1]	0	10
C5	{3'`b0`, LPID} PGroupID StashGroupID TagGroupID	0	8
	RSVDC[23:16][f]	0	8
	Tag	0	16
	TU	0	4
	ChunkValid	0	2
	RespErr	0	2
	BE	0	64
	Data	0	512

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.10 – *Continued from previous page*

Common	Field name	Width (bits)	
		WrReqDataS	**WrReqDataL**
Total Non-data (bits)[g]		128	286
Total (bits)[g]		640	798

[a] This field is shared among MECID or StreamID, MPAM, and RSVDC. See Union1.

[b] For WrReqDataS, this field is derived from Opcode2.

[c] The most significant bit of the MPAM field encodes PMG. The two least significant bits encode MPAMSP[1:0]. The remaining bits encode PartID and are zero-extended, if required.

[d] For WrReqDataS, this field has 64 bytes implicitly.

[e] The TagOp value in the Request is the same as the TagOp value in the Data message.

[f] A maximum of 24 bits for the RSVDC field is permitted in WrReqDataS or WrReqDataL messages.

[g] The total bit count does not include the RsvdZero bits.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.7 WritePush data extension for Multi-request, WrDataExt

For support of WritePush in a Multi-request transaction, Table B5.11 shows the format of the data extension message, WrDataExt.

Table B5.11: WrDataExt message fields

Common	Field name	Width (bits)
	MsgType	4
	SharedCrdt	1
	ResPlane[a]	3
	RsvdZero	2
	Opcode3	2
	RespErr[1][b]	1
	Union2	49
	Union3	66
	Data	512
Total Non-data (bits)[c]		126
Total (bits)[c]		638

[a] The ResPlane field must have a constant value for all WrReqDataL and WrDataExt messages that are part of the same multi-request WritePush transaction.

[b] Compressed to OK or DERR.

[c] The total bit count does not include the RsvdZero bits.

Note

The WrDataExt message does not include a TxnID field. When multiple Resource Planes are supported, the ResPlane field allows a Receiver to associate WrDataExt messages to the correct multi-request WritePush transaction.

WrDataExt messages follow the same packing rules defined in B3.3.1 *Rules and restrictions*.

Additionally, the multiple messages that make up a multi-request WritePush transaction do not need to be tightly packed next to each other. For example, non-related Snoop or Request messages could be placed between WrReqDataL and WrDataExt messages in a container, or between two WrDataExt messages.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.8 Snoop message fields

Table B5.12 shows the fields in a Snoop message.

Table B5.12: Snoop message fields

Common	Field name	Width (bits)
	MsgType	4
	QoS	4
	TgtID	11
	SrcID	11
	TxnID	12
	PBHA	4
	StashLPIDValid	1
	StashLPID	5
	Opcode	5
	Addr	48
	PAS[1:0]	2
	DoNotGoToSD	1
	RetToSrc	1
	TraceTag	1
	MPAM	15
	MECID	16
	RsvdZero	19
Total (bits)[a]		141

[a] The total bit count does not include the RsvdZero bits.

Note

The 48-bit Addr field is populated with Addr[48:1] from the CHI SNP channel, which corresponds to the Addr[51:4] of the Physical Address (PA) [1]. In CHI, Addr[0] on the SNP channel is only used as part identifier for two-part DVM operations and not for general snoop processing [1].

For CHI C2C, Addr[0] does not need to be transported as DVM operations are not propagated on the SNP channel.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.9 Uncredited Miscellaneous message fields

The fields in a MiscU message are shown in Table B5.13.

The number of Payload bits used by a MiscU message is dependent on the MISC message type.

The following key is used:

X Variable field width, where the precise value is dependent on the MiscOp field value.

Table B5.13: MiscU message fields

Field name	Width (bits)
MsgType	4
MiscOp	4
Payload	X
Total (bits)	8 + X

MiscOp describes the message subtypes supported by Uncredited Miscellaneous message class.

The encoding of MiscOp field in a MiscU message is shown in Table B5.14.

Table B5.14: MiscOp field encodings in a MiscU message

MiscOp	MiscU message
`0b0000`	Reserved
`0b0001`	Reserved
`0b0010`	Activation
`0b0011`	Connect
`0b0100`	CrdtGrant
`0b0101`	Properties
`0b0110`	LinkStatus
`0b0111`	Event
`0b1000`	IDEMsg
Others	Reserved

B5.9.1 Activation message format

Table B5.15 shows the Activation MiscU message fields.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.15: Activation message fields

Field name	Width (bits)	Value
MsgType	4	0b0000
MiscOp	4	0b0010
ActivationOp	4	
PropertyReq	1	0 or 1[a]
RsvdZero	19	
Total (bits)	32	

[a] When ActivationOp == ActivateReq, else inapplicable and must be 0.

Table B5.16 show the encoding of ActivationOp in the Activation message.

Table B5.16: ActivationOp field encodings in MiscU.Activation messages

ActivationOp	Message
0b0000	ActivateReq
0b0001	ActivateAck
0010	DeactivateReq
0011	DeactivateAck
0100	DeactivateHint
Others	Reserved

B5.9.1.1 ActivateReq and ActivateAck

The ActivateReq and ActivateAck message pair is used to move the interface from the STOP state to the RUN state.

An ActivateReq message can optionally request a property exchange on completion of interface activation. See B5.9.1.4 *PropertyReq* for more information.

See STOP and ACTIVATE state entries in B12.2.4 *Activation and Deactivation flow* for when ActivateReq and ActivateAck messages are sent and the interface response on receiving them.

B5.9.1.2 DeactivateReq and DeactivateAck

The DeactivateReq and DeactivateAck message pair is used to move the interface from the RUN state to the STOP state.

B5.9.1.3 DeactivateHint

DeactivateHint is a message informing the remote interface about the readiness of the Transmitter to deactivate the interface. See B12.2 *Interface activation and deactivation* for details on activation messages.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.9.1.4 PropertyReq

The PropertyReq field value determines whether a property exchange is required at the completion of interface activation.

Table B5.17 shows the PropertyReq field encodings.

Table B5.17: PropertyReq field encodings

PropertyReq	Description
`0b0`	Properties must not be exchanged.
`0b1`	Property messages must be sent after ActivateAck is sent and received.

This field is applicable when ActivationOp is ActivateReq.

This field is inapplicable and must be zero for other Activation message types.

If either the sent or the received, or both, ActivateReq message has PropertyReq = 1, property exchange is required. That is, both sides must send Properties messages.

If both the sent and received ActivateReq messages have PropertyReq = 0, Properties messages must not be sent by either side.

See B14.1 *Introduction* for an overview of the Property exchange process.

B5.9.2 Connect message format

The Connect MiscU message is used to connect and disconnect an interface from coherency and DVM. See B12.3 *Coherency connect and disconnect* and B12.4 *DVM connect and disconnect* on how these messages are used.

A Transmitter is only permitted to use Connect messages when the interface is in the RUN state.

Table B5.18 shows the Connect message fields.

Table B5.18: Connect message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b0011`
ConnectOp	4	
RsvdZero	20	
Total (bits)	32	

Table B5.19 shows the encoding of ConnectOp field in Connect messages.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.19: ConnectOp field encodings in MiscU.Connect messages

ConnectOp	Message
0000	CohConnectReq
0001	CohConnectAck
0010	CohDisconnectReq
0011	CohDisconnectAck
0100	DVMConnectReq
0101	DVMConnectAck
0110	DVMDisconnectReq
0111	DVMDisconnectAck
Others	Reserved

B5.9.3 CrdtGrant message format

The CrdtGrant MiscU message is used for granting credits for messages.

Table B5.20 shows the CrdtGrant message fields.

Table B5.20: CrdtGrant message fields

Field name	Width (bits)	Value
MsgType	4	0b0000
MiscOp	4	0b0100
REQShCredit	3	
RSPCredit	3	
DATShCredit	3	
SNPCredit	3	
MISCCredit	3	
REQ0Credit	3	
REQ1Credit	3	
REQ2Credit	3	
REQ3Credit	3	
REQ4Credit	3	
REQ5Credit	3	
REQ6Credit	3	
REQ7Credit	3	
DAT0Credit[a]	3	

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.20 – *Continued from previous page*

Field name	Width (bits)	Value
DAT1Credit[a]	3	
RsvdZero	27	
Total (bits)	80	

[a] When WritePush_Support and DVMPush_Support are both False, this field is inapplicable and must be zero.

B5.9.4 Properties message format

The MiscU.Properties message is used by each Protocol layer to exchange its advertised property values with the remote Protocol layer.

A sequence of multiple Properties messages can be exchanged, if both connected interfaces support this, as indicated by the value of the negotiated Num_Properties_Msg property. See B14.1 *Introduction* for more detail.

In this specification, the Properties messages in the sequence will be individually referred to as MiscU.Properties(N), where N is between 1 and the negotiated Num_Properties_Msg value.

Table B5.21 shows the Properties field encodings. For more information on the payload of each MiscU.Properties(N) message, see B14.5 *Property packing in MiscU.Properties messages*.

Table B5.21: Properties message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b0101`
Payload	152	
Total (bits)	160	

B5.9.5 LinkStatus message format

The LinkStatus MiscU message is used by the Link layer to inform the Protocol layer the completion of Link layer initialization and change in power status of the link. This message is sent during the interface initialization as described in B13.2 *Initialization flow* and the Link layer is permitted to send whenever the physical link power state changes.

Table B5.22 shows the LinkStatus message fields.

Table B5.22: LinkStatus message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b0110`

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.22 – *Continued from previous page*

Field name	Width (bits)	Value
FlitFormat	3	
LinkPowerState	3	
RsvdZero	18	
Total	32	

B5.9.5.1 FlitFormat

The FlitFormat field in the LinkStatus message is used by the Link layer to inform the Protocol layer of how many bytes are required to populate the Link layer specific fields, such as link header, FEC, and CRC.

Table B5.23 shows the mapping of the FlitFormat values to the Protocol layer container format.

Table B5.23: Encoding of FlitFormat in LinkStatus message

FlitFormat[2:0] value	Bytes required	Compatibility
`000`	Reserved	
`001`	6	Format X, Format Y, and Format Z.
`010`	20	Format Y.
`011`	16	Format Z.
Others	Reserved	

Note

The FlitFormat field is expected to be reflected in the C2C_PortCtrlStatus1.LinkLayerFlitFormat register field.

B5.9.5.2 LinkPowerState

The LinkPowerState field in the LinkStatus message allows the Link layer to inform the protocol of the power state of the link. This information is sent to the Protocol layer during link initialization and also whenever there is a change in link power state.

Table B5.24 shows the LinkPowerState message fields.

Table B5.24: LinkPowerState field encodings in a LinkStatus message

LinkPowerState	Power state
`000`	Disabled
`001`	Active
`100`	Reset

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.24 – *Continued from previous page*

LinkPowerState	Power state
Others	Reserved

Note

The Protocol layer upon receiving LinkStatus message with Reset may be required to reset some protocol configuration settings.

B5.9.6 Event message format

The Event MiscU message is used for exchanging different types of events between a Transmitter and a Receiver connected by a C2C link.

Table B5.25 shows the Event message fields.

Table B5.25: Event message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b0111`
UseReqAck	1	
EventOp	1	
EventType	4	
RsvdZero	18	
Total	32	

When multiple Logical Links are supported, a Transmitter is permitted to send any Event message across any Logical Link.

When Unpack Multihop routing is used, it is not possible to use the Event message to exchange events between non-neighboring chips as the message format does not include routing information.

If the specific event type requires routing between two chips that are only connected through an Unpack Multihop routing path, an IMPLEMENTATION DEFINED event broadcasting mechanism is required.

Support for the Event message is controlled by the Event_Support_Rx and Event_Support_Tx properties.

B5.9.6.1 UseReqAck

The UseReqAck field in the Event message can be used by a Transmitter to request that the Receiver acknowledges receipt of the event.

Table B5.26 shows the mapping of the UseReqAck values in the Event message.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.26: UseReqAck field encodings in an Event message

UseReqAck	Description
`0b0`	When EventOp is EventReq, the Receiver does not send an EventAck response.
`0b1`	When EventOp is EventReq, the Receiver must send an EventAck in a timely manner.

B5.9.6.2 EventOp

The EventOp field in the Event message indicates whether the message is an Event request or an Event acknowledgement.

Table B5.27 shows the mapping of the EventOp values in the Event message.

Table B5.27: Encoding of EventOp in Event message

EventOp	Description
`0b0`	EventReq. This is an Event request.
`0b1`	EventAck. This is an Event acknowledgement.

When the UseReqAck field is 0, the EventOp field must be 0.

An Event message with the EventOp field set to EventAck is required to have the UseReqAck field set to 1.

An Event message with the EventOp field set to EventAck is required to only be sent in response to an Event message with the EventOp field set to EventReq and UseReqAck set to 1.

For each EventType, a Transmitter is only permitted to have one outstanding EventReq that requires an acknowledgment, indicated by UseReqAck set to 1. The Transmitter:

- Must wait for the matching EventAck before sending another EventReq of the same EventType with UseReqAck set to 1.
- Is permitted to send additional EventReq of the same EventType with UseReqAck set to 0, before an EventAck response is observed. This use case is expected to be rare.

B5.9.6.3 EventType

The EventType field in the Event message indicates the type of event being transmitted.

Table B5.28 shows the mapping of the EventType values in the Event message.

Table B5.28: EventType field encodings in an Event message

EventType	Description
`0b0000`	This is a CPU event.
Others	Reserved.

The EventType field of an Event message with the EventOp field set to EventAck is required to match the EventType value specified in the associated Event message with the EventOp field set to EventReq.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.9.7 IDEMsg message format

The IDEMsg MiscU message is used to start, operate, and terminate an IDE session across a C2C interface.

There are two groups for IDE messages:

- Local. Local IDE messages are exchanged between the egress container selection logic, within the Packetization layer, and the IDE engine within the Link layer on the same chip. Some local IDE messages cause a remote IDE message to be exchanged between IDE engines. Some other local IDE messages terminate at the local IDE engine and do not result in a remote IDE message being exchanged across the link. Since local IDE messages are only exchanged between components that are contained within the same chip and therefore do not directly affect chip interoperability, their use in an implementation is OPTIONAL.
- Remote. These are exchanged between two Link layer IDE enginers across the C2C link. The definition of such messages follows the chosen Link layer IDE architecture protocol, and is outside the scope of this specification.

IDE messages are only permitted to be placed in granule G0 of a C2C container. This applies to local and remote IDE messages.

If the IDE message in a container is:

- IDE.MAC, the container can include other protocol messages in the other granules.
- Any other IDE message, the container must not include any other protocol messages.

Table B5.29 shows the IDEMsg message fields.

Table B5.29: IDEMsg message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b1000`
IDEOp	4	
Payload	X	
Total	12 + X	

Table B5.30 shows the mapping of the IDEOp values in the IDE message.

Table B5.30: Encoding of IDEOp in IDE message

IDEOp	Description
`0b0001`	IDE.MAC
`0b0010`	IDE.SetupReq
`0b0011`	IDE.SetupAck
`0b0100`	IDE.HaltReq
`0b0101`	IDE.HaltAck
`0b0110`	IDE.Pause
Others	Reserved

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.9.7.1 IDE.MAC

The IDE.MAC message is sent from the egress container selection logic within the Packetization layer to the local IDE engine to indicate that a MAC must be computed and placed in granule G0 of the container. The IDE.MAC message value causes an equivalent remote IDE message being sent as part of a transmitted container, after including the computed MAC in G0.

Table B5.31 shows the IDE.MAC message field encodings.

Table B5.31: IDE.MAC message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b1000`
IDEOp	4	`0b0001`
RsvdZero	4	
Payload[a]	96	
Total (bits)	112	

[a] Must be all zeroes.

B5.9.7.2 IDE.SetupReq

The IDE.SetupReq message is sent by the local Link layer IDE engine to the egress container selection logic within the Packetization layer to request that a new IDE session starts.

> **Note**
>
> It is expected tht this message will be sent by the Link layer after software has programmed the required IDE engine registers to set up the IDE session.

Optionally, the IDE.SetupReq message can be used by the local IDE engine upon an IDE key refresh operation, to request that the egress container selection logic temporarily stops sending containers.

> **Note**
>
> For example, the IDE.SetupReq message can be used to avoid backpressure from the Link layer, when this does not include sufficient container buffering resources.

Table B5.32 shows the IDE.SetupReq local message.

Table B5.32: IDE.SetupReq message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b1000`

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.32 – *Continued from previous page*

Field name	Width (bits)	Value
IDEOp	4	0b0010
EpochSize	4	
MACInsertionDelay	4	
KeyRefreshDelay	32	
RsvdZero	12	
Total (bits)	64	

B5.9.7.2.1 EpochSize

The EpochSize field in the IDE.SetupReq message indicates the IDE epoch size for the IDE session, measured in the number of containers. The EpochSize field is only valid at the start of the IDE session.

Table B5.33 shows the mapping of the EpochSize values in the IDE.SetupReq message.

Table B5.33: Encoding of EpochSize in IDE.SetupReq message

EpochSize	Description
0b0000	Reserved.
0b0001	2 containers.
Others	Reserved.

B5.9.7.2.2 MACInsertionDelay

The MACInsertionDelay field in the IDE.SetupReq message indicates the required MAC insertion delay for the IDE session, measured in the number of containers. The MACInsertionDelay field is only valid at the start of the IDE session.

Table B5.34 shows the mapping of the MACInsertionDelay values in the IDE.SetupReq message.

Table B5.34: Encoding of MACInsertionDelay in IDE.SetupReq message

MACInsertionDelay	Description
0b0000	1 container.
0b0001	2 containers.
Others	Reserved.

B5.9.7.2.3 KeyRefreshDelay

The KeyRefreshDelay field in the IDE.SetupReq message indicates a hint to the egress container selection logic. The KeyRefreshDelay is measured in the number of containers and is naturally encoded.

The KeyRefreshDelay field indicates that it may be beneficial to wait for the specified number of containers after receiving the IDE.SetupReq message, before sending the first container to the local IDE engine.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B5.9.7.3 IDE.SetupAck

The IDE.SetupAck message acknowledges that an IDE session can begin and is sent by the egress container selection logic within the Packetization layer. Containers sent after the IDE.SetupAck message:

- Start a new IDE epoch.
- Are required to be encrypted and integrity protected.
- Includes at least one Reserved G0 granule per IDE epoch in the container indicated by the MACInsertionDelay field.

Optionally, after sending the IDE.SetupAck message, the container selection logic may decide to delay sending protocol containers for a time determined by the KeyRefreshDelay field.

Table B5.35 shows the IDE.SetupAck local message.

Table B5.35: IDE.SetupAck message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b1000`
IDEOp	4	`0b0011`
RsvdZero	20	
Total (bits)	32	

B5.9.7.4 IDE.HaltReq

The IDE.HaltReq message indicates the IDE session in progress is going to be stopped. The IDE.HaltReq message is sent by the local Link layer IDE engine to the egress container selection logic within the Packetization layer. The local Link layer is only permitted to send this message when an IDE session is ongoing.

> **Note**
>
> It is expected that this message is sent by the Link layer after the software has programmed the required registers to terminate the IDE session.

Upon receipt of the IDE.HaltReq message, the container selection logic is no longer required to include at least one Reserved G0 granule per IDE epoch.

Table B5.36 shows the IDE.HaltReq local message.

Table B5.36: IDE.HaltReq message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b1000`
IDEOp	4	`0b0100`

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.36 – *Continued from previous page*

Field name	Width (bits)	Value
RsvdZero	20	
Total (bits)	32	

B5.9.7.5 IDE.HaltAck

The IDE.HaltAck message indicates that the IDE session has terminated. The IDE.HaltAck message is sent by the egress container selection logic within the Packetization layer. The containers sent after the IDE.HaltReq message are not required to be encrypted or integrity protected.

Table B5.36 shows the IDE.HaltAck local message.

Table B5.37: IDE.HaltAck message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b1000`
IDEOp	4	`0b0101`
RsvdZero	20	
Total (bits)	32	

B5.9.7.6 IDE.Pause

The IDE.Pause message signals the protocol idleness from the egress container selection logic within the Packetization layer to the Link layer IDE engine. The response of the IDE engine to the IDE.Pause message is outside the scope of this specification.

After the protocol idleness behavior has been detected and before the IDE.Pause message is sent, the container selection logic must send an IDE epoch size of empty containers. This allows space for the necessary MAC to be inserted for release of all prior non-empty containers on the remote side.

The container selection logic can resume protocol traffic by sending a new non-empty protocol container. The non-empty protocol container signals the start of a new IDE epoch.

Table B5.38 shows the IDE.Pause local message.

Table B5.38: IDE.Pause message fields

Field name	Width (bits)	Value
MsgType	4	`0b0000`
MiscOp	4	`0b1000`
IDEOp	4	`0b0110`

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B5.38 – *Continued from previous page*

Field name	Width (bits)	Value
RsvdZero	20	
Total (bits)	32	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B6
Node ID management

This chapter describes the management of node identifiers for each C2C agent. It contains the following sections:

- B6.1 *Introduction*
- B6.2 *ChipID*
- B6.3 *AgentID*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B6.1 Introduction

Each C2C agent is required to have a node identifier, NodeID. This allows for the following:

- Identification of Requesters and the routing of related messages to the appropriate C2C agents, including Responses, Data, and Snoops.
- Coherency management to track nodes that had previously coherently cached a copy of the line.
- Efficient transaction ordering.
- Identification of a stash target in Stash transactions.

A NodeID is required to be assigned to the following components:

- Request Node.
- Home Node.
- Subordinate Node.
- DVM Node.

A NodeID is 11 bits. Figure B6.1 shows the structure of a NodeID field with:

- N-bit width for a ChipID field, where N can be any value between 0 and 6.
- (11 - N) - bit width for an AgentID field.

Figure B6.1: NodeID field structure

Note

For on-chip usage, a chip can re-map the C2C NodeID values to the local NodeID values in an IMPLEMENTATION DEFINED manner.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B6.2 ChipID

The ChipID field is a unique value assigned to a chip. The ChipID field is embedded within the higher-order bits of the 11-bit NodeID of a C2C agent.

A chip can be assigned multiple ChipID values. When a chip is assigned multiple ChipID values, the values must be contiguous and not interleave with the range of values assigned to any other chip. The range of values assigned are stored in the C2C_ChipIDCtrl.StartChipID and C2C_ChipIDCtrl.EndChipID register fields.

The ChipID field width is permitted to be 0 to 6 bits. The number of bits used for the ChipID field is stored in the NumChipIDBits field.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B6.3 AgentID

Each Request Node, Home Node, Subordinate Node, and DVM Node on a chip must be assigned an AgentID value.

AgentID values assigned to protocol agents on each chip have the following conditions that are required:

- The AgentID value must be unique within each node type.
- Different node types on a chip can share the same AgentID value.
  However, the Subordinate Node and Home Node must use different AgentID values when DWT is supported locally. This permits the write data from the remote chip to be routed to the correct target node type on the Completer chip.

Note

It is expected that the same C2C link will not be used to transport Request Node to Home Node traffic and Home Node to Subordinate Node traffic at the same time in the same direction. When this expectation is not met, IMPLEMENTATION DEFINED mechanisms must exist to be able to distinguish the two types of traffic, and to guarantee independent progress where required.

B6.3.1 Request Node enhanced NodeID assignment

AgentID values can be assigned to Request Nodes so that Caching Request Nodes can be easily distinguished from Non-caching Request Nodes. This improves the efficiency associated with any ID-based structures at a Receiver that handles messages from the two types of Request Nodes in a different way. For example, ID re-mapping tables or snoop filters.

An optional mechanism is permitted for the Transmitter chip to indicate that Caching RN and Non-caching RN AgentID values are assigned in order to distinguish them by a specific NodeID bit, NodeID[X]. When this mechanism is used:

- The C2C_ChipCap.OverlappedID field is set to 1.
- X is defined to be `X = ceil(log2(max(NumCachingRN, NumNonCachingRN)))`.
- NodeID[X]=0 identifies a Caching Request Node.
- NodeID[X]=1 identifies a Non-Caching Request Node.
- When X is greater than 0, the assignment of NodeID[X-1:0] for the two types of Request Nodes must be zero-compacted. That is, from 0 to (NumCachingRN - 1) for Caching Request Nodes, and from 0 to (NumNonCachingRN - 1) for Non-caching Request Nodes.

When this mechanism is not used, it is required that:

- The C2C_ChipCap.OverlappedID field is set to 0.
- AgentID assignment for Caching Request Nodes is zero-compacted.
- AgentID assignment for Non-Caching Request Nodes is from NumCachingRN to (NumCachingRN+NumNonCachingRN - 1).

Note

A Receiver chip is permitted to ignore use of this mechanism and to consider all incoming NodeID bits.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B7
Message routing

This chapter describes message routing and the supporting software-visible structures. It contains the following sections:

- B7.1 *Introduction*
- B7.2 *Message routing structures*
- B7.3 *Request and response routing*
- B7.4 *Snoop request routing*
- B7.5 *Multihop routing*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B7.1 Introduction

Messages are routed by one of the following:

- Address decoding.
- Using the message TgtID field.
- A combination of address decoding and TgtID field. This is only applicable in Snoop transactions. For more details on Snoop transactions, see B7.4 *Snoop request routing*.

Table B7.1 shows the message routing methods permitted for a C2C interface. The following key is used:

Y Yes, permitted.

N No, not permitted.

Table B7.1: Message routing methods

Message type	Address decode	TgtID
Request	Y	N
Response	N	Y
Data	N	Y
Snoop	Y	Y

Miscellaneous message types are different and are typically sent only between C2C interfaces.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B7.2 Message routing structures

To support message routing, the following hardware structures are defined:

- An Address decoder, with the following types:
  - Chip
  - *Port Aggregation Group* (PAG)
- An ID-based Routing Table.

B7.2.1 Chip address decoder

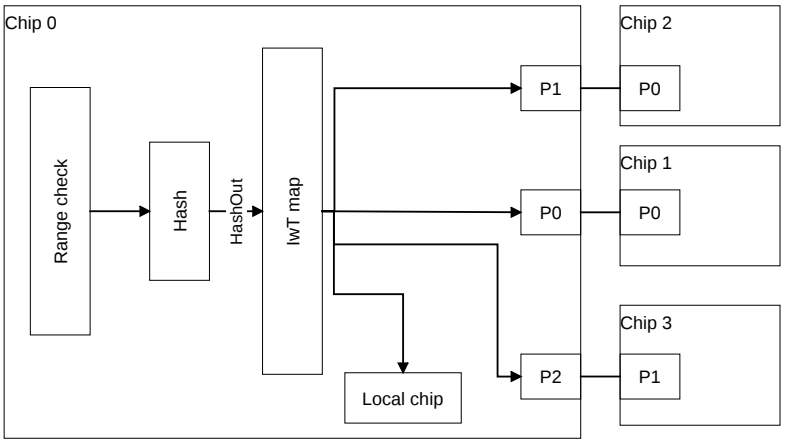
An address decoder maps an address range to at least one target.

A memory map may use multiple address decoders, each covering a different address range, using high-order address bits to select among different sets of targets. The ranges of address decoders must not overlap.

For each address decoder, the complete address range may map to a single target or may be interleaved across multiple targets.

The output of an address decoder can be a port, including a logical link if applicable, or a PAG. Port Aggregation Groups provide a second-level address decoding to interleave across a set of aggregated ports. See B7.2.2 *Port Aggregation Group address decoder* for more details.

Figure B7.1 shows the structure of a typical interleaved address decoder.



Px = Port x, where x represents a number

Figure B7.1: Structure of a typical interleaved address decoder

The request address flows through the decoder in the following steps to obtain the target value:

1. The address of the request is checked against the range of all the available address decoders to determine which address decoder to use. The decoder that matches the address is used for further processing the request. Only one address decoder will be used for any given address.
2. The address is put through a hash function with programmed interleave way. A hash function with an interleave way of 1, 2, 4, 8, or 16 is required to be supported, along with an optional three-way interleave. The hash logic is not applicable in a single-target address decoder. The default HashOut value for a single-target address decoder is 0.
3. In an interleaved address decoder, the HashOut result is used to look up the entry in the *Interleave way to Target* (IwT) table. The IwT is required to include at least one entry. The target value in an entry can be:
   - A Port with a Logical Link number.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- A PAG.
- A local target. The Target value must be `0xFF`. The local chip may further use proprietary local address decoder structures to target any of the local agents.

The number of applicable IwT entries are limited to the hash function interleave ways.

Multiple table entries can have the same target value. The number of entries per target can be different.

In a single-target address decoder, only entry [0] of the IwT table is applicable. All other entries are inapplicable.

Figure B7.2 shows an example IwT mapping for a 16-way interleaved address decoder with four targets, where one target is local and three targets are Port Aggregation Groups, that is, PAG0, PAG1, and PAG2. In Figure B7.2, the distribution of address ranges among the targets is uneven. The local to remote ratio is 6:10, where the remote is further unevenly distributed.

63 … 48 47	32 31 … 16 15		0	
Local	PAG1	PAG0	Local	Entry 0
PAG0	PAG2	Local	PAG0	Entry 4
PAG1	Local	PAG0	PAG2	Entry 8
Local	PAG0	Local	PAG0	Entry 12

Figure B7.2: Example of an IwT mapping for an interleaved address decoder with four targets

Figure B7.3 shows an IwT mapping with a single-target decoder. The target is Port1 and LogLink0, where all other values are inapplicable and reserved.

63 … 48 47	32 31 … 16 15		0	
Reserved	Reserved	Reserved	Port1 LogLink0	Entry 0
Reserved	Reserved	Reserved	Reserved	Entry 4
Reserved	Reserved	Reserved	Reserved	Entry 8
Reserved	Reserved	Reserved	Reserved	Entry 12

Figure B7.3: Example of an IwT mapping for a single-target address decoder

B7.2.2 Port Aggregation Group address decoder

Message transport between two chips can use multiple ports. The multiple ports can be aggregated together, with the message traffic distributed across the ports.

The second-level decoding using a PAG address decoder is optional. A PAG address decoder provides an additional layer of port aggregation. However, interleaving across aggregated ports can be achieved solely by use of the address decoder if a PAG address decoder is not used.

The characteristics of PAG address decoders are:

- A PAG address decoder can be used for routing either Request or Snoop messages. For more details, see B7.3 *Request and response routing* and B7.4 *Snoop request routing*.
- The permitted number of PAG address decoders per chip is 0 to 64.
- A PAG address decoder does not use an address range, as the range is implicit.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- For a typical PAG address decoder, the implied address range is:
  - For a Request, similar to the structure shown in Figure B7.1, the address range in the address decoder that targets the PAG address decoder. See B7.3 *Request and response routing*.
  - For a Snoop, all address ranges supported by the Home that is source of the snoop. See B7.4 *Snoop request routing*.
- A hash function with an interleave way of 1, 2, 4, 8, or 16 is required to be supported along with an optional three-way interleave.
- A 16-entry *Interleave way to Port* (IwP) table is defined. The programming of the IwP table is similar to the programming of the address decoder IwT table described in B7.2.1 *Chip address decoder*, with the exception that IwP table entries are only allowed to target a port number and a Logical Link number.

B7.2.3 ID Route Table

The ID Route Table can assist in routing messages using the *Target ID* (TgtID). When C2C_ChipIDRouteTableCap.NumIDRouteEntries is a non-zero value, this indicates ID Route Table support for an implementation.

Entries are indexed by a slice of the TgtID, representing the ChipID. The table lookup determines whether the route is local or remote. Specifically, the table lookup can result in:

- A port number, and Logical Link number.
- A PAG number.
- A local target, indicated by the value `0xFF`.

The ID Route Table can be used at various stages of the message routing depending on message type, and depending on whether port aggregation or Multihop routing is supported. Namely, it can be used at the on-chip source node, at ingress ports and at egress ports.

The ID Route Table can also be used at ingress ports for routing containers, when Pass-through Multihop routing is supported. For more details, see B7.5.7 *Pass-through Multihop routing*.

For credit management purposes, the ID Route Table may be looked up at the ingress port, where the container is unpacked, using the container ChipSrcID to retrieve the Logical Link number for the incoming container. See B7.5.2 *Pass-through Multihop routing* for more information.

Note

Alternatively, other mechanisms that do not use an ID Route Table can be used at the unpacker of a container, to associate the ChipSrcID to the appropriate Logical Link.

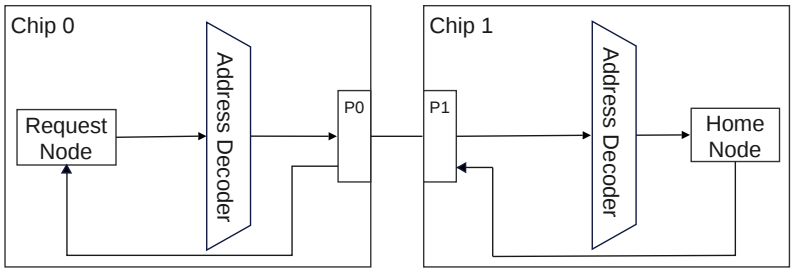
Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B7.3 Request and response routing

This section describes routing of request and response messages using C2C routing structures. Different example topologies are considered, including the use of port aggregation.

B7.3.1 Single-hop system, no port aggregation

In Figure B7.4, a request is routed from an Request Node in Chip 0 to a Home Node in Chip 1, and a response is routed on the reverse path. In Figure B7.4, the message path consists of a single hop, and the two chips are connected by a single link.



Px = Port x, where x represents a number

Figure B7.4: Request and response routing, single-hop path, no port aggregation

The following steps are taken to perform the routing in Figure B7.4:

1. In Chip 0:
   a. At the source Request Node, the request address is range-checked against all available Address Decoders. The matching Address Decoder is used to determine the correct egress port. In Figure B7.4, this is P0. A Logical Link number will also be determined, if applicable.
   b. The Request message is routed to the determined egress port, P0, in an IMPLEMENTATION DEFINED manner. At the egress port, the message is packed into a container and transmitted to Chip 1.
2. In Chip 1:
   a. At the ingress port, the received container is unpacked into individual messages. The Request message address is compared against available Address Decoders to determine if the request is destined to a local agent or if it must be forwarded to an egress port. In Figure B7.4, the request is destined to a local agent.
   b. The Request message is routed to its local destination in an IMPLEMENTATION DEFINED manner. The local routing must include information to identify the used ingress port and the associated Logical Link number, if applicable.
   c. At the Home Node, the Response message is routed to the port identified in step (2b). In Figure B7.4, this is P1. At the egress port, the Response message is packed into a container and transmitted to Chip 0.
3. In Chip 0:
   a. At the ingress port, the received container is unpacked into individual messages. The Response message is determined to be destined for a local agent by inspecting its ChipID or by equivalent IMPLEMENTATION DEFINED means.
   b. The Response message is routed to its local destination in an IMPLEMENTATION DEFINED manner.

B7.3.2 Single-hop system with port aggregation

In Figure B7.5, a request is routed from an Request Node in Chip 0 to a Home Node in Chip 1, and a response is routed on the reverse path. In Figure B7.5, the message path consists of a single hop, and the two chips are

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

connected by multiple links using port aggregation.

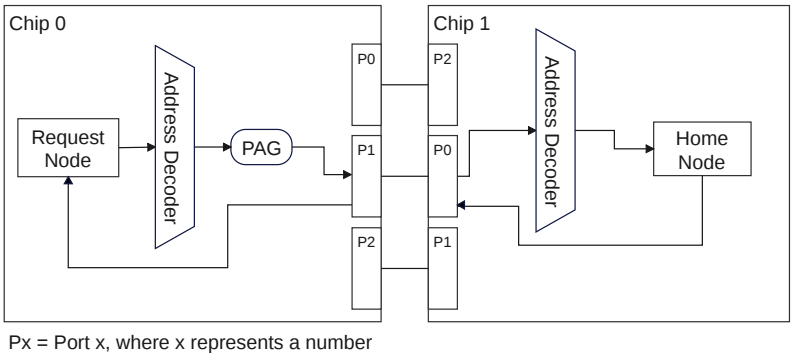


Figure B7.5: Request and response routing, single-hop path with port aggregation

The following steps are taken to perform the routing in Figure B7.5:

1. In Chip 0:
   a. At the source Request Node, the request address is range-checked against all available Address Decoders. In Figure B7.5, the output of the selected Address Decoder is a PAG number.
   b. The selected PAG decoder is used to identify the egress port to which the Request message will be sent. In Figure B7.5, this is P1. A Logical Link number will also be identified, if applicable.
   c. The Request message is routed to the determined egress port, P1, in an IMPLEMENTATION DEFINED manner. At the egress port, the message is packed into a container and transmitted to Chip 1.
2. In Chip 1:
   a. At the ingress port, the received container is unpacked into individual messages. The Request message address is compared against available Address Decoders to determine if the request is destined to a local agent or if it must be forwarded to an egress port. In this example, the request is destined to a local agent.
   b. The Request message is routed to its local destination in an IMPLEMENTATION DEFINED manner. The local routing must include information to identify the used ingress port within the set of aggregated ports and the associated Logical Link number, if applicable. This is to allow routing of the response back to the same port.
   c. At the Home Node, the Response message is routed to the port identified in step (2b). In Figure B7.5, this is P0. At the egress port, the Response message is packed into a container and transmitted to Chip 0.
3. In Chip 0:
   a. At the ingress port, the received container is unpacked into individual messages. The Response message is determined to be destined for a local agent by inspecting its ChipID or by equivalent IMPLEMENTATION DEFINED means.
   b. The Response message is routed to its local destination in an IMPLEMENTATION DEFINED manner.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B7.4 Snoop request routing

It is required that the requests issued by a Request Node, and snoops to the same address, target the Request Node using the same port.

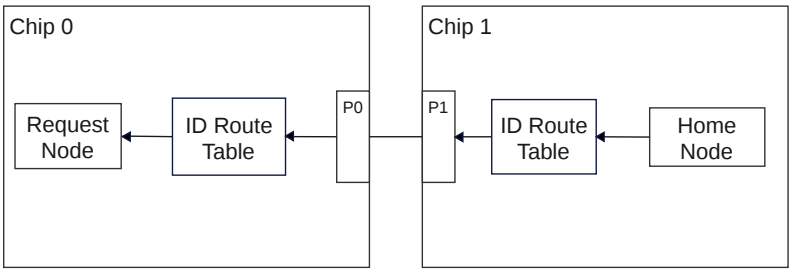
When requests and snoops use PAG address decoders between a pair of chips connected through aggregated ports, the address hash function used by the PAG address decoders for:

- Allocating requests from one chip must be the same hash function for all PAG address decoders for a given target Home.
- Snoops in the opposite direction from a Home must be a single-hash function. The hash function must also be the same one used by requests to that address range.

This section describes routing of snoop messages using C2C routing structures. Different example topologies are considered, including use of port aggregation.

B7.4.1 Single-hop system, no port aggregation

In Figure B7.6, a snoop message is routed from a Home Node in Chip 1 to a Request Node in Chip 0. In Figure B7.6, the message path consists of a single hop, and the two chips are connected by a single link.



Px = Port x, where x represents a number

Figure B7.6: Snoop routing, single-hop path, no port aggregation

The following steps are taken to perform the routing:

1. In Chip 1:
   - At the source Home Node, the egress port to use can be identified by looking up an ID Route Table using the source ChipID of the original allocating Request message.
   - Alternatively, the routing information can also be obtained by the Home Node by tracking the port on which the allocating request was received, and the associated Logical Link number to use, if applicable.
   - The snoop message is routed to the determined port, that is, P1.
   - At the egress port, the message is packed into a container and transmitted to Chip 0.
2. In Chip 0:
   - At the ingress port, the received container is unpacked into individual messages. The snoop message is determined to be destined for a local agent by looking up an ID Route Table with the ChipID included in the message, or by equivalent IMPLEMENTATION DEFINED means.
   - The snoop message is routed to its local destination in an IMPLEMENTATION DEFINED manner.

B7.4.2 Single-hop system with port aggregation

In Figure B7.7, a snoop message is routed from a Home Node in Chip 1 to a Request Node in Chip 0. In Figure B7.7, the message path consists of a single hop, and the two chips are connected by multiple links using port

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

aggregation.

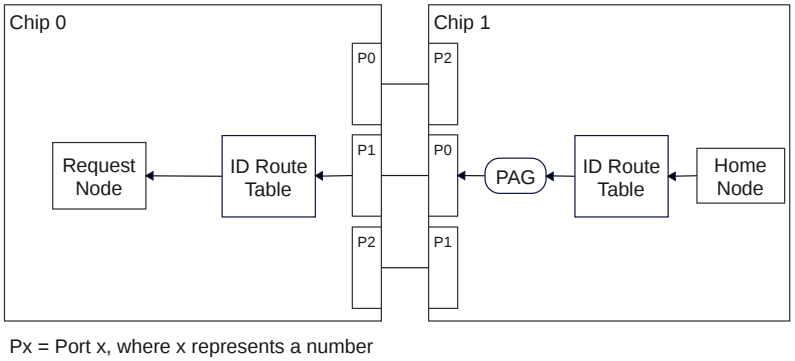


Figure B7.7: Snoop routing, single-hop path with port aggregation

The following steps are taken to perform the routing:

1. In Chip 1:
   - At the source Home Node, a PAG number can be identified by looking up an ID Route Table using the source ChipID included in the original allocating Request message, as shown in Figure B7.7.
   - The selected PAG decoder is used with the message address to identify the egress port to which the request message will be sent, that is, P0. A Logical Link number will also be returned, if applicable.
   - Alternatively, if snoop PAG hashing is not used, the port number can be directly obtained by the Home Node by tracking the port on which the allocating request was originally received, and the associated Logical Link number to use, if applicable.
   - The snoop message is routed to the determined port, that is, P0.
   - At the egress port, the message is packed inside a container and transmitted to Chip 0.
2. In Chip 0:
   - At the ingress port, the received container is unpacked into individual messages. The snoop message is determined to be destined for a local agent by looking up an ID Route Table with the ChipID included in the message, or by equivalent IMPLEMENTATION DEFINED means.
   - The snoop message is routed to its local destination in an IMPLEMENTATION DEFINED manner.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B7.5 Multihop routing

Multihop routing allows a message to be routed to a chip that is not physically directly connected to the source chip. The ability to route messages through one or more intermediate chips is required in topologies that are not fully connected. The container header includes fields to support Multihop routing. See B3.2 *Protocol header* for more information.

The intermediate chips must guarantee forward progress of the Multihop messages passing through them.

Unpack and Pass-through are the supported Multihop routing techniques.

This section contains the following subsections:

- B7.5.1 *Unpack Multihop routing*
- B7.5.2 *Pass-through Multihop routing*
- B7.5.3 *Unpack and Pass-through combination*
- B7.5.4 *MultihopSupport capability*
- B7.5.5 *Request and response Unpack Multihop routing with port aggregation*
- B7.5.6 *Snoop Unpack Multihop routing with port aggregation*
- B7.5.7 *Pass-through Multihop routing*

B7.5.1 Unpack Multihop routing

For Unpack Multihop routing, an intermediate chip:

- At the ingress port, unpacks the container into individual messages.
- Using on-chip interconnect resources, routes these messages either to a local destination or to an egress port.
- At the egress port, packs the messages into a new container.

All of the forwarded messages from the original container are not required to be re-packed into a single container.

The new container can include messages routed to the next chip that were not present in the original unpacked container.

The intermediate chip is required to guarantee forward progress of each received message from its ingress port to egress port.

Figure B7.8 shows the Unpack Multihop routing technique in a sparsely-connected topology composed of four chips.

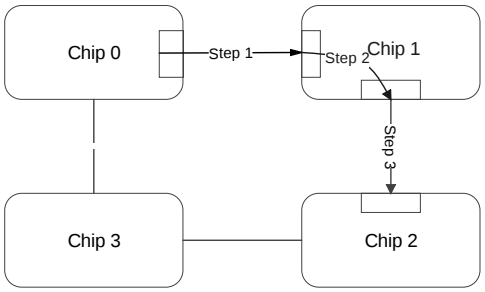


Figure B7.8: Unpack Multihop routing example

In Figure B7.8, a message going from Chip 0 to Chip 2 is unpacked from its container at Chip 1 ingress port and then re-packed into a new container at Chip 1 egress port.

The characteristics of Unpack routing are:

- No further C2C crediting is required beyond single-hop requirements.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- Unpacking at each hop allows a given container to have a combination of messages bound for different chips.
- The unpacking and packing across each intermediate chip may add cycles to the total message transport latency.

Multihop routing paths in the forward and reverse direction must be set up to avoid routing deadlocks.

> **Note**
>
> In particular classes of system topologies, Multihop routing paths may require additional considerations, such as the allocation of dedicated buffer resources.

For different message types, the following routing rules apply:

- The route for a response to a request must be identical to the route taken by the associated request, but in the reverse direction.
- The route for a snoop issued by a Home Node to a Request Node must be identical to the route that a request from that Request Node to the same address would take, but in the reverse direction.
- The route for a response to a snoop must be identical to the route taken by the associated snoop, but in the reverse direction.

B7.5.2 Pass-through Multihop routing

For Pass-through Multihop routing, the containers are not unpacked at intermediate chips. Instead, the container is directly routed to a target chip identified by the ChipTgtID field in the container protocol header.

When Pass-through Multihop routing is supported, containers are permitted to be sent to target chips that are not neighboring the source chip. The paths taken by the containers can traverse multiple hops. The links from the source chip to the remote, non-neighboring chips, are referred to as Multihop Logical Links.

Different messages in a container routed with Pass-through Multihop routing are permitted, but not required, to have different final destination chips from each other and from the container target.

The chip responsible for unpacking the container is not required to be the final destination for the messages it contains.

The path to the container target must be a common subset of all the individual paths that the messages included in that container would take if routed individually.

Figure B7.9 shows Pass-through Multihop routing in a sparsely-connected topology composed of four chips.

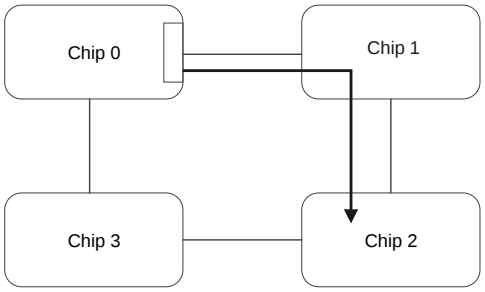


Figure B7.9: Pass-through Multihop routing example

In Figure B7.9, a message going from Chip 0 to Chip 2 is routed through Chip 1 as part of the same container originally packed at Chip 0 egress port. The container is only unpacked at its destination, Chip 2 ingress port.

The characteristics of Pass-through Multihop routing are:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- Unpacking and packing containers at each hop is eliminated.
- Two types of credits are required to be able to send a container. Message class credits are required from the target chip for each of the included messages. Container credits are required from intermediate chips. See B8.8.2 *Pass-through Multihop routing* for more detail on Pass-through Multihop routing flow control.
- Requiring credits from each target chip may limit the scalability of the system that can be supported.
- Packing may be less efficient, or it may take longer to fill a container with messages destined to the same non-neighboring chip.

When Pass-through Multihop routing is used, forward and reverse routing paths are considered to be the same if the messages are packed into and unpacked out of containers at the same ports.

B7.5.3 Unpack and Pass-through combination

A message is permitted to mix different Multihop routing techniques within a path from source chip to target chip. For example, part of the transport may use Pass-through routing while another part of the path uses Unpack.

The Multihop routing technique used at an egress port is not permitted to dynamically change per destination. That is, for an egress port that is set to use Pass-through routing to reach a specific target chip, all traffic packed within that port and destined for the same target chip must be packed in containers sent on a Multihop Logical Link.

Requests targeting different address ranges within the same target chip are allowed to use different Logical Links. For example, this can be achieved when the two address ranges are covered by separate address decoders.

B7.5.4 MultihopSupport capability

The MultihopSupport capability indicates whether the interface supports Multihop routing and which techniques are supported.

The MultihopSupport capability is not a hardware-negotiated property. The MultihopSupport capability is an attribute of each port that is expected to be discovered by the software at the system initialization time. The software is expected to use this information for setting up routing paths.

Table B7.2 shows the MultihopSupport capability encodings, as defined in the C2C_PortCap1 register.

Table B7.2: MultihopSupport encodings

Bits	Field name	Description	
[3:2]	Reserved		
[1]	Pass-through		
		`0b0`	The interface does not support Pass-through routing.
		`0b1`	The interface supports Pass-through routing. The port is capable of receiving containers not targeted for the local chip. The containers are forwarded to an egress port on the same chip without unpacking into individual messages.
[0]	Unpack		
		`0b0`	The interface does not support Unpack routing.
		`0b1`	The interface supports Unpack routing. The port is capable of forwarding received messages to an egress port on the same chip

When MultihopSupport[1:0] is `0b00`, the interface does not support Multihop routing. That is, an ingress port does not support forwarding of unpacked messages or entire containers to an egress port on the same chip. ChipTgtID, ChipSrcID, and ContCredit protocol header fields are inapplicable and must be 0. When

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

MultihopSupport[1:0] is `0b11`, it is permitted for a port to support both Unpack and Pass-through Multihop routing techniques.

Additionally, the C2C_PortCap1.NumLogLinks register field indicates the number of Logical Links supported by the port:

- If NumLogLinks is 1, the port must not have MultihopSupport[1] as `0b1`. That is, the port does not support packing a container with a ChipTgtID value matching a non-neighboring chip.
- If NumLogLinks is greater than 1, the port may support packing a container with a ChipTgtID value matching a non-neighboring chip.

A maximum of 4 Logical Links per port is supported. Logical Link 0 is expected to connect to the neighboring chip.

B7.5.5 Request and response Unpack Multihop routing with port aggregation

In Figure B7.10, a request is routed from an Request Node in Chip 0 to an Home Node in Chip 2, through an intermediate Chip 1 using Unpack Multihop routing. A response is then routed back on the reverse path. In Figure B7.10, the message path consists of two hops, and each of the two pairs of chips are connected by multiple links using port aggregation.

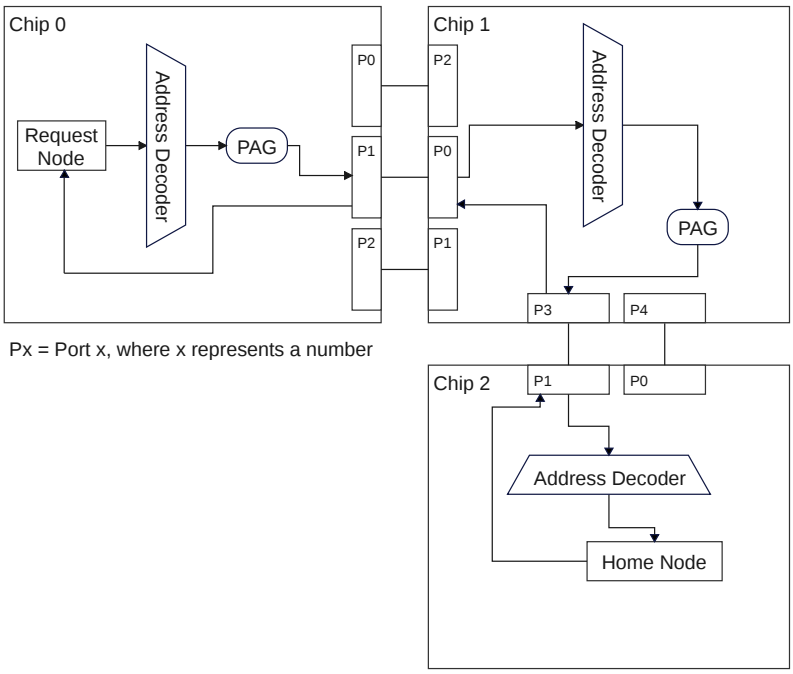


Figure B7.10: Request and response routing, Multihop path with port aggregation

The following steps are taken to perform the routing in Figure B7.10:

1. In Chip 0:
   a. At the source Request Node, the request address is range-checked against all available Address Decoders. In Figure B7.10, the output of the selected Address Decoder is a PAG number.
   b. The selected PAG decoder is used to identify the egress port to which the Request message will be sent. In Figure B7.10, this is P1. A Logical Link number will also be returned, if applicable.
   c. The Request message is routed to the determined egress port, P1, in an IMPLEMENTATION DEFINED manner. At the egress port, the message is packed into a container and transmitted to Chip 1.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

2. In Chip 1:
   a. At the ingress port, the received container is unpacked into individual messages. The Request message address is compared against available Address Decoders to determine if the request is destined to a local agent or if it must be forwarded to an egress port. In Figure B7.10, the request is destined to Chip 2, which is connected to Chip 1 through the aggregated ports, and the Address Decoder points to a PAG to hash across the aggregated ports.
   b. The selected PAG decoder is used to identify the egress port that the Request message will use to reach Chip 2. In Figure B7.10, this is P3. A Logical Link number will also be returned, if applicable.
   c. The Request message is routed to the determined egress port, P3, in an IMPLEMENTATION DEFINED manner. The local routing must include information to identify the used ingress port within the set of aggregated ports and the associated Logical Link number, if applicable. This is to allow routing of the response back to the same port in the intermediate chip.
   d. At the egress port, the message is packed into a container and transmitted to Chip 2.
3. In Chip 2:
   a. At the ingress port, the received container is unpacked into individual messages. The Request message address is compared against available Address Decoders to determine if the request is destined to a local agent or if it must be forwarded to an egress port. In Figure B7.10, the request is destined to a local agent.
   b. The Request message is routed to its local destination in an IMPLEMENTATION DEFINED manner. The local routing must include information to identify the used ingress port within the set of aggregated ports and the associated Logical Link number, if applicable. This is to allow routing of the response back to the same port.
   c. At the Home Node, the Response message is routed to the port identified in step (3b). In Figure B7.10, this is P1. At the egress port, the message is packed into a container and transmitted to Chip 1.
4. In Chip 1:
   a. At the ingress port, the received container is unpacked into individual messages. The Response message is determined to be destined for a remote agent in an IMPLEMENTATION DEFINED manner and the port that was used in this chip by the original Request message is identified as mentioned in step (2c).
   b. The Response message is routed to the determined port. In Figure B7.10, this is P0. At the egress port, the message is packed into a container and transmitted to Chip 0.
5. In Chip 0:
   a. At the ingress port, the received container is unpacked into individual messages. The Response message is determined to be destined for a local agent by inspecting its ChipID or in an equivalent IMPLEMENTATION DEFINED manner.
   b. The Response message is routed to its local destination in an IMPLEMENTATION DEFINED manner.

Note

It is required that the request hashing across the entire multihop path is programmed such that the number of aggregated ports used is the same on the entire journey. In Figure B7.10, this number is 2.

B7.5.6 Snoop Unpack Multihop routing with port aggregation

In Figure B7.11, a snoop message is routed from a Home Node in Chip 2 to a Request Node in Chip 0. The snoop is routed through an intermediate Chip 1 using Unpack Multihop routing. In Figure B7.11, the message path consists of two hops, and each of the two pairs of chips are connected by multiple links using port aggregation.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

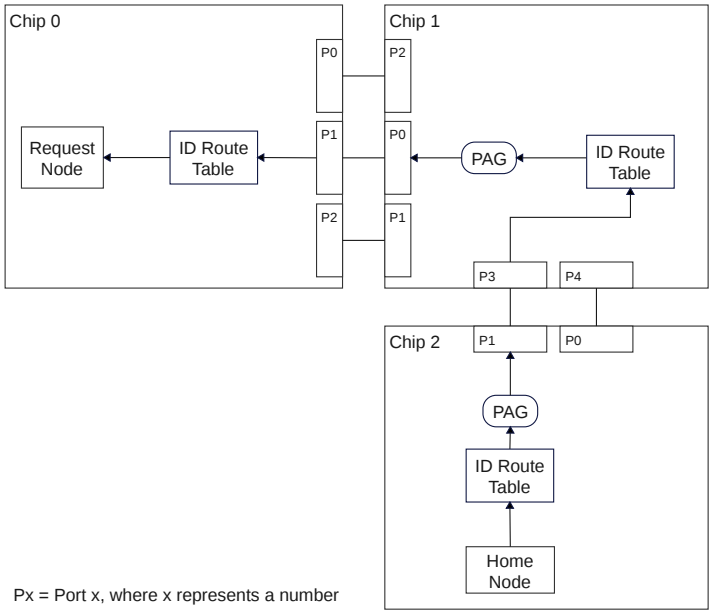


Figure B7.11: Snoop routing, multihop path with port aggregation

The following steps are taken to perform the routing:

1. In Chip 2:
   - At the source Home Node, an ID Route Table can be looked up using the source ChipID included in the original allocating Request message, as shown in Figure B7.11, to obtain a PAG number.
   - The selected PAG decoder is used with the message address to identify the egress port to which the Request message will be sent, that is, P1. A Logical Link number will also be returned, if applicable.
   - Alternatively, if snoop PAG hashing is not used, the port number can be directly obtained by the Home Node by tracking the port on which the allocating request was originally received, and the associated Logical Link number to use, if applicable.
   - The snoop message is routed to the determined port, that is, P1.
   - At the egress port, the message is packed inside a container and transmitted to Chip 1.
2. In Chip 1:
   - At the ingress port, the received container is unpacked into individual messages. An ID Route Table is looked up to determine that the snoop message is destined for a remote agent. The lookup returns a PAG number.
   - The selected PAG decoder is used with the message address to identify the egress port to which the Request message will be sent, P0 in this example. A Logical Link number will also be returned, if applicable.
   - Alternatively, if snoop PAG hashing is not used, the port number can be obtained by tracking the ingress port on which the allocating request was originally received in this intermediate chip, and the associated Logical Link number to use, if applicable.
   - The snoop message is routed to the determined egress port. At the port, the message is packed into a container and transmitted to Chip 0.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

3. In Chip 0:
   - At the ingress port, the received container is unpacked into individual messages. The snoop message is determined to be destined for a local agent by looking up an ID Route Table with the ChipID included in the message, or by equivalent IMPLEMENTATION DEFINED means.
   - The snoop message is routed to its local destination in an IMPLEMENTATION DEFINED manner.

B7.5.7 Pass-through Multihop routing

In Figure B7.12, messages are packed into a container that is routed from Chip 0 to Chip 2, through an intermediate Chip 1. This is applicable only when Pass-through Multihop routing is supported for the ports shown in Figure B7.12.

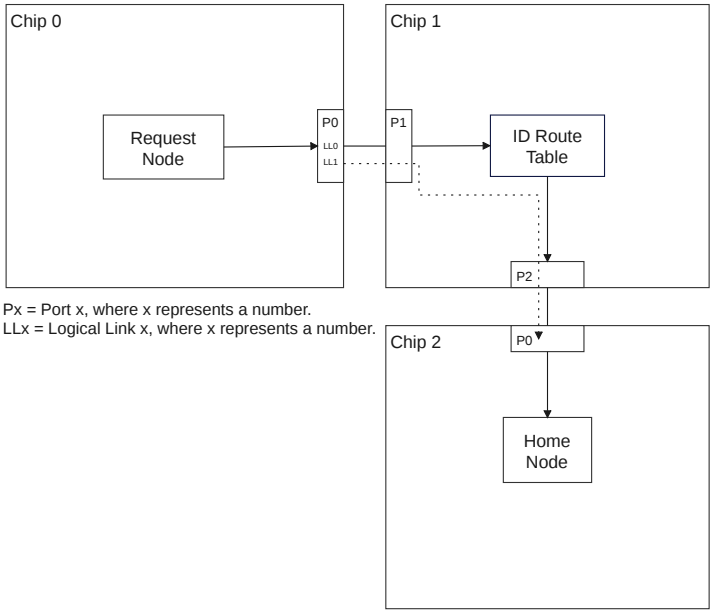


Figure B7.12: Container routing

The following steps are taken to perform the routing:

1. In Chip 0:
   - A C2C container is packed that is destined for Chip 2. Protocol messages can be included in this container if they share the same Logical Link number, as described in B7.5.2 *Pass-through Multihop routing*. In Figure B7.12, Chip 0 uses Logical Link 1 to send containers to Chip 2, as shown by a dashed arrow in Figure B7.12. When the container is packed:
     - The values of ChipTgtID and ChipSrcID in the protocol header are populated appropriately for the associated Logical Link number. The pair of identifiers are read from the C2C_LogLinkCtrlStatus2 register.
     - The value of NumHops must be set to indicate the number of hops to the target chip for the associated Logical Link, as indicated in the C2C_LogLinkCtrlStatus2.NumHops register field. In Figure B7.12, NumHops is set to `0b001` to indicate 2 hops.
2. In Chip 1:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- At the ingress port, the NumHops container header field is checked to determine if the container must be unpacked at this ingress port. In Figure B7.12, NumHops is greater than 0 so the container is not unpacked at this ingress port.
- An ID Route Table is looked up using the container header ChipTgtID field, to determine the egress port to which to route the container. In Figure B7.12, this is egress port P2.
- As the container is not unpacked, the NumHops field in the container header must be decremented before the container is forwarded onto the next chip in the path.
- The container is routed to the determined egress port in an IMPLEMENTATION DEFINED manner. At the port, the container is forwarded to Chip 2.

3. In Chip 2:
   - At the ingress port, the NumHops container header field is checked to determine if the container must be unpacked at this ingress port. In Figure B7.12, NumHops is 0 and the container is unpacked into its composing messages. The messages can be routed individually as described in the previous sections.
   - When the container is unpacked, the ChipSrcID field of the container header is used to associate the container to the correct Logical Link, for credit management purposes. Specifically, message class credits corresponding to the unpacked messages must be added back to the correct credit pool, and any message class credits granted via the container header must be associated to the correct outgoing Logical Link.

Note

Different mechanisms exist to associate the ChipSrcID of the received container to the appropriate Logical Link. For example, the ChipSrcID can be compared with the ChipTgtID values of all the supported outgoing Logical Links. Alternatively, the ChipSrcID can be used to lookup an ID Route Table.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B8
Flow control

This chapter describes the message flow control for a C2C interface. It contains the following sections:

- B8.1 *Introduction*
- B8.2 *Message credits*
- B8.3 *Resource Planes*
- B8.4 *Data credit pools*
- B8.5 *Multi-request flow control*
- B8.6 *Ordered interface*
- B8.7 *DVM flow control*
- B8.8 *Multihop flow control*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.1 Introduction

Messages are required to adhere to the following conditions:

- A message must not be dropped or lost in transport, that is, every message that is sent by a Transmitter must be received and processed by the Receiver.
- Incoming packets from the Link layer or Physical layer must be accepted without applying backpressure, in accordance with industry wide physical design standards. See UCIe [2] for more information.
- Response and Data channel messages must be accepted without dependency on forward progress of any Request or Snoop messages.
- Snoop channel message must progress without dependency on forward progress of Request channel messages.
- WritePush requests must not block forward progress of any data that is not associated with a WritePush.

The following mechanisms are used to support the flow control requirements:

- Separate credits per message class are provided to ensure messages from an individual message class can progress, while other message class messages are back pressured.
- Multiple Resource Planes and DAT credit pools are supported within REQ and DAT message classes to support forward progress guarantees for different message groups within each message class.
- Ordering of messages within a message class and an RP is also required.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.2 Message credits

To support flow control requirements, it is required that:

- Messages from each message class must be flow controlled independently. See Figure B8.1 for more information.
- Uncredited MISC messages do not require a message credit and the Receiver must guarantee to accept such a message.
- To send a WrReqDataS or WrReqDataL message, the Transmitter needs to have one REQ and one DAT message credit. See B8.4 *Data credit pools*.

Figure B8.1 shows that for each message class, the Receiver must provide message credits to the Transmitter. This ensures a Transmitter has the appropriate credits before sending a message.

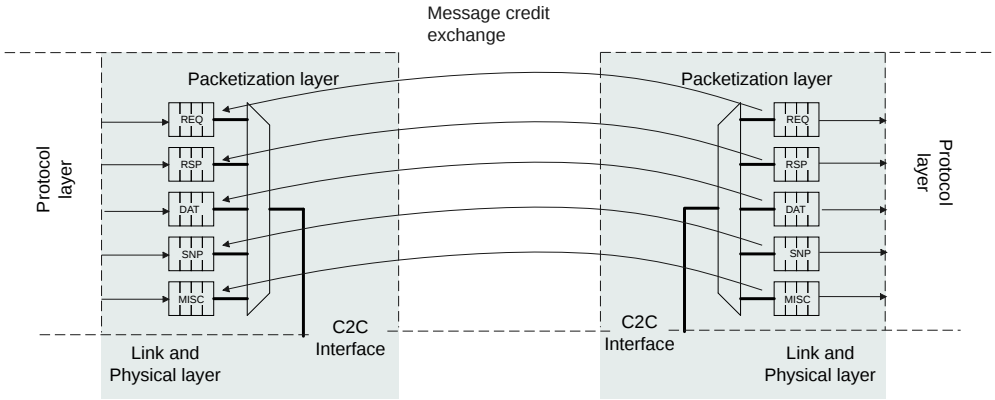


Figure B8.1: Message credits

Credits can be granted using either the Protocol header or using a MiscU message.

Granted credits are used when a message in the associated message class is sent.

Table B8.1 shows the message type and credits required for each message.

Table B8.1: Message class credits for each message type

Message type	Credits required
ReqS ReqL	1 REQ
DataS DataL	1 DAT
WrReqDataS WrReqDataL	1 REQ and 1 DAT
WrDataExt	1 DAT
Snoop	1 SNP
Resp	1 RSP
Resp2	2 RSP

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.1 – *Continued from previous page*

Message type	Credits required
MiscU	None

Table B8.2 shows the maximum number of credits per Logical Link that a Transmitter is required to track for each credit type, where x is a numerical value.

Table B8.2: Maximum number of credits per Logical Link

Credit type	Credit limit
REQRPx	63
REQSh	1023
DATDPx	1023
DATSh	1023
SNP	1023
RSP	1023
MISC	1023

Note

For each supported Logical Link, the number of message class credits to be granted to the connected Transmitter can be programmed using the C2C_LogLinkCrdt1, C2C_LogLinkCrdt2, C2C_LogLinkCrdt3, C2C_LogLinkCrdt4, and C2C_LogLinkCrdt5 registers.

B8.2.1 Credit grant using a Protocol header

Credits can be granted to the Transmitter using the message credit grant field, MsgCredit, in the Protocol Header. The subfields of the MsgCredit field are shown in Figure B8.2. The MsgCredit field can be used to grant credits for all message classes, except MISC.

A container with MiscU.Activation or MiscU.Properties messages must not be used to grant credits using the Protocol header. See B12.2 *Interface activation and deactivation* for details on MiscU.Activation messages.

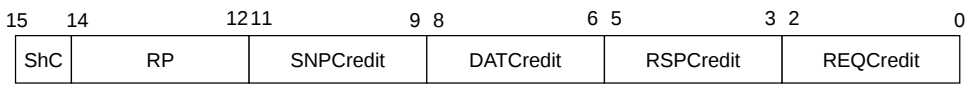


Figure B8.2: Message credit fields in Protocol Header

SharedCrdt (ShC) and RP fields are included to be used when multiple REQ Resource Planes or DAT credit pools are supported. ShC and RP field values are applicable to both REQCredit and DATCredit fields, but are not applicable to RSPCredit or SNPCredit fields.

When ShC = 0 The REQCredit and DATCredit both correspond to credit pools indicated by the RP field value and:

– When WritePush_Support and DVMPush_Support are both False, DATCredit must be 0.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- When either WritePush_Support or DVMPush_Support, or both, are True, DATCredit must be 0 when the RP value is greater than 1.
- REQCredit must be 0 when the RP field indicates a value that is higher than the maximum REQ RP supported.
- The maximum permitted value for the RP field is defined as follows:
  - When WritePush_Support and DVMPush_Support are both False, it is equal to the maximum REQ RP supported.
  - Otherwise, it is the higher of the maximum REQ RP supported and 1.

When ShC = 1 The REQCredit and DATCredit both correspond to shared credit pool. The RP field is inapplicable and must be zero.

See B8.3 *Resource Planes* and B8.4 *Data credit pools* for details of the shared credit and dedicated credit pool usage.

B8.2.2 Credit grant using a MiscU message

Credits for all message channels can be granted using a MISC channel message by sending an explicit credit grant message on the MISC channel. See B5.9.3 *CrdtGrant message format*.

Table B8.3 shows the encoding of the message credit field.

Table B8.3: Message credit field encoding

Message credit[2:0]	Num credit
000	0
001	1
010	2
011	4
100	8
101	16
110	Reserved
111	Reserved

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.3 Resource Planes

Multiple resource planes are typically used to enable:

- Forward progress guarantees against different requests. For example, to ensure the forward progress of PCIe posted writes is not dependent on the progress of other write transactions.
- Service time guarantees. For example, messages that are part of real-time traffic which need to be serviced within required latency bounds.
- Simplification of the protocol. For example, eliminating Request retry without creating a transport deadlock, even when two phases of a transaction use the same transport.

This specification supports request *Resource Planes* (RP) to provide transport independence between requests. When using Resource Planes:

- Messages using different Resource Planes must not block one another from the local Protocol layer to the remote Protocol layer.
- The following types of credits can be issued:
  - Dedicated credits are for a specific RP and can only be used for messages belonging to that RP.
  - Shared credits can be used for messages assigned to any RP.
- Messages using the same RP must remain in-order, even if they are using shared credits.
- Any RP for which the Transmitter has credits can be used to send messages.
- It is recommended, but not required, that a Transmitter uses shared credits before dedicated credits.

> **Note**
>
> The container header can only be used to return shared credits or dedicated RP credits for one RP at a time. To return dedicated credits for multiple Resource Planes, a MiscU.CrdtGrant message must be used. However, since the MiscU.CrdtGrant message occupies a container granule, this has the effect of reducing-packing efficiency. If an implementation prioritizes the use of shared credits over the use of dedicated credits, it will avoid any reduction to overall packing efficiency.

There can be up to eight Resource Planes for Request messages.

A Receiver can divide its buffers between shared and dedicated credit pools. The number of message credits for each dedicated credit pool is permitted to be different.

The Receiver must have available at least one dedicated credit per request RP supported and at least one credit for shared credit pool.

A Request message must:

- Indicate if it is using shared pool credit.
- Include the RP to which it belongs to, even when using shared pool credit.

SharedCrdt and ResPlane fields are included in request messages to support Resource Planes:

- SharedCrdt. Indicates if a request is using a shared credit.

When SharedCrdt = 0	The request is using dedicated RP credit. The RP number is indicated by the ResPlane field value.
When SharedCrdt = 1	The request is using shared pool credit. The RP to which the message belongs must be indicated by the ResPlane field value, even though shared credits are being used.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- ResPlane[2:0]. Indicates the RP to which the request belongs.

  The field value can be 0 to the value of the negotiated Num_RP_REQ property. The Num_RP_REQ property defines the maximum number of dedicated RP supported by the Request message class. See B15.11.9 *Transmitter property registers* and B15.11.8 *Receiver property registers* for the Num_RP_REQ encoding.

See B5.3 *Request message fields* for the ReqS and ReqL message format.

B8.3.1 Request Resource Plane mapping

Request RP mapping provides a mechanism to map both Request Resource Planes used on-chip to Resource Planes used on the C2C link, and the Resource Planes used on the C2C link to the Resource Planes used on-chip.

C2C_LogLinkRPMapTable is programmed for each supported Logical Link at each C2C interface to define RP mapping. In the Transmitter, Tx, direction, the table maps an on-chip RP number to a C2C RP number. In the Receiver, Rx, direction, the table maps a C2C RP number to an on-chip RP number.

For each target chip that is connected to an egress port via a Logical Link, it is possible to define a different RP mapping.

The following conditions apply:

- An entry is only applicable when a Valid bit is set to 1.
- Mappings can be one-to-one or many-to-one. One-to-many mapping is not permitted.

The characteristics of the number of Resource Planes at each interface are:

- The negotiated C2C number of Request Resource Planes are permitted to be different in each direction.
- When the output side of the RP mapping table has a greater number of Resource Planes than the input side, some Resource Planes on the output side are not used. Potentially, this results in under-utilization of RP resources.

The system software must program the RP mapping tables.

B8.3.2 Request resource plane attributes

For each request Resource Plane, the following attributes are defined:

- A per-port RP C2C_PortCap1.FwdProgressRP[7:0] capability vector. C2C_PortCap1.FwdProgressRP[x] indicates if the hardware resources for the chip implementation in the receive direction are such that the internal RP[x] can be considered free-flowing.
  - An RP is considered free-flowing if transactions that use it are guaranteed to complete in a timely manner, without dependencies on other transactions.
- A per-logical-link RP attribute control table, programmable through the C2C_LogLinkRPAttrTable. This table allows software to control use of each request RP in the transmit direction. For each entry, the programmable attributes are:
  - Bit[1], DAT1En, allows the Transmitter to use DAT1Credit credits for WrReqData* and WrDataExt messages sent on the RP. This bit can only be set when Bit[0] is also set.
  - Bit[0], WrPushEn, allows the Transmitter to use WrReqData* and WrDataExt messages on the RP.

The expectation is that, for each Logical Link, software discovers the RP attribute capabilities of the Receiver port, and uses that information to set the attributes on the Transmitter side.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.4 Data credit pools

Credits that are used to transfer messages with data, DAT credits, are split into the following credit pools:

- DAT0Credit, a dedicated credit used to transfer Data messages that need progress to be guaranteed over any blocked WritePush message. The sending of a message using DAT0Credit must not be dependent on the progress of messages using credits from the other data credit pools.
- DAT1Credit, a dedicated credit used to transfer WrReqData* messages that need guaranteed forward progress. The sending of a message using DAT1Credit must not be dependent on progress of messages using credits from the other data credit pools
- DATShCredit, a shared credit that can be used to transfer any data message. The use of DATShCredit by messages relating to one request RP must not block or be blocked by messages relating to another request RP.

Table B8.4 shows which message types are permitted to use which credit types.

Table B8.4: Data credit options for each message type

Message type	Messages	Credit type
Data	Read response with data, Snoop response with data, WritePull data response.	DATShCredit, DAT0Credit
WrReqData*	WritePush that require forward progress guarantees.	DATShCredit, DAT1Credit
	WritePush that do not require forward progress guarantees.	DATShCredit

The choice of using a dedicated or shared credit can depend on the required progress of the transfer. For example:

- A PCIe write which must make forward progress, and is using the WrReqData* message, is expected to use a dedicated credit when no shared credits are available.
- A DVM message which can be stalled by other messages can wait for a shared credit to be available and does not need dedicated credit support.

At least one credit of DATShCredit type must be available.

If either WritePush_Support or DVMPush_Support are True, at least one credit of each dedicated DAT credit type must be made available.

If both WritePush_Support and DVMPush_Support are False, DAT0Credit and DAT1Credit are not required and credits for them must not be sent.

B8.4.1 Credits for DataS and DataL messages

The DataS and DataL fields include a SharedCrdt field with the following encoding:

When SharedCrdt = 0 The data message is using a DAT0Credit.

When SharedCrdt = 1 The data message is using a DATShCredit.

See B5.5 *Data message fields* for the DataS and DataL message format.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.4.2 Credits for WrReqData messages

A WrReqData* message includes a request and write data, so needs one REQ message credit and one DAT message credit.

The request part of a WrReqData* message can be allocated an RP in the same way as other request messages.

Any WrReqData* message that:

- Requires forward progress guarantees is permitted to use either DATShCredit or DAT1Credit. It is recommended, but not required, to use DATShCredit before DAT1Credit.

  Note

  Independent forward progress between write transactions that use the WritePush flow is not guaranteed when multiple Resource Planes are allowed to use the same DAT credit types.

- Does not require forward progress guarantees must use DATShCredit.

The credit and RP options for WrReqData* messages are encoded in SharedCrdt and ResPlane fields as follows:

- SharedCrdt

  When SharedCrdt = 0 The request is using a REQxCredit. The data is using a DAT1Credit.

  When SharedCrdt = 1 The request is using a REQShCredit. The data is using a DATShCredit.

- ResPlane[2:0]

  Indicates the RP to which the request is assigned.

See B5.6.2 *Message fields* for the WrReqData message format.

Note

A write message that needs both Request and Data shared credits to progress using WritePush semantics may change to using WritePull to progress on the transaction, when simultaneous availability of both shared credits is not provided in a timely manner.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.5 Multi-request flow control

For all multi-request messages, except WritePush, the Transmitter only requires a single REQ message credit to be able to send the request.

For a multi-request WritePush:

- A Transmitter is:
  - Permitted to send a multi-request WrReqDataL message when it has at least one REQ message credit and one DAT message credit available. The Transmitter is then permitted to wait for additional DAT message credits to arrive before sending the remaining WrDataExt messages. A Transmitter must use the DAT credits it has been granted in a timely manner.
  - Permitted to use any combination of DATShCredit and DAT1Credit DAT message credit types for WrDataExt messages that are part of the same multi-request WritePush transaction. The SharedCrdt field of the WrDataExt message encodes the type of DAT message credit used by each WrDataExt message.
  - Permitted to have multiple outstanding WritePush transactions on a single RP.
  - Not permitted to interleave data associated with multiple single or multi-request WritePush transactions on a single RP. That is, all of the data associated with a WritePush transaction must have been transmitted before a new WritePush transaction can begin on the same RP, including all applicable WrDataExt messages.
  - Permitted to interleave data associated with multiple WritePush transactions that are on different Resource Planes.
- A Receiver must be able to receive WrDataExt messages for the negotiated maximum length of WritePush multi-request. A Receiver is permitted to achieve this by credit re-use of a smaller set of DAT credits.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.6 Ordered interface

The C2C interface has the ordering requirements described in this section, in addition to the ordering requirements of the CHI protocol [1].

Containers must be received at the C2C interface in the same order that they were sent by the C2C interface on the remote side of the link.

It is required that:

- At the Transmitter:
  - Messages from the same message class and, where applicable, the same RP must be placed in the container in the same order that they are received from the Protocol layer, that is in granules starting from the lowest number first to higher numbers.
  - Messages from different message classes do not have such ordered placement restrictions.
- At the Receiver:
  - The Packetization layer must maintain the order of placement of messages of a given message class within the received container as well as order in which containers are received.
  - The messages within a message class of the same RP value must be forwarded to the Protocol layer in the same order as they are received.

See B2.4 *Multiple interfaces* for ordering requirements when using multiple interfaces between two components.

The ordered interface is a required feature and hence an interface property to control its presence is not included.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.7 DVM flow control

The following rules must be followed by the Requester and the Completer of DVM transactions to guarantee transaction forward progress is made.

A DN:

- Must have only one DVMSync request outstanding to remote DVM nodes.
- Permitted to have any number of DVMReq requests outstanding to remote DVM nodes.
- Must be able to accept one DVMSync from each remote DN.

Note

This requirement that all DVMSync requests must be sunk at the destination DN is placed to avoid a request path being blocked at the C2C interface.

At a DN, a DVMSync must not block forward progress of DVMReq transactions. That is, a DN must guarantee forward progress of DVMReq transactions received from remote nodes.

Note

For example, this guarantee can be obtained by reserving a buffer for received DVMReq transactions.

DVMReq transactions forward progress at the C2C interface must not depend on forward progress of DVMSync transactions.

See B9.3.1 *DVMReq* and B9.3.2 *DVMSync* for more details.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.8 Multihop flow control

This section describes rules and considerations related to flow control for Multihop routing of messages and containers.

B8.8.1 Unpack Multihop routing

The rules for flow control of messages routed using Unpack Multihop routing are:

- Messages must receive message class credits from the next chip.
- Intermediate chips must guarantee forward progress of messages from their ingress port to the egress port.

Note

When Multihop messages routed using the Unpack mechanism share on-chip transport resources with messages generated locally and targeting the same egress ports, forward-progress guarantees can be obtained by using additional Receiver resources to remove the on-chip generated messages from the transport path. This clears the path for Unpack Multihop messages to reach the egress port.

When the Unpack Multihop messages do not share on-chip transport paths with any other traffic, the message forward progress can be obtained by using reserved Receiver resources for the messages.

B8.8.2 Pass-through Multihop routing

The flow control of messages routed using Pass-through Multihop routing is handled in the following manner:

1. For a message to be included in a container, the sender of the message must have credits of the appropriate message class from the container target chip.
2. The intermediate chips on the path must guarantee forward progress for the container from ingress port to egress port.
3. In order for the container to be forwarded to the next chip on the path, the sender egress port must have a container credit from the connected ingress port in the next chip on the path. The only exception to this rule is when NumHops is 0, indicating that the container will be unpacked at the next chip on the path. For the last hop on the path, the message class credits obtained at step (1) above to include individual messages in the container guarantee that the container can be accepted at the Receiver.
4. When the container reaches the target chip ingress port where it is unpacked, the Receiver needs to associate the ChipSrcID of the received container to the correct Logical Link for credit management purposes. See B7.5.7 *Pass-through Multihop routing* for detail on how this can be achieved.

Note

For Pass-through Multihop routing, it is possible to reach transport-level deadlocks where Logical Links are setup and routed in a loop, such that no container credits are available to progress containers on any of the links. Software should setup container routing paths to avoid the formation of such deadlock loops.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B9
DVM transactions

This chapter describes how DVM transactions are supported in C2C. It contains the following sections:

- B9.1 *Introduction*
- B9.2 *DVM Node, DN*
- B9.3 *DVM transaction flows*
- B9.4 *DVM transaction responses*
- B9.5 *DVM with Multihop routing*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B9.1 Introduction

All CHI DVM transactions are supported over the C2C interface [1]. A DVM Node (DN) is defined which is responsible for propagating local DVM transactions to remote chips and performing the required local invalidations as requested by remote DVM requests.

DVM Nodes also manage flow control to guarantee DVM transaction forward progress.

DVM message flows are optimized by introducing new response types and reducing the number of messages.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B9.2 DVM Node, DN

A DN processes DVM operations to and from remote chips. A DN performs the following functions:

- Sends DVM requests to remote DVM Nodes for local DVM transactions. When sending the request to multiple nodes, it may use unicasting DVM requests to each remote DN or a hierarchical tree communication pattern to send to a subset of remote DN which further forward the request to another subset and so spanning the DVM request to all DVM nodes in the system.
- Collects responses for DVM requests it sent to remote DN.
- Performs locally required invalidations for remote DVM requests.
- Coordinates interaction between local and remote DVM transactions.

The number of DVM Nodes in the system is limited to 64.

B9.2.1 Multiple DVM Nodes per chip

A chip is permitted to include one or more DVM Nodes. Each DVM Node, regardless of whether it is part of a chip with multiple DVM Nodes or whether it is the only DVM Node on the chip, must be associated with a unique ChipID. That is, each DVM Node on a chip must be assigned a different ChipID.

Conversely, a chip that is assigned multiple ChipID values, can have a number of DVM Nodes that are less than or equal to the number of assigned ChipID values, including no DVM Nodes.

Note

Issue A permits a maximum of one DVM Node per chip.

The AgentID value for a DVM Node must be 0. Such a DVM Node NodeID assignment enables a DVM Node to be routed on-chip by ChipID value alone.

B9.2.2 DVM Node assignment

A DN is permitted to share ID name space with Request Nodes and Home Nodes on the same chip.

This requires DVM responses which are targeting a DN to be identified separately from responses targeting Request Nodes. This is achieved by Response opcode types, defined in B9.4 *DVM transaction responses*.

Note

The ability to share Node ID name space minimizes the total number of unique identifiers required in the system.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B9.3 DVM transaction flows

The two DVM transactions over C2C are:

- DVMReq
- DVMSync

In this specification, DVMReq refers to DVMOp(NonSync) and DVMSync refers to DVMOp(Sync). These use the same DVMOp Request opcodes defined in CHI [1], and are distinguished by the DVMType field.

B9.3.1 DVMReq

The DVMReq transaction flow is illustrated in Figure B9.1. The flow is similar to a 32-byte write partial transaction flow. The DVMReq transaction can use WritePush or WritePull semantics.

When using WritePush, the DVMReq is transmitted using a WrReqDataL message.

When using WritePull, the DVMReq is transmitted using either a ReqS or ReqL message.

Note

The ReqS message can be used by the DVMReq message as Addr[3:0] is not used on the on-chip CHI REQ channel DVMOp message packing [1].

The DVMReq is permitted to use WritePush semantics only when DVMPush_Support property is True.

The Completer responses are:

- When a WritePush flow is used, CompDVM.
- When a WritePull flow is used, CompDVM, DBIDRespDVM, or a combined CompDBIDRespDVM.

See B9.4 *DVM transaction responses* for response message details.

The DVM transaction payload is distributed across the Addr and Data fields of the request, as defined in on-chip CHI [1]. CompDVM and DVMData are differentiated from Comp and Data responses for non-DVM transactions by using different opcodes.

Figure B9.1 shows the transaction flow for a DVMReq.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

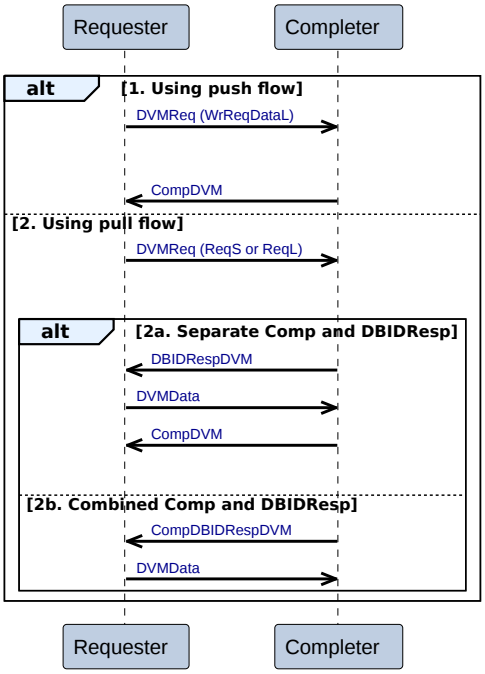


Figure B9.1: DVMReq transaction flow

> **Note**
>
> On-chip CHI is used to pass DVM transactions from a local DN or ingress port to the C2C Requester [1].
>
> After being sent across the C2C interface, the C2C Completer uses on-chip CHI to pass the transaction to the local DN or an egress port [1].

B9.3.2 DVMSync

DVMSync uses a dedicated transaction flow, as shown in Figure B9.2. The flow is similar to a Dataless request transaction flow with a single response.

The DVMSync request is transmitted using either a ReqS or ReqL message type with the DVMSync opcode. The complete response is given using either Resp or Resp2 message type with the CompSyncDVM opcode.

All required payload is in the request address fields and so the DVM transaction data response is eliminated.

The DVMSync flow is used when the on-chip Opcode field indicates the DVMOp transaction and the DVMType field indicates Synchronization.

Request is tracked by SrcID alone. There can be only one DVMSync transaction outstanding from each source. Thus, the TgtID in the response is used to match the response to its corresponding request.

In the request, the TxnID is inapplicable and must be 0.

In the response, the TxnID field is inapplicable and must be 0.

Figure B9.2 shows the transaction flow for a DVMSync.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

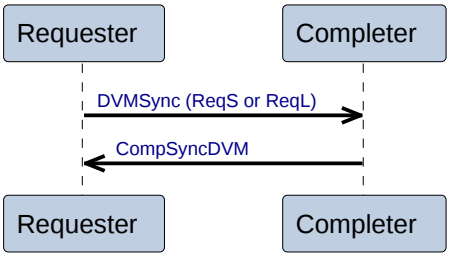


Figure B9.2: DVMSync transaction flow

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B9.4 DVM transaction responses

The response messages for DVMReq and DVMSync messages are:

- For DVMReq:
  - CompDVM
  - DBIDRespDVM
  - CompDBIDRespDVM
  - DVMData
- For DVMSync:
  - CompSyncDVM

Table B9.1 shows the opcodes for the DVM transaction non-data responses.

Table B9.1: DVM transaction response opcode encodings

Opcode[4:0]	Response message
`0x1C`	CompDVM
`0x1D`	DBIDRespDVM
`0x1E`	CompDBIDRespDVM
`0x1F`	CompSyncDVM

Table B9.2 shows the opcode for a DVM transaction data response.

Table B9.2: DVMReq data response opcode encodings

Opcode[4:0]	Data message
`0xD`	DVMData

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B9.5 DVM with Multihop routing

This section describes rules and considerations for DVM support that relate to Multihop routing. For information on Multihop routing see B7.5 *Multihop routing*.

The fundamental principle of DVM message routing remains that a sequence of one or more DVMOp requests and a DVMSync, as issued by a DVM Node, must reach all the other DVM Nodes in the system.

When multiple Logical Links are supported, there can be a maximum of one DVMSync transaction outstanding from each source node per Logical Link.

The mechanism for routing DVM transactions is to propagate the messages out from the source DVM Node.

Each ingress port that receives an incoming DVMOp or DVMSync may do both the following:

- Each ingress port must forward the message to all the local DVM Nodes.
- Each ingress port may be required by the defined DVM broadcast tree to forward the message to other chips via local egress ports.

The exact definition of the propagation and any software-visible supporting structures is IMPLEMENTATION DEFINED.

When Port Aggregation Groups are used, all DVM requests from the same source DVM Node must use the same port within the aggregated ports. It is the responsibility of the source DVM Node to guarantee that this requirement is met.

The Responder must set the TgtID value in the response to the SrcID of the corresponding DVM request.

When Port Aggregation Groups are used, the response must use the same port as the corresponding DVM request.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B10

RME-DA and RME-CDA support

This chapter describes the C2C support for RME *Device Assignment* (DA) and RME *Coherent Device Assignment* (CDA), which are part of Arm *Confidential Compute Architecture*. It contains the following sections:

- B10.1 *Introduction*
- B10.2 *Fields supporting RME-DA and RME-CDA*
- B10.3 *Host and device responsibilities*
- B10.4 *Request Nodes per device*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B10.1 Introduction

RME-DA is part of the *Realm Management Extension* architecture that enables the secure assignment of assignable device interfaces. A *Device Permission Table* (DPT) contains the permission attributes associated with physical addresses and specifies a corresponding set of DPT checks that apply to incoming accesses from devices.

In this section, the term, RME-DA, refers to a device that is IO coherent and the term, RME-CDA, refers to a device that has the capability of being fully coherent.

This specification supports RME-DA and RME-CDA in multiple accelerator device topologies, where the accelerators are connected to a host. See Figure B10.1.

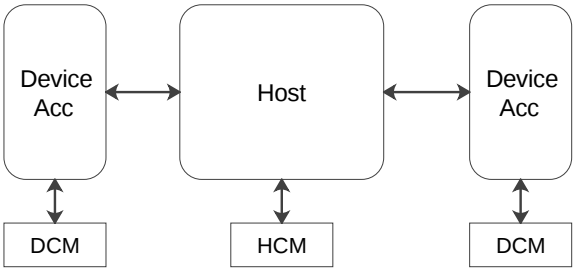


Figure B10.1: A host connected to two Accelerator devices

A host, made from a single or multiple SoC dies, forms a *Trusted Computing Base* (TCB). A host:

- Is responsible for enforcing the security policies required by the system.
- Can be connected to multiple devices.
- Can have memory termed as *Host Coherent Memory* (HCM) directly attached to it. The HCM can be coherently accessed by the host, or by any fully coherent or IO coherent devices.
- Must not send a Stash Snoop to a device, unless the line is already legally permitted to be cached by the device.
- Is permitted to send DVMReq or DVMSync requests towards a device.

A device is a single physical chip(let) and is treated as a single unit from a trust perspective and individually attested. A device:

- Is trusted in its entirety for a set of Realms only and trusted to maintain Realm separation where required.
- Can have private memory-mapped resources.
- Can have one or more interfaces.
- Can have memory termed as *Device Coherent Memory* (DCM) directly attached to it. The DCM can be coherently accessed by the host, the device, and by other coherent devices.
- Can access the DCM of another coherent device only where that access is channeled through a host compute node.
- Only operates in the Realm and Non-secure PAS.
- Is expected, but not required, to include the following fields in all requests to the host:
  - *Security State* (SecSID1) field
  - *Stream ID* (StreamID) field
- Only receives snoop requests for regions of memory owned by Realms that trust the device.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- Must not be directly connected to another device.
- Is not permitted to send DVMReq or DVMSync requests to a host.

The host and the devices can have multiple Request Nodes and Home Nodes.

It is expected that the host will perform additional checks on certain messages from or to a device. These checks can ensure that:

- Requests coming from the device to HCM are permitted to access that memory. This means the device must not issue a request with a random PA and expect to get access to that location.
- Snoops coming from the device are only for DCM. This means the device must not issue a request with arbitrary PA to gain access to a location.
- Snoops going to the device for cached HCM locations only expose accesses that the device is permitted to observe. This means the device cannot build up a picture of all trusted system activity for more targeted attacks.

For more information on RME-DA and RME-CDA, see [1].

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B10.2 Fields supporting RME-DA and RME-CDA

The fields used to enable support for RME-DA and RME-CDA include:

StreamID Stream Identifier. StreamID is a unique identifier of request streams originating from one or a set of Requesters associated with the same System MMU context. The StreamID field in messages at an interface is applicable as defined by the DevAssign_Support properties.

> **Note**
>
> When interfacing to PCIe, StreamID is the Requester ID associated with a single PCIe endpoint.

SecSID1 Secure Stream Identifier. SecSID1 qualifies the Security state of StreamID. The SecSID1 field in messages at an interface is applicable as defined by the DevAssign_Support properties.

MECID Memory Encryption Context Identifier. MECID is an identifier assigned to memory accesses, associating each access with a memory encryption context. The MECID value is unique to each Realm. The MECID field in messages at an interface is applicable as defined by the DevAssign_Support and MEC_Support properties.

For more details on these fields and their applicability, see [1].

> **Note**
>
> It is expected that before link activation, a device advertises DevAssign_Support_Tx with a one-hot encoding of either Device_StreamID_SecSID1 or Device_NoStreamID_NoSecSID1.
>
> For more information on the DevAssign_Support_Tx property, see Table B14.9.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B10.3 Host and device responsibilities

For details on the responsibilities of a host or device in a system that supports RME-DA or RME-CDA, see [3].

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B10.4 Request Nodes per device

In an accelerator-attached configuration when the device DevAssign_Support_Tx property indicates an interface is represented as a device:

- At the interface, the maximum number of Request Nodes permitted is 16. From this, a maximum of 8 nodes can be Caching Request Nodes. The remaining can be IO coherent Request Nodes. The Caching Request Nodes must be assigned node ID values within the range 0 to 7.
- The host must precisely track device caching to determine if a Caching RN on that device has cached a copy of the line.

Note

The manner in which memory caching is tracked at a Home determines the efficiency of snoop filter implementation.

Precisely knowing where a line is cached results in a Home sending targeted snoops to that cache.

With coarse grain tracking, a Home needs to resort to multi-casting the snoop to a subset of caches or broadcasting the snoops to all the caches except the Requester. Both multicasting and broadcasting of snoops involves a Home sending multiple unicast snoops.

A fine grain tracker at Home, typically a snoop filter or a directory, increases in size with respect to the increase in the number of caches to be tracked precisely.

To manage the increase in the size of a fine grain tracking snoop filter, an architectural upper limit on the number of caching agents per interface is placed. A host that supports RME-CDA can preemptively size its cache tracking structure to that limit.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B11
Integrity and Data Encryption

This chapter describes the mechanisms used to support link *Integrity and Data Encryption* (IDE) across a C2C link. It contains the following sections:

- B11.1 *Introduction*
- B11.2 *Format Z and IDE*
- B11.3 *Multihop routing with IDE*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B11.1 Introduction

IDE provides confidentiality and integrity guarantees to the traffic transported over a C2C link. Confidentiality is achieved by encrypting the message ahead of transmission and decrypting it upon receipt. Integrity is achieved by including a *Message Authentication Code* (MAC) that is checked at the destination to ensure the message information has not been corrupted.

Dedicated IDE engines perform data encryption and decryption for confidentiality, MAC insertion, and integrity checks. Figure B11.1 shows the IDE engines are located within the Link layer at each side of the C2C link. The specific details of the IDE engine are outside the scope of this specification.

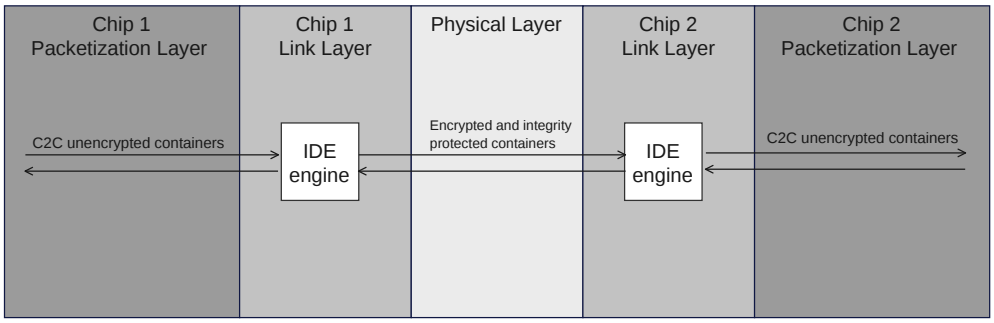


Figure B11.1: IDE engine location

At the Transmitter IDE engine, the MAC is calculated over a sequence of consecutive containers, which is referred to as an IDE epoch. The length of this sequence is referred to as the IDE epoch size. At the Receiver IDE engine, the authentication checks compare the received MAC against a version of the code computed locally on the received containers for the same IDE epoch.

To allow for MAC computation latency, the Transmitter is permitted to send containers belonging to a new IDE epoch in a pipeline-like manner before it has sent the MAC associated with the previous IDE epoch. The Transmitter sends the MAC associated with IDE epoch N as part of a container that belongs to IDE epoch (N + 1). The time between the transmission completion of all containers for an IDE epoch to the transmission of the associated MAC is known as the MAC insertion delay. The MAC insertion delay is measured in the number of containers.

MAC must be sent in the same order that the containers for the associated IDE epochs are sent.

Figure B11.2 shows an example sequence for the MAC placement of three IDE epochs with a MAC insertion delay of two containers, and an IDE epoch size of two containers.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

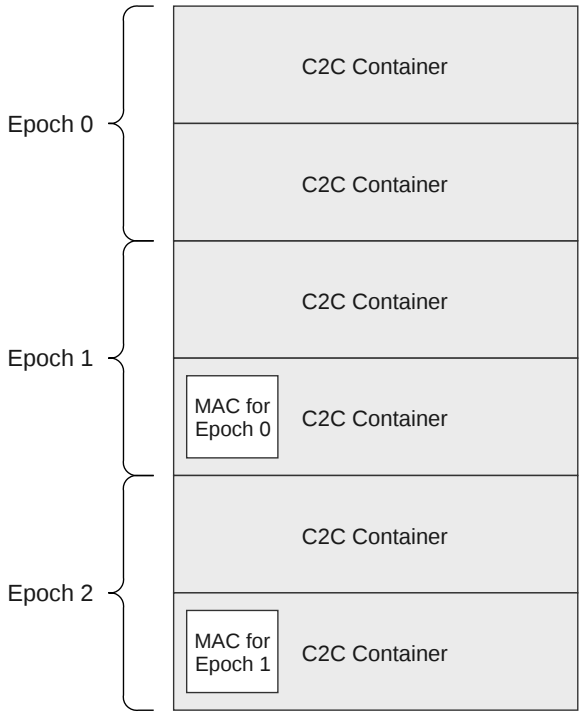


Figure B11.2: Example of MAC transmission with MAC insertion delay = 2 and IDE epoch size = 2

IDE messages are used to start, operate, and terminate an IDE session across the C2C interface. For information on IDE messages, see B5.9.7 *IDEMsg message format*.

B11.1.1 Limitations

The applicability of encryption and integrity protection in any multi-chip topology is limited to a single hop. That is, the point at which a C2C container is decrypted and authenticated is always across a single physical link from where it was encrypted and integrity protected. This is known as Link-level IDE support.

The Receiver Link layer must not pass decrypted containers to the Protocol layer until the MAC associated with the IDE epoch has been received and all the containers included in the IDE epoch have passed the authentication check. This is known as containment mode.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B11.2 Format Z and IDE

For IDE protection, container Format Z is required for traffic exchanged across an IDE-enabled link. See B3.1 *Containers* for information on Format Z containers. However, the use of container Format Z is permitted even when IDE protection is not required. See B11.2.1 *Encryption and integrity protection to container fields* for a description of the expected encryption and integrity protection scope for the container Format Z content. Before the IDE session begins, the C2C packer and unpacker for the associated Logical Link must be programmed to start using container Format Z, if that is not already in use.

When an IDE session is active, granule G0 can be used by the Link layer IDE engine to transmit the MAC across the link in one container per IDE epoch. The packer of the C2C container is required to reserve granule G0 for MAC insertion by the IDE engine at least once per IDE epoch. The applicability of granule G0 reservation and additional message rollover rules is restricted to container Format Z. See B11.2.2 *IDE message rollover rules* and B11.2.3 *Packer reservation of granule G0 for MAC insertion* for more information.

B11.2.1 Encryption and integrity protection to container fields

For Format Z, it is expected that the scope of encryption and integrity protection to be applied by the Link layer IDE engine onto the container as it is transmitted across the link is the following:

- All LinkHdr fields are not encrypted or integrity-protected. See Figure B3.1 for the layout of the LinkHdr fields in Format Z.
- Table B11.1 shows the Format Z fields that are encrypted and integrity-protected.
  The following key is used:

  Y Yes.

  - No.

Table B11.1: Encrypted and integrity-protected fields in Format Z

Field	Expected IDE scope	
	Encryption	**Integrity protection**
LinkHdr	-	-
Granule G0[3:0] with MAC or an IDE message	-	-
Granule G0[3:0] without MAC or an IDE message	-	Y
Granule G0[111:4] with MAC or an IDE message	-	-
Granule G0[111:4] without MAC or an IDE message	Y	Y
Granule G0[159:112]	Y	Y
Granule G1 to G10	Y	Y
Granule G11[47:0]	Y	Y
Granule G11[63:48]	-	Y

B11.2.2 IDE message rollover rules

The message rollover packing rules for container Format Z are modified such that when a message spans across consecutive containers, and the packer has been programmed to start reserving granule G0, the message must continue in granule G1 of the next container.

This change is required, such that:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- MAC insertion by an IDE engine does not clash with any protocol message rollover content.
- Protocol message rollover content is appropriately protected by the scope of encryption and integrity protection, as described in B11.2.1 *Encryption and integrity protection to container fields*.

When the packer has not been programmed to reserve granule G0, the message rollover uses granule G0 of the next container.

When the packer has been programmed to reserve granule G0, granule G0 is permitted to be used for protocol messages in containers where it is not reserved for MAC insertion.

When G0 is not reserved for MAC insertion and is used for sending protocol messages:

- If there is no previous container message rollover, G0 can be used to start protocol messages of any size, except DataL messages.
- If there is a previous container message rollover, G0 can only include a message that is limited to one granule in size and does not have an ordering dependency on the message that is rolling over.

When the packer has been programmed to reserve granule G0, granule G0 is excluded from the first group of the granule group tight packing rules, as defined in B3.3.1 *Rules and restrictions*. That is, protocol messages can be packed in the first granule group starting from G1 even if G0 is empty.

B11.2.3 Packer reservation of granule G0 for MAC insertion

Container header bit, G0IDECtrl, can be used in conjunction with the MsgStart[0] container header bit by a packer to indicate that granule G0 has been reserved for MAC insertion, and to provide information about granule position of any message rollover.

The G0IDECtrl field is inapplicable and must be 0 in the container formats other than Format Z.

Before an IDE session begins at an IDE engine, the container packer must be programmed to start reserving at least one granule G0 per IDE epoch to allow for MAC insertion. When the packer starts reserving granule G0, it must assume that the containers will traverse a link with an ongoing IDE session and must follow the message rollover rules in B11.2.2 *IDE message rollover rules*.

When a C2C packer is programmed to start reserving granule G0 for MAC insertion, the minimum requirement is to reserve one granule G0 for each IDE epoch. The required minimum G0 reservation frequency is defined by the C2C_LogLinkCtrlStatus1.G0ReserveFrequency register field. A packer implementation is permitted to over-reserve granule G0.

For more information on the G0IDECtrl field, see B3.2 *Protocol header*.

Table B11.2 shows how the G0IDECtrl field can be used with the MsgStart[0] bit to transport information about G0 reservation for MAC insertion and granule position of any message rollover. Table B11.2 is only applicable when the IDE_G0_Reservation property has negotiated to True.

Table B11.2: G0IDECtrl field encodings

{G0IDECtrl, MsgStart[0]}	No ongoing rollover	Ongoing rollover
`0b00`	– G0 is not suitable for MAC insertion. – No rollover. – G0 is empty. – Protocol message may start in G1.	– G0 is not suitable for MAC insertion. – Rollover in G0.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B11.2 – *Continued from previous page*

{G0IDECtrl, MsgStart[0]}	No ongoing rollover	Ongoing rollover
`0b01`	– G0 is not suitable for MAC insertion. – No rollover. – Single-granule or multiple-granule protocol message starts in G0.	– G0 is not suitable for MAC insertion. – Rollover in G1. – Single-granule protocol message in G0.
`0b10`	– G0 is suitable for MAC insertion. – No rollover. – Protocol message may start in G1.	– G0 is suitable for MAC insertion. – Rollover in G1.
`0b11`	– Encoding is Reserved. It must not be used.	– G0 is not suitable for MAC insertion. – Rollover in G1. – G0 is empty.

The container packer must set the {G0IDECtrl, MsgStart[0]} to `0b10` in all containers that are packed with G0 reserved for MAC insertion.

A C2C packer must only set {G0IDECtrl, MsgStart[0]} to `0b10` in a container if it can guarantee that it will set the same two bits to `0b10` for all subsequent IDE epochs in a manner that is compatible with the requested C2C_LogLinkCtrlStatus1.G0ReserveFrequency register field.

The Link layer IDE engine can insert a MAC in containers with {G0IDECtrl, MsgStart[0]} set to `0b10`.

To understand whether the granule G0 has valid protocol content, a C2C unpacker can check the {G0IDECtrl, MsgStart[0]} container header bits, as defined in Table B11.2. When a rollover is expected, the {G0IDECtrl, MsgStart[0]} container header bits also identify the starting granule for the rollover.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B11.3 Multihop routing with IDE

An IDE-protected link can be found at one or more intermediate hops as part of a Multihop routing path. Containers that are sent on the IDE-protected link may have been packed at either the local egress port or forwarded as received from ingress ports located on the same chip:

- When Unpack multihop routing is supported, containers packed locally may include messages generated from on-chip agents, as well as messages received from ingress ports on the same chip.
- When Pass-through multihop routing is supported, there may be containers that have been forwarded unchanged from ingress ports as part of a logical link that traverses the IDE-protected link. In this scenario, the containers have been packed at a remote chip egress port. Figure B11.3 shows how two different streams of on-chip containers are combined by the egress port container selection logic before being sent to the Link layer IDE engine.

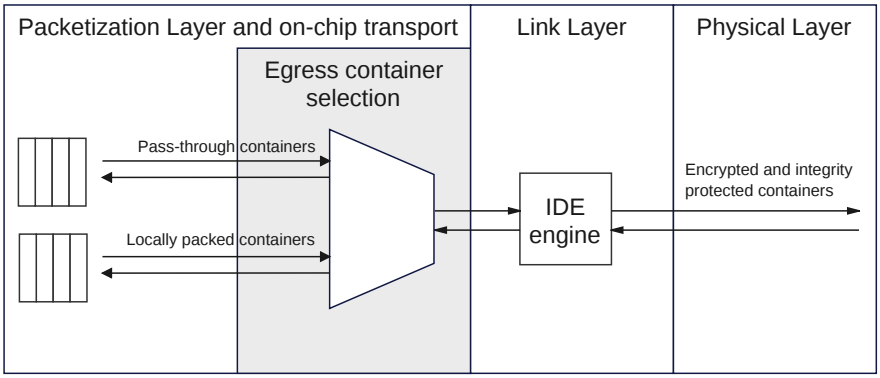


Figure B11.3: IDE with Pass-through multihop routing

In Pass-through multihop routing, all the local and remote container packers for Logical Links that traverse the IDE-enabled link must be programmed by the software to reserve at least one granule G0 per IDE epoch for MAC insertion and to follow the message rollover rules defined in B11.2.2 *IDE message rollover rules* before the IDE session begins at the IDE engine.

The egress container selection logic must combine containers so that the IDE engine receives a sequence of un-encrypted containers that:

- Includes at least one container with {G0IDECtrl, MsgStart[0]} = `0b10` per IDE epoch, in a position compatible with the MAC insertion delay.
- Complies with existing rules on the container rollover sequence interleave. That is, no container rollover sequences can be interleaved on the C2C link. See B3.3.2 *Container rollover limit* for more information.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B12
Interface state management

This chapter describes the C2C interface state management. It contains the following sections:

- B12.1 *Introduction*
- B12.2 *Interface activation and deactivation*
- B12.3 *Coherency connect and disconnect*
- B12.4 *DVM connect and disconnect*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B12.1 Introduction

Hardware flows manage the connectivity state of the interface and, in turn, the link.

The three different connect and disconnect set of flows affect the fully coherent traffic, DVM messages, and all protocol messages.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B12.2 Interface activation and deactivation

Interface activation and deactivation is used to enable or disable transport of all protocol traffic across the interface. The MiscU.Activation message type supports interface activation and deactivation. See B5.9 *Uncredited Miscellaneous message fields* for details of these messages.

B12.2.1 Activity states

The interface can be in one of four activity states.

STOP The interface is disconnected from all protocol activity.

ACTIVATE The interface is preparing to enable protocol activity.

RUN The interface is enabled for protocol activity.

DEACTIVATE The interface is preparing to disconnect from all protocol activity.

See B12.2.4 *Activation and Deactivation flow* for details of the interface behavior in each state.

The permitted state transitions are:

- STOP to ACTIVATE
- ACTIVATE to RUN
- RUN to DEACTIVATE
- DEACTIVATE to STOP

Figure B12.1 shows the permitted state transitions.

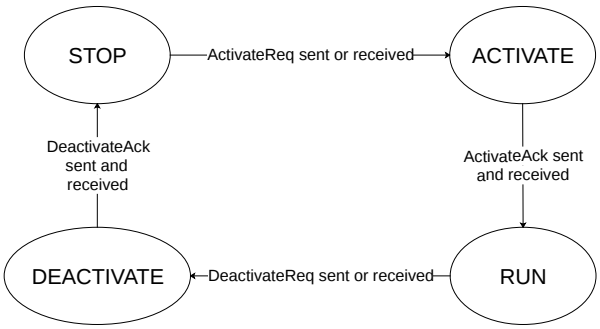


Figure B12.1: Activation deactivation state transition

The remote credits are implicitly returned when the interface moves into the STOP state. That is, all Transmitter message credit counts are set to 0 and Receiver message credits are reset to the values they were at the interface initialization.

B12.2.2 DeactivateHint usage constraints

The usage constraints of a DeactivateHint message include the following:

- An interface is only permitted to send this message in the RUN activation state.
- An interface is permitted to send a DeactivateHint and then resume activity on the link.
- An interface is permitted to send multiple DeactivateHint messages, either with or without link activity between them.
- An interface is permitted to send a DeactivateReq without first sending a DeactivateHint message.
- If an interface has sent a DeactivateHint, it is permitted to send a DeactivateReq without waiting for any response to the DeactivateHint.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

> **Note**
>
> The sender may use heuristics to decide on when to send the DeactivateHint message. For example, the heuristics might use the length of time the link has been inactive, that is, the time period for which no message is sent or received across the link.

- The Receiver is permitted to respond to the DeactivateHint message in the following manner:
  - Can drop the hint without any state change.
  - Accept the hint and respond with DeactivateReq when the conditions for sending the message are met.

B12.2.3 Messages

The following five messages are defined to support interface activation flow:

- ActivateReq
- ActivateAck
- DeactivateReq
- DeactivateAck
- DeactivateHint

See Table B5.15 for their message format and Table B5.16 for ActivationOp encoding.

B12.2.4 Activation and Deactivation flow

This section describes the behavior of the interface in each activation state.

There are two options for conditions under which the interface deactivation can be initiated. The two deactivation conditions, defined by the Deactivation_Support property, are:

1. Protocol agnostic: DeactivateReq is permitted to be sent from the RUN state at any time and does not need to check protocol message flows. Additionally, the protocol messages must not be sent after DeactivateReq is sent until the interface comes back to RUN state.
2. Protocol aware: DeactivateReq is permitted to be sent only when interfaces are in the RUN state and are completely quiesced in both directions. A reason for quiescing the interface may be for powering down the interface or the chip. Quiesce condition requires both sides must not have any protocol traffic to send. That is, all outstanding requests must have completed, no response messages must be sent, and no new request or snoop messages must be sent across the link.

The interface behavior for each activation and deactivation state includes:

STOP

- The credits are reset in this state.
- The Receiver has all the credits but must not send any credits.
- The Transmitter has no credits and is only permitted to send LinkStatus to the local Protocol layer or ActivateReq messages.
- If an interface wants to start sending other messages, it must send an ActivateReq message.
- When an interface has sent or received an ActivateReq message, it must move into the ACTIVATE state.

ACTIVATE

- If not already sent, send an ActivateReq message.
- If not already received, wait for an ActivateReq message.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- After receiving the ActivateReq message, ActivateAck must be sent in a timely manner in response to the request.
- When an interface has sent and received an ActivateAck message, it must move into the RUN state.

RUN

- Interface properties are exchanged, if requested.
- Then, credits must be sent. It is recommended that credits are only sent for Resource Planes negotiated using interface property exchange.
- Credited messages can be sent.
- To indicate to the remote side that the interface is prepared to move to the STOP state, it can send a DeactivateHint message. The DeactivateHint message can be ignored by the remote side or the remote side can send a DeactivateReq message.
- To initiate a move to the STOP state, when the required activity is completed, the interface sends a DeactivateReq message.
- When an interface has sent or received a DeactivateReq message, it must move into the DEACTIVATE state.

DEACTIVATE

- If not already sent, when required activity is completed, send a DeactivateReq message.
- Credits must be sent, until DeactivateReq is received.
- Credited messages must not be sent, after the DeactivateReq message is sent.
- DeactivateAck must be sent in a timely manner after a DeactivateReq is received.
- Credits must not be sent after the DeactivateAck message is sent.
- When an interface has sent and received a DeactivateAck message, it must move into the STOP state.

B12.2.5 Activation trigger, ActTrig

A 2-bit software-visible activation control field, C2C_LogLinkCtrlStatus1.ActTrig, enables the hardware to initiate an Activate or Deactivate flow.

Table B12.1 shows the encoding of the ActTrig field.

Table B12.1: ActTrig bit encoding

ActTrig[1:0]	Description
00	No action.
01	Send ActivateReq.
10	Send DeactivateReq.
11	Reserved.

It is sufficient to set the trigger bits on one side to start the activation or deactivation sequence.

For example, if an Accelerator is attached to a host using the C2C link, the triggers can be implemented only on the host side.

The software can write the ActTrig field in the following ways:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- The software is only permitted to write the ActTrig[0] bit to `0b1`. ActTrig[0] is cleared automatically by hardware as part of the activation flow. An implementation must ensure that the ActTrig[0] bit is cleared when the interface moves to the RUN state.
  - The software is only expected to write the ActTrig[0] bit to `0b1` when C2C_LogLinkCtrlStatus1.ActState field is STOP. An implementation must ignore writes to the ActTrig[0] bit when the interface is in any state other than STOP.
- The software is only permitted to write the ActTrig[1] bit to `0b1`. ActTrig[1] is cleared automatically by hardware as part of the deactivation flow. An implementation must ensure that the ActTrig[1] bit is cleared when the interface moves to the STOP state.
  - The software is only expected to write the ActTrig[1] bit to `0b1` when C2C_LogLinkCtrlStatus1.ActState field is RUN. An implementation must ignore writes to the ActTrig[1] bit when the interface is in any state other than RUN.

Before an interface deactivation sequence can begin, the software is required to disconnect coherency domains for both directions, and DVM.

B12.2.6 Activation state, ActState

A 2-bit, software-visible status field, C2C_LogLinkCtrlStatus1.ActState, is defined for each Logical Link.

This field indicates the interface activation state for the Logical Link.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B12.3 Coherency connect and disconnect

Interface coherency connect and disconnect messages are used to connect or disconnect the interface from coherency management requirements.

B12.3.1 Coherency connect states

For each C2C interface, there are two separate coherency states:

1. The state of local Requesters participating in a coherency domain that is managed by remote Home Nodes.
2. The state of local Home Nodes managing coherency for remote Requesters.

Each of these coherency states can independently have one of the following values:

CohDisabled The Requester caches do not include coherent data.

CohConnect An interface is requesting to be permitted to cache coherent data.

CohEnabled The Requesters participate in coherent data exchange.

CohDisconnect The Requesters are requesting to be removed from participating in coherent data exchange.

The permitted state transitions are:

- CohDisabled to CohConnect
- CohConnect to CohEnabled
- CohEnabled to CohDisconnect
- CohDisconnect to CohDisabled

The interface moving from CohDisconnect to CohDisabled state must retain and not zero out the SNP credits held from the remote side.

Figure B12.2 shows the permitted state transitions for the coherency state of local Requesters.

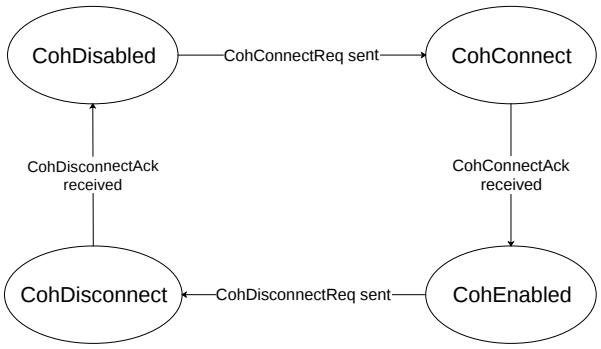


Figure B12.2: Coherency domain local Request Node state transitions

Figure B12.3 shows the permitted state transitions for the coherency state of local Home Nodes.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

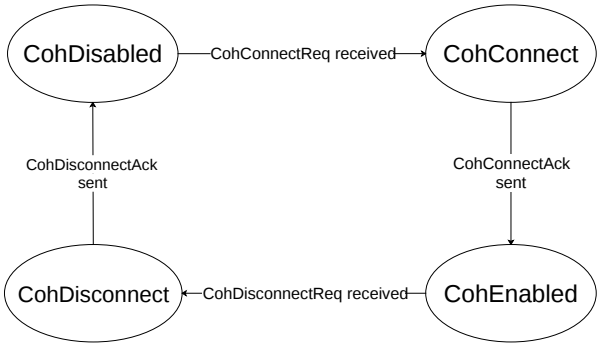


Figure B12.3: Coherency domain local Home Node state transitions

B12.3.2 Messages

As part of MiscU.Connect message type, there are four messages defined to support interface Coherency connect flow:

1. CohConnectReq
2. CohConnectAck
3. CohDisconnectReq
4. CohDisconnectAck

See Table B5.18 for the MiscU.Connect message format and Table B5.19 for ConnectOp encoding.

A Transmitter is only permitted to issue a CohConnectReq message when the Snoop_Transactions_Rx property has negotiated to True.

Table B12.2 describes the behavior of the two interfaces during connecting and disconnecting to a coherency domain.

Table B12.2: C2C coherency connect state description

State	Interface behavior for	
	Local Request Nodes	**Local Home Nodes**
CohDisabled	Requesters at this interface must not contain coherent data. Must not send Snoopable Allocating requests. Does not: – Receive snoop requests from the remote. – Influence coherency connect state in the reverse direction. Permitted to grant SNP message credits to remote. Must remain in CohDisabled state until CohConnectReq is sent and then must move to CohConnect state.	Must not send snoop requests to remote. Must be able to handle receiving SNP message credits. Does not influence coherency connect state in the reverse direction. Must remain in CohDisabled state until CohConnectReq is received and then must move to CohConnect state.
CohConnect	Permitted to send SNP message credits. Must be prepared to respond to snoop requests. Must remain in CohConnect state until CohConnectAck is received and then must move to CohEnabled state.	Must be able to handle receiving SNP message credits. Must send CohConnectAck in a timely manner and then move to CohEnabled state.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B12.2 – *Continued from previous page*

State	Interface behavior for	
	Local Request Nodes	**Local Home Nodes**
CohEnabled	Permitted to send Snoopable Allocating requests. – Requesters at this interface are permitted to cache coherent data. Must grant SNP credits. Must respond in a coherence compliant manner to any received snoop requests. When desired to disconnect from coherency domain and the Requester caches on the interface do not include coherent data then send CohDisconnectReq and move to CohDisconnect state.	Must be able to handle receiving SNP credits. Permitted to send snoop requests as required to maintain coherency. Must remain in CohEnabled state until CohDisconnectReq message is received and then must move to CohDisconnect state.
CohDisconnect	Must not send Snoopable Allocating requests. Must continue granting SNP credits. The remote side might have snoop requests queued to be sent. Must continue to respond to snoop requests. Must remain in CohDisconnect state until it receives CohDisconnectAck and then must move to CohDisabled state.	Complete outstanding snoop requests. Must send CohDisconnectAck message in a timely manner and move to CohDisabled state.

Note

The coherency states of local Requesters and local Home Nodes are independent from each other. An interface Transmitter may continue to send Snoop requests while the Receiver on the same side of the interface is in coherency disabled state.

B12.3.3 Coherency Trigger, CohTrig

A 2-bit software-visible control field, C2C_LogLinkCtrlStatus1.CohTrig, is defined for each Logical Link. The CohTrig field enables software to request connection or disconnection of local Requesters to a coherency domain that is hardware-managed from across the link.

Table B12.3 shows the encoding of the CohTrig field.

Table B12.3: CohTrig bit encoding

CohTrig[1:0]	Description
`00`	No action.
`01`	Send CohConnectReq.
`10`	Send CohDisconnectReq.
`11`	Reserved.

Note

For each C2C interface, there are two separate coherency states:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

1. The state of local Requesters participating in a coherency domain that is managed by remote Home Nodes.
2. The state of local home nodes managing coherency for remote Requesters.

However, each C2C interface includes a single CohTrig field because a coherency connect-disconnect sequence is always initiated from the requester side.

The software can write the CohTrig field in the following ways:

- The software is only permitted to write the CohTrig[0] bit to `0b1`. The CohTrig[0] bit is cleared automatically by hardware as part of the coherency connection flow. An implementation must ensure that the CohTrig[0] bit is cleared before moving to the CohEnabled state.
  - The software is only expected to write the CohTrig[0] bit to `0b1` when the associated C2C_LogLinkCtrlStatus1.CohStateRN field is CohDisabled. An implementation must ignore writes to the CohTrig[0] bit when the interface is in any state other than CohDisabled.
- The software is only permitted to write the CohTrig[1] bit to `0b1`. The CohTrig[1] bit is cleared automatically by hardware as part of the coherency disconnection flow. An implementation must ensure that the CohTrig[1] bit is cleared before moving to the CohDisabled state.
  - The software is only expected to write the CohTrig[1] bit to `0b1` when the associated C2C_LogLinkCtrlStatus1.CohStateRN field is CohEnabled. An implementation must ignore writes to the CohTrig[1] bit when the interface is in any state other than CohEnabled.

B12.3.4 Coherency states, CohStateRN and CohStateHN

Two 2-bit software-visible status fields, C2C_LogLinkCtrlStatus1.CohStateRN and C2C_LogLinkCtrlStatus1.CohStateHN, are defined for each Logical Link:

- CohStateRN indicates the coherency state of local Requesters participating in a coherency domain that is managed by remote Home Nodes.
- CohStateHN indicates the coherency state of remote Requesters participating in a coherency domain managed by local Home Nodes. It is expected that CohStateHN is used for debug purposes.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B12.4 DVM connect and disconnect

The DVM connect and disconnect messages are used to connect and disconnect the interface from address translation tables management.

B12.4.1 DVM connect states

A C2C interface can be in one of four DVM connect states. See B12.4.3 *DVM connect and disconnect flow* for details of each state.

DVMDisabled The interface is removed from DVM and does not send or receive DVM requests.

DVMConnect An interface is requesting to join DVM.

DVMEnabled An interface is participating in DVM activities.

DVMDisconnect An interface is requesting to be taken out of DVM.

When an interface removes itself from DVM, it neither sends or receives DVM requests.

The permitted state transitions are:

- DVMDisabled to DVMConnect
- DVMConnect to DVMEnabled
- DVMEnabled to DVMDisconnect
- DVMDisconnect to DVMDisabled

Figure B12.4 shows the permitted state transitions.

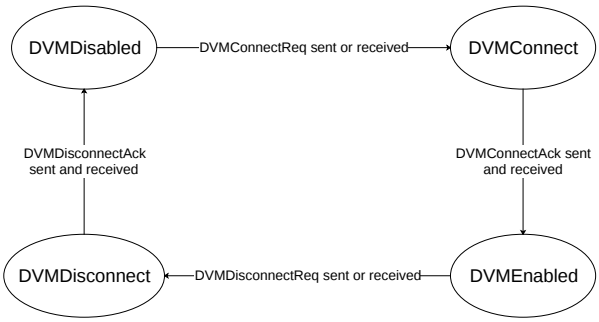


Figure B12.4: DVM connect state transitions

B12.4.2 Messages

The following four messages, grouped together as part of MiscU.Connect message type, are defined to support interface DVM connect flow:

- DVMConnectReq
- DVMConnectAck
- DVMDisconnectReq
- DVMDisconnectAck

See Table B5.18 for the MiscU.Connect message format and Table B5.19 for ConnectOp encoding.

A Transmitter is only permitted to issue a DVMConnectReq message when the DVM_Support_Rx property has negotiated to True, or in response to a received DVMConnectReq.

B12.4.3 DVM connect and disconnect flow

The interface behavior in each DVM connect state includes:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

DVMDisabled

- The Requesters on the interface must not cache address translations.
- The interface must not send or receive DVM transactions.
- The interface is permitted to receive a DVMConnectReq message.
- A DVMConnectReq message can be sent if the interface wants to participate in DVM activities.
- When an interface has sent or received a DVMConnectReq message, it must move into the DVMConnect state.

DVMConnect

- If not already sent, then send a DVMConnectReq.
- If not already received, wait for DVMConnectReq message.
- Once DVMConnectReq is received, send DVMConnectAck in a timely manner.
- Must remain in DVMConnect state until DVMConnectAck is sent and received and then move to DVMEnabled state.

DVMEnabled

- Permitted to send DVM transactions.
- Can receive DVM transactions.
- If the interface wants to be removed from DVM activities, it must wait for all issued transactions to complete, including any required DVMSync, and then send DVMDisconnectReq.
- When an interface has sent or received a DVMDisconnectReq message, it must move into the DVMDisconnect state.

DVMDisconnect

- If not already sent, wait for all issued DVM transactions to complete, including any required DVMSync, and then send a DVMDisconnectReq message.
- After sending a DVMDisconnectReq, must not send any DVM requests.
- Must continue to respond to DVM requests until a DVMDisconnectReq is received.
- Must send DVMDisconnectAck in a timely manner after DVMDisconnectReq is received.
- When an interface has sent and received DVMDisconnectAck message, it must move into the DVMDisabled state.

B12.4.4 DVM Trigger, DVMTrig

A 2-bit software-visible control field, C2C_LogLinkCtrlStatus1.DVMTrig, is included for each Logical Link. This field enables software to request connection or disconnection to DVM activities across the C2C link.

Table B12.4 shows the encoding of the DVMTrig field.

Table B12.4: DVMTrig bit encoding

DVMTrig[1:0]	Description
00	No action.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B12.4 – *Continued from previous page*

DVMTrig[1:0]	Description
`01`	Send DVMConnectReq.
`10`	Send DVMDisconnectReq.
`11`	Reserved.

Note

For all the DVM traffic transmitted on a C2C interface, there is a unified DVM connection state. To ensure the synchronization of the DVM connection state machines on both sides of the interface, a DVMConnectReq or DVMDisconnectReq is sent from one side of the interface. This results in a corresponding DVMConnectReq or DVMDisconnectReq message to be received from the remote side.

As a result, it is sufficient to use the DVMTrig field on one side of the interface to cause a DVM connection or disconnection.

The software can write the DVMTrig field in the following ways:

- The software is only permitted to write the DVMTrig[0] bit to `0b1`. The DVMTrig field is cleared automatically by hardware as part of the DVM connection flow. An implementation must ensure that the DVMTrig[0] bit is cleared before moving to the DVMEnabled state.
  - The software is only expected to write the DVMTrig[0] bit to `0b1` when the associated C2C_LogLinkCtrlStatus1.DVMState field is DVMDisabled. An implementation must ignore writes to the DVMTrig[0] bit when the interface is in any state other than DVMDisabled.
- The software is only permitted to write the DVMTrig[1] bit to `0b1`. The DVMTrig field is cleared automatically by hardware as part of the DVM disconnection flow. An implementation must ensure that the DVMTrig[1] bit is cleared before moving to the DVMDisabled state.
  - The software is only expected to write the DVMTrig[1] bit to `0b1` when the associated C2C_LogLinkCtrlStatus1.DVMState field is DVMEnabled. An implementation must ignore writes to the DVMTrig[1] bit when the interface is in any state other than DVMEnabled.

B12.4.5 DVM State, DVMState

A 2-bit software-visible status field, C2C_LogLinkCtrlStatus1.DVMState, is defined for each Logical Link. This field indicates the DVM state for the Logical Link.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B13
Interface initialization

This chapter describes the C2C interface initialization. It contains the following sections:

- B13.1 *Introduction*
- B13.2 *Initialization flow*
- B13.3 *PortID exchange*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B13.1 Introduction

C2C interface initialization establishes the interface to enable the sending and receiving of protocol messages.

The initialization flow has the following steps:

1. The Protocol layer coordinates with the Link layer to check if the remote Protocol layer is up within the interface.
2. For each supported Logical Link, the Protocol layer exchanges interface properties using MiscU.Properties messages. This sets up the interface to start exchanging message credits followed by protocol messages.

Note

Discovery, enumeration, and negotiation of global or common capabilities, such as Req_Addr_Width, Node ID assignment, address maps, routing table set up, is not managed by the hardware property exchange flows.

The exchange of capabilities that are not part of the hardware property exchange flow is expected to be handled by software.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B13.2 Initialization flow

The key features of the initialization flow include:

- The Protocol layers on the two sides of the link are guaranteed to be able to receive Miscellaneous messages before link initialization is completed. This enables the Protocol layers to exchange MiscU.Activation messages using the default container format.

Figure B13.1 shows the possible transaction flows for interface initialization.

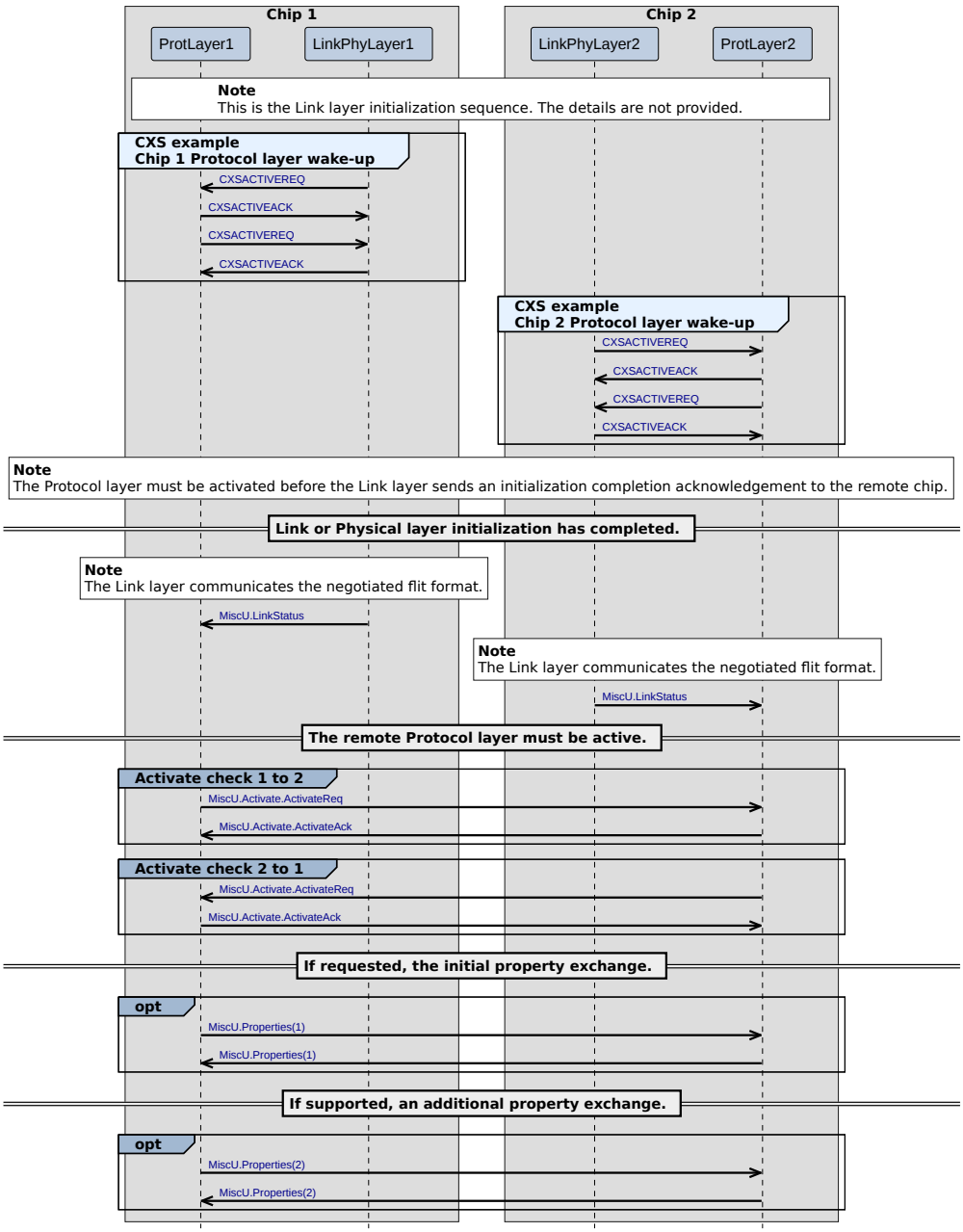


Figure B13.1: C2C link initialization

As shown in Figure B13.1, the sequence for an interface initialization is:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

1. The interface initialization starts with a link reset and initialization.

   Before the link initialization is complete, the Link layer must wake up the Protocol layer. The Link layer to Protocol layer wake-up handshake method is IMPLEMENTATION DEFINED.

   An example wake-up handshake using the CXS interface between the Link layer and Protocol layer is shown in Figure B13.1.

   - The Link layer sends a **CXSACTIVEREQ** signal to the local Protocol layer.
   - Once the **CXSACTIVEREQ** signal is HIGH, the Protocol layer sends a **CXSACTIVEACK** response.
   - In response to the **CXSACTIVEREQ** signal from the Link layer, the protocol performs the same handshake in the opposing direction. That is, a **CXSACTIVEREQ** signal is sent from the local Protocol layer to the Link layer, followed by a **CXSACTIVEACK** response from the Link layer. The handshake in the opposing direction can be performed as late as during the end-to-end, that is, the Protocol layer-to-remote-Protocol layer Activate handshake.

2. After the Link layer initialization has completed, the Link layer must be able to receive and recognize ActivateReq from the remote side. This permits the Link layer to receive the ActivateReq request message, as described in step (4).

3. The Link layer sends a MiscU.LinkStatus message to the local Protocol layer.

   The MiscU.LinkStatus message informs the Protocol layer of the link initialization completion and includes information for the negotiated flit format.

4. The Protocol layer must ensure that the remote Protocol layer is active:

   - Each Protocol layer sends an ActivateReq message to the remote Protocol layer.
   - The Protocol layer responds to the received ActivateReq with a ActivateAck message.

5. At this stage, both sides of the interface are ready to exchange the supported property values using the MiscU.Properties messages, if requested in the ActivateReq message.

   When property exchange is required, it is possible to exchange multiple MiscU.Properties messages, as indicated by the Properties.Num_Properties_Msg field exchanged with the first MiscU.Properties message. See B14.5 *Property packing in MiscU.Properties messages* for more information on each individual MiscU.Properties message.

   The exchange of MiscU.Properties messages occurs in the following manner:

   - The local Protocol layer sends a MiscU.Properties(1) message to the remote Protocol layer.
   - Once the local Protocol layer completes property exchange with the remote Protocol layer, it is possible to determine whether any further MiscU.Properties messages are required to be exchanged.
   - If required, the local Protocol layer sends a MiscU.Properties(2) message to the remote Protocol layer and waits to receive a MiscU.Properties(2) message from the remote Protocol layer, if it has not been received already.

6. The Protocol layer is permitted to grant credits to the remote Protocol layer.

Note

When multiple Logical Links are supported, steps (4), (5), and (6) are performed for each Logical Link separately.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B13.3 PortID exchange

A mechanism based on a dedicated property is defined to support the hardware exchange of PortID between two interfaces. This enables C2C interfaces to identify the PortID of the connected port on the remote side of the link without relying on any software accessing the C2C_PortCtrlStatus1 register for this interface.

The PortID property is exchanged using the MiscU.Properties(2) message. The PortID of the physically connected interface is recorded in the PropInfdUniform2 registers.

The required result of the property negotiation is not defined for the PortID property.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B14
Protocol layer property exchange

This chapter describes the exchange of properties for the Protocol layer. It contains the following sections:

- B14.1 *Introduction*
- B14.2 *On-chip payload properties*
- B14.3 *On-chip opcode and flow properties*
- B14.4 *C2C-specific properties*
- B14.5 *Property packing in MiscU.Properties messages*
- B14.6 *Property packing into registers*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B14.1 Introduction

After the interface initialization is complete, if the activation sequence determines that property exchange is not required:

- The Negotiated property values are set to match the Advertised property values.
- The Informed property values are set to match the Advertised property values.

After the interface initialization is complete, if the activation sequence determines that property exchange is required, the interface properties must be communicated in both directions before any protocol messages are sent.

A property is used to declare a capability.

The interface properties are grouped into three classifications:

1. On-chip payload
   - Details either the presence or width, or both, of certain on-chip fields.
   - Allows for the optimization of the payload fields prior to the message selection, potentially allowing for more efficient packing.
2. On-chip opcode or flow
   - Details the on-chip support of certain classifications of transactions.
   - Ensures that the interface does not deadlock by preventing unsupported messages being sent across the C2C link.
3. C2C-specific
   - Details which specification release the interface supports, to ensure backward compatibility.
   - Details support for C2C-specific features.

Both sides of the Logical Link send MiscU.Properties messages with an indication of their capabilities. The number of MiscU.Properties messages exchanged is defined by the negotiation result of the Num_Properties_Msg included in the first MiscU.Properties message.

MiscU.Properties messages can only be sent as part of the Activation sequence. To trigger a new property exchange, deactivation and re-activation of the Logical Link is required.

The property values in each MiscU.Properties message are driven from a set of registers that define the capabilities of the chip sending the message. See B14.6 *Property packing into registers* for more information.

There are certain properties that have Receiver, Rx, and Transmitter, Tx, variants:

Property_Rx This indicates the capability of the chip sending the MiscU.Properties message as a Receiver. The Transmitter chip must be able to terminate any messages that the Receiver chip indicates it does not support.

Property_Tx This indicates the capability of the chip sending the MiscU.Properties message as a Transmitter. Awareness at the Receiver chip of what it could receive enables potential power optimizations.

Where there is no _Tx or _Rx extension, the property is classified as Uniform and applies to the interface in both directions. Unless otherwise stated, when a Receiver property value indicates support for receiving the request associated with a particular transaction flow, the same property also indicates support for transmitting all applicable responses associated with the same transaction flow.

It is permitted for software to update the Advertised register values prior to sending a MiscU.Properties message to reduce the advertised capabilities of the interface. Software is not permitted to update the Advertised register values to a property support level that is higher than what is indicated in the Supported registers. Violating this rule results in UNPREDICTABLE behavior.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Upon receipt of a MiscU.Properties message, the Receiver must compare each of the communicated properties against its own properties to determine the result of the Negotiated property values. Reserved fields in the Properties message must be set to 0 by the Transmitter and ignored by the Receiver.

Unless otherwise stated, the negotiation of properties is as follows:

- Properties that are defined as encoding increasing levels of support negotiate to the highest value that is supported by both interfaces.
- Properties that are defined as bit vectors negotiate to the bitwise logical AND operation between the two interface vectors.

When the property values align, no modification of behavior is required.

When the properties differ, the Transmitter adapts its behavior accordingly. The required adaptation depends upon the property classification and the differences seen in the:

- On-chip payload
  - If the Transmitter, _Tx, property is True and the Receiver, _Rx, property is False, it is permitted, but not required, that the associated field value is set to 0 prior to message selection and container packing.
  - If the Transmitter, _Tx, property is False and the Receiver, _Rx, property is True, it is required that the associated field value is set to 0 prior to message selection and container packing.
  - If the Transmitter, _Tx property is a larger value than the Receiver, _Rx, property, it is permitted, but not required, that the most significant bits that are not supported at the Receiver are set to 0 at the Transmitter prior to message selection and container packing.
  - If the Transmitter, _Tx, property is a smaller value than the Receiver, _Rx, property, it is required that the associated field value is 0 extended at the Transmitter prior to message selection and container packing.
- On-chip opcode and flow
  - If the local Transmitter, _Tx, property is True and the Receiver, _Rx, property is False, the Transmitter must terminate or downgrade any associated on-chip message in an on-chip protocol-compliant manner. It is IMPLEMENTATION DEFINED how this downgrade or termination takes place.

    Note

    For example:

    - Downgrade includes converting a SnpUniqueStash to a SnpUnique, if the target chip does not support stashing but has a copy of the data that is needed to complete a transaction.
    - Termination includes a Requester performing an Atomic transaction locally when it is established that there is no downstream atomic support. This might be achieved through the use of an on-chip **BROADCASTATOMIC** signal on the Requester or an alternative control register.

  - If the Transmitter, _Tx, property is False and the Receiver, _Rx, property is True, there should be no further action required. The associated on-chip protocol messages should not be presented to the Transmitter for communication across the C2C link.
  - Optionally, if the Receiver, _Rx, property is True and the Transmitter, _Tx, property is False, the Receiver chip might be able to save power by turning off certain portions of logic.
- C2C specific
  - If the Transmitter, _Tx, property is True and the Receiver, _Rx, property is False, the Transmitter must not use the associated feature or transaction flow.
  - If the Transmitter, _Tx, property is False and the Receiver, _Rx, property is True, the Receiver might be able to save power by turning off certain portions of logic.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- If the Transmitter, _Tx, and Receiver, _Rx, properties have values other than True or False, and these values differ between both sides of the link, additional statements will describe the expected behavior.
- If the Uniform properties on either side of the link do not match, additional statements will describe the expected behavior.

The properties in opposing directions on a link can differ. That is, it may be possible that certain transactions can be initiated in one direction, but not the other.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B14.2 On-chip payload properties

This section describes the properties for on-chip payload, including:

- B14.2.1 *Uniform properties*
- B14.2.2 *Receiver and Transmitter properties*
- B14.2.3 *Expectations when exchanged on-chip payload properties differ*

B14.2.1 Uniform properties

There are no on-chip payload Uniform properties.

B14.2.2 Receiver and Transmitter properties

Table B14.1 lists the properties that are used to describe specific on-chip field payload characteristics of the chip as a Receiver.

Table B14.1: Receiver on-chip payload properties

Property	Width (bits)	Issue introduced	Description
MPAM_Support_Rx	2	A	This property indicates support for MPAM. `0b00` False. The MPAM field is not present. `0b01` True. The MPAM field is present. Others Reserved.
PBHA_Support_Rx	2	A	This property indicates support for PBHA. `0b00` False. The PBHA field is not present. `0b01` True. The PBHA field is present. Others Reserved.
MEC_Support_Rx	2	A	This property indicates support for MEC. `0b00` False. MECID field is not present. `0b01` True. MECID field is present. Others Reserved
RSVDC_REQ_Rx	4	A	This property indicates the width of the RSVDC field on the REQ channel. `0b0000` 0 bits `0b0001` 4 bits `0b0010` 8 bits `0b0011` 12 bits `0b0100` 16 bits `0b0101` 24 bits `0b0110` 32 bits Others Reserved

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.1 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
RSVDC_DAT_Rx	4	A	This property indicates the width of the RSVDC field on the DAT channel. `0b0000` 0 bits `0b0001` 4 bits `0b0010` 8 bits `0b0011` 12 bits `0b0100` 16 bits `0b0101` 24 bits `0b0110` 32 bits Others Reserved
DataTarget_Support_Rx	2	B	This property indicates what level of support an interface has for the CHI DataTarget Request field. `0b00` Baseline. The interface supports use of the CHI Issue G DataTarget Request field, as defined in the *AMBA® CHI Architecture Specification Issue G* [1]. `0b01` v2. The interface supports use of the CHI Issue H DataTarget Request field, as defined in the *AMBA® CHI Architecture Specification Issue H* [1]. Others Reserved.

Table B14.2 lists the properties that are used to describe specific on-chip field payload characteristics of the chip as a Transmitter.

Table B14.2: Transmitter on-chip payload properties

Property	Width (bits)	Issue introduced	Description
MPAM_Support_Tx	2	A	This property indicates support for MPAM. `0b00` False. The MPAM field is not present. `0b01` True. The MPAM field is present. Others Reserved.
PBHA_Support_Tx	2	A	This property indicates support for PBHA. `0b00` False. The PBHA field is not present. `0b01` True. The PBHA field is present. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.2 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
MEC_Support_Tx	2	A	This property indicates support for MEC. `0b00` False. MECID field is not present. `0b01` True. MECID field is present. Others Reserved
RSVDC_REQ_Tx	4	A	This property indicates the width of RSVDC on the REQ channel. `0b0000` 0 bits `0b0001` 4 bits `0b0010` 8 bits `0b0011` 12 bits `0b0100` 16 bits `0b0101` 24 bits `0b0110` 32 bits Others Reserved
RSVDC_DAT_Tx	4	A	This property indicates the width of RSVDC on the DAT channel. `0b0000` 0 bits `0b0001` 4 bits `0b0010` 8 bits `0b0011` 12 bits `0b0100` 16 bits `0b0101` 24 bits `0b0110` 32 bits Others Reserved
DataTarget_Support_Tx	2	B	This property indicates what level of support an interface has for the CHI DataTarget Request field. `0b00` Baseline. The interface supports use of the CHI Issue G DataTarget Request field, as defined in the *AMBA® CHI Architecture Specification Issue G* [1]. `0b01` v2. The interface supports use of the CHI Issue H DataTarget Request field, as defined in the *AMBA® CHI Architecture Specification Issue H* [1]. Others Reserved.

B14.2.3 Expectations when exchanged on-chip payload properties differ

Table B14.3 details the expected outcome when on-chip payload properties differ between two sides of a link.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.3: Expectations when exchanged on-chip payload properties differ

Property	Expectations
Received DataTarget_Support_Rx and sent DataTarget_Support_Tx	If the received DataTarget_Support_Rx value is Baseline and the sent DataTarget_Support_Tx is v2, the Transmitter must ensure that the Unique subfield is 0, and that the CacheLevel subfield is set to non-zero values only when allowed by the CHI Issue G protocol [1]. If the received DataTarget_Support_Rx is v2 and the sent DataTarget_Support_Tx value is Baseline, there are no functional issues.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B14.3 On-chip opcode and flow properties

This section describes the properties for on-chip opcode and flow, including:

- B14.3.1 *Uniform properties*
- B14.3.2 *Receiver and Transmitter properties*
- B14.3.3 *Expectations when on-chip opcode and flow properties differ*

B14.3.1 Uniform properties

There are no on-chip Uniform opcode and flow properties.

B14.3.2 Receiver and Transmitter properties

Table B14.4 lists the properties that are used to describe specific on-chip opcode and flow support of the chip as a Receiver.

Table B14.4: Receiver on-chip opcode or flow properties

Property	Width (bits)	Issue introduced	Description
Snoop_Transactions_Rx	2	A	This property indicates support for Snoop transactions. When this property negotiates to False, the Transmitter is not allowed to send any snoopable allocating requests. `0b00` False. Snoop transactions are not supported. `0b01` True. Snoop transactions are supported. Others Reserved.
DVM_Support_Rx	2	A	This property indicates support for DVM messages. `0b00` False. DVM transactions are not supported. `0b01` True. DVM transactions are supported. Others Reserved.
Atomic_Transactions_Rx	2	A	This property indicates support for Atomic transactions. `0b00` False. Atomic transactions are not supported. `0b01` True. Atomic transactions are supported. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.4 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
Cache_Stash_Transactions_Rx	2	A	This property indicates support for Stashing requests and snoops. `0b00` False. Stash transactions are not supported. `0b01` True. Stash transactions are supported. Others Reserved. **Note** Issue A included a single Cache_Stash_Transactions_Rx property. Issue B introduces separate Cache_Stash_Requests_Rx and Cache_Stash_Snoops_Rx properties, while also retaining the single Cache_Stash_Transactions_Rx property for backwards compatibility. Implementations compatible with Issue B of this specification must advertise Cache_Stash_Transactions_Rx to True if both Cache_Stash_Requests_Rx and Cache_Stash_Snoops_Rx are True. Otherwise, Cache_Stash_Transactions_Rx must be advertised to False.
Persist_Transactions_Rx	2	A	This property indicates support for CMO transactions to the PoP and PoDP. `0b00` False. Persist CMO transactions are not supported. `0b01` True. Persist CMO transactions are supported. Others Reserved.
MTE_Support_Rx	2	A	This property indicates support for MTE. `0b00` False. MTE transactions are not supported. `0b01` Reduced MTE transactions are supported. MTE transactions with the TagOp field set to *Match* are not supported. MTE transactions with other TagOp field values are supported. `0b10` Full MTE transactions are supported. MTE transactions with any TagOp field value are supported. `0b11` Reserved
Deferrable_Write_Rx	2	A	This property indicates support for WriteNoSnpDef. `0b00` False. Deferrable Write transactions are not supported. `0b01` True. Deferrable Write transactions are supported. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.4 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
RME_Support_Rx	2	A	This property indicates support for Realm Management Extensions, including Realm and Root Physical Address Spaces and PoPA Cache Maintenance Operations. `0b00` False. RME transactions are not supported. `0b01` True. RME transactions are supported. Others Reserved.
CMO_Transactions_Rx	2	A	This property indicates support for CleanShared, CleanInvalid, and MakeInvalid CMO transactions, and their combined write variants. This property also affects other properties that control different CMO transaction types. `0b00` False. CleanShared, CleanInvalid and MakeInvalid CMO transactions and their combined variants are not supported. No other CMO transactions or combined variants are supported, regardless of the value of associated properties. `0b01` True. CleanShared, CleanInvalid and MakeInvalid CMO transactions and their combined variants are supported. Support for other CMO transactions and their combined variants are controlled by individual properties. Others Reserved
Cache_Stash_Requests_Rx	2	B	This property indicates whether Cache Stash requests are supported. `0b00` False. Cache Stash requests are not supported. `0b01` True. Cache Stash requests are supported. Others Reserved.
Cache_Stash_Snoops_Rx	2	B	This property indicates whether Cache Stash snoops are supported. `0b00` False. Cache Stash snoops are not supported. `0b01` True. Cache Stash snoops are supported. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.4 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
MultiReq_WritePull_Support_Rx	4	B	This property indicates the maximum length of a Multi-request that the interface supports for a WritePull transaction flow. WritePull transaction flows include WriteNoSnpFull, WriteNoSnpPtl, WriteNoSnpZero, WriteUniqueFull, WriteUniquePtl, and WriteUniqueZero. `0b0000` False. Multi-request transactions are not supported. MultiReq must be 0. `0b0001` Up to 2 requests. Multi-request transactions must not cross a 128-byte boundary. `0b0010` Up to 4 requests. Multi-request transactions must not cross a 256-byte boundary. `0b0011` Up to 8 requests. Multi-request transactions must not cross a 512-byte boundary. `0b0100` Up to 16 requests. Multi-request transactions must not cross a 1KB boundary. `0b0101` Up to 32 requests. Multi-request transactions must not cross a 2KB boundary. `0b0110` Up to 64 requests. Others Reserved.
MultiReq_WritePush_Support_Rx	4	B	This property indicates the maximum length of a Multi-request that the interface supports for a WritePush transaction flow. WritePush transaction flows include WriteNoSnpFull, WriteNoSnpPtl, WriteUniqueFull, and WriteUniquePtl. `0b0000` False. Multi-request transactions are not supported. MultiReq must be 0. `0b0001` Up to 2 requests. Multi-request transactions must not cross a 128-byte boundary. `0b0010` Up to 4 requests. Multi-request transactions must not cross a 256-byte boundary. `0b0011` Up to 8 requests. Multi-request transactions must not cross a 512-byte boundary. `0b0100` Up to 16 requests. Multi-request transactions must not cross a 1KB boundary. `0b0101` Up to 32 requests. Multi-request transactions must not cross a 2KB boundary. `0b0110` Up to 64 requests. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.4 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
MultiReq_NonWrite_Support_Rx	4	B	This property indicates the maximum length of a Multi-request that the interface supports for a non-write transaction flow. Non-write transaction flows include ReadNoSnp, ReadNoSnpSep, ReadOnce, ReadOnceCleanInvalid, and ReadOnceMakeInvalid. `0b0000` False. Multi-request transactions are not supported. MultiReq must be 0. `0b0001` Up to 2 requests. Multi-request transactions must not cross a 128-byte boundary. `0b0010` Up to 4 requests. Multi-request transactions must not cross a 256-byte boundary. `0b0011` Up to 8 requests. Multi-request transactions must not cross a 512-byte boundary. `0b0100` Up to 16 requests. Multi-request transactions must not cross a 1KB boundary. `0b0101` Up to 32 requests. Multi-request transactions must not cross a 2KB boundary. `0b0110` Up to 64 requests. Others Reserved.
CacheLineID_Accurate_Rx	2	B	This property indicates whether the Receiver requires that all messages for which CacheLineID is applicable have accurate content in the field. When at least one of the MultiReq_WritePull_Support_Tx, MultiReq_WritePush_Support_Tx, MultiReq_NonWrite_Support_Tx properties is not False, CacheLineID_Accurate_Rx must be True. `0b00` False. The interface does not require the received CacheLineID field content to be accurate in applicable message types. `0b01` True. The interface requires the received CacheLineID field content to be accurate in applicable message types. Others Reserved.
Physical_Storage_Transactions_Rx	2	B	This property indicates whether an interface supports the receipt of CMO transactions to the PoPS. `0b00` False. The interface does not support CMO transactions to the PoPS. `0b01` True. The interface supports CMO transactions to the PoPS. Others Reserved.

Table B14.5 lists the properties that are used to describe specific on-chip opcode and flow support of the chip as a

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Transmitter.

Table B14.5: Transmitter on-chip opcode or flow properties

Property	Width (bits)	Issue introduced	Description
Snoop_Transactions_Tx	2	A	This property indicates support for Snoop transactions. `0b00` False. Will not send Snoop transactions. `0b01` True. Can send Snoop transactions. Others Reserved.
DVM_Support_Tx	2	A	This property indicates support for DVM messages. `0b00` False. Will not send DVM transactions. `0b01` True. Can send DVM transactions. Others Reserved.
Atomic_Transactions_Tx	2	A	This property indicates support for Atomic transactions. `0b00` False. Will not send Atomic transactions. `0b01` True. Can send Atomic transactions. Others Reserved.
Cache_Stash_Transactions_Tx	2	A	This property indicates support for Stashing requests and snoops. `0b00` False. Will not send Stash transactions. `0b01` True. Can send Stash transactions. Others Reserved. **Note** Issue A included a single Cache_Stash_Transactions_Tx property. Issue B introduces separate Cache_Stash_Requests_Tx and Cache_Stash_Snoops_Tx properties, whilst retaining Cache_Stash_Transactions_Tx for backwards compatibility. Implementations compatible with Issue B must advertise Cache_Stash_Transactions_Rx to True if either the Cache_Stash_Requests_Tx or Cache_Stash_Snoops_Tx, or both, are True. Otherwise, Cache_Stash_Transactions_Tx property must be advertised to False.
Persist_Transactions_Tx	2	A	This property indicates support for CMO transactions to the PoP and PoDP. `0b00` False. Will not send Persist CMO transactions. `0b01` True. Can send Persist CMO transactions. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.5 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
MTE_Support_Tx	2	A	This property indicates support for MTE. `0b00` False. Will only send transactions with TagOp *Invalid*. `0b01` Reduced MTE transaction support. Will not send transactions with TagOp *Match*. Can send transactions with other TagOp values. `0b10` Full MTE transaction support. Can send transactions with any TagOp value. `0b11` Reserved.
Deferrable_Write_Tx	2	A	This property indicates support for WriteNoSnpDef. `0b00` False. Will not send Deferrable Write transactions. `0b01` True. Can send Deferrable Write transactions. Others Reserved.
RME_Support_Tx	2	A	This property indicates support for Realm Management Extensions, including Realm and Root Physical Address Spaces and PoPA Cache Maintenance Operations. `0b00` False. Will not send RME transactions. `0b01` True. Can send RME transactions. Others Reserved.
CMO_Transactions_Tx	2	A	This property indicates support for sending CleanShared, CleanInvalid, and MakeInvalid CMO transactions, and their combined write variants. This property also affects other properties that control different CMO transaction types. `0b00` False. CleanShared, CleanInvalid and MakeInvalid CMO transactions and their combined variants will not be sent. No other CMO transactions or combined variants will be sent, regardless of the value of associated properties. `0b01` True. CleanShared, CleanInvalid and MakeInvalid CMO transactions and their combined variants can be sent. Support for sending other CMO transactions and their combined variants are controlled by individual properties. Others Reserved
Cache_Stash_Requests_Tx	2	B	This property indicates whether Cache Stash requests are supported. `0b00` False. Will not send Stash requests. `0b01` True. Can send Stash requests. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.5 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
Cache_Stash_Snoops_Tx	2	B	This property indicates whether Cache Stash snoops are supported. `0b00` False. Will not send Stash snoops. `0b01` True. Can send Stash snoops. Others Reserved.
MultiReq_WritePull_Support_Tx	4	B	This property indicates the maximum length of a Multi-request that the interface supports for a WritePull transaction flow. WritePull transaction flows include WriteNoSnpFull, WriteNoSnpPtl, WriteNoSnpZero, WriteUniqueFull, WriteUniquePtl, and WriteUniqueZero. `0b0000` False. Multi-request transactions will not be sent. MultiReq must be 0. `0b0001` Up to 2 requests. Multi-request transactions will not cross a 128-byte boundary. `0b0010` Up to 4 requests. Multi-request transactions will not cross a 256-byte boundary. `0b0011` Up to 8 requests. Multi-request transactions will not cross a 512-byte boundary. `0b0100` Up to 16 requests. Multi-request transactions will not cross a 1KB boundary. `0b0101` Up to 32 requests. Multi-request transactions will not cross a 2KB boundary. `0b0110` Up to 64 requests. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.5 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
MultiReq_WritePush_Support_Tx	4	B	This property indicates the maximum length of a Multi-request that the interface supports for a WritePush transaction flow. WritePush transaction flows include WriteNoSnpFull, WriteNoSnpPtl, WriteUniqueFull, and WriteUniquePtl. `0b0000` False. Multi-request transactions will not be sent. MultiReq must be 0. `0b0001` Up to 2 requests. Multi-request transactions will not cross a 128-byte boundary. `0b0010` Up to 4 requests. Multi-request transactions will not cross a 256-byte boundary. `0b0011` Up to 8 requests. Multi-request transactions will not cross a 512-byte boundary. `0b0100` Up to 16 requests. Multi-request transactions will not cross a 1KB boundary. `0b0101` Up to 32 requests. Multi-request transactions will not cross a 2KB boundary. `0b0110` Up to 64 requests. Others Reserved.
MultiReq_NonWrite_Support_Tx	4	B	This property indicates the maximum length of a Multi-request that the interface supports for a non-write transaction flow. Non-write transaction flows include ReadNoSnp, ReadNoSnpSep, ReadOnce, ReadOnceCleanInvalid, and ReadOnceMakeInvalid. `0b0000` False. Multi-request transactions will not be sent. MultiReq must be 0. `0b0001` Up to 2 requests. Multi-request transactions will not cross a 128-byte boundary. `0b0010` Up to 4 requests. Multi-request transactions will not cross a 256-byte boundary. `0b0011` Up to 8 requests. Multi-request transactions will not cross a 512-byte boundary. `0b0100` Up to 16 requests. Multi-request transactions will not cross a 1KB boundary. `0b0101` Up to 32 requests. Multi-request transactions will not cross a 2KB boundary. `0b0110` Up to 64 requests. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.5 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
CacheLineID_Accurate_Tx	2	B	This property indicates whether all messages issued by a Transmitter for which CacheLineID is applicable have accurate content in the field. When at least one of the MultiReq_WritePull_Support_Rx, MultiReq_WritePush_Support_Rx, MultiReq_NonWrite_Support_Rx properties is not False, CacheLineID_Accurate_Tx must be True. `0b00` False. The CacheLineID field for applicable messages issued by this interface must not be considered accurate. `0b01` True. All messages for which the CacheLineID field is applicable transmitted by this interface will include accurate content in the field. Others Reserved.
Physical_Storage_Transactions_Tx	2	B	This property indicates whether an interface supports sending CMO operations to the PoPS. `0b00` False. The interface will not send CMO operations to the PoPS. `0b01` True. The interface can send CMO operations to the PoPS. Others Reserved.

B14.3.3 Expectations when on-chip opcode and flow properties differ

Table B14.6 details the expected outcome when on-chip opcode or flow properties differ between two sides of a link.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.6: Expectations when exchanged on-chip opcode or flow properties differ

Property	Expectations
Received MultiReq_WritePull_Support_Rx and sent MultiReq_WritePull_Support_Tx	If the received MultiReq_WritePull_Support_Rx value is greater than the sent MultiReq_WritePull_Support_Tx, there are no functional issues. If the received MultiReq_WritePull_Support_Rx value is less than the sent MultiReq_WritePull_Support_Tx value, the Transmitter must not: – Issue larger multi-request WritePull transactions than indicated by MultiReq_WritePull_Support_Rx. – Generate any multi-request WritePull transaction that crosses the byte-boundary limit indicated by the received MultiReq_WritePull_Support_Rx. **Note** If the Transmitter receives larger transactions locally, the transactions must be broken down into smaller multi-request transactions before being sent across the link. If the received MultiReq_WritePull_Support_Rx value is False, the multi-request WritePull message formats must not be used by the Transmitter.
Received MultiReq_WritePush_Support_Rx and sent MultiReq_WritePush_Support_Tx	If the received MultiReq_WritePush_Support_Rx value is greater than the sent MultiReq_WritePush_Support_Tx, there are no functional issues. If the received MultiReq_WritePush_Support_Rx value is less than the sent MultiReq_WritePush_Support_Tx value, the Transmitter must not: – Issue larger multi-request WritePush transactions than indicated by MultiReq_WritePush_Support_Rx. – Generate any multi-request WritePush transaction that crosses the byte-boundary limit indicated by the received MultiReq_WritePush_Support_Rx. **Note** If the Transmitter receives larger transactions locally, the transactions must be broken down into smaller multi-request transactions before being sent across the link. If the received MultiReq_WritePush_Support_Rx value is False, the multi-request WritePush message formats must not be used by the Transmitter.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.6 – *Continued from previous page*

Property	Expectations
Received MultiReq_NonWrite_Support_Rx and sent MultiReq_NonWrite_Support_Tx	If the received MultiReq_NonWrite_Support_Rx value is greater than the sent MultiReq_NonWrite_Support_Tx, there are no functional issues. If the received MultiReq_NonWrite_Support_Rx value is less than the sent MultiReq_NonWrite_Support_Tx value, the Transmitter must not: – Issue larger multi-request NonWrite transactions than indicated by MultiReq_NonWrite_Support_Rx. – Generate any multi-request NonWrite transaction that crosses the byte-boundary limit indicated by the received MultiReq_NonWrite_Support_Rx. **Note** If the Transmitter receives larger transactions locally, the transactions must be broken down into smaller multi-request transactions before being sent across the link. If the received MultiReq_NonWrite_Support_Rx value is False, the multi-request non-write message formats must not be used by the Transmitter.
Received CacheLineID_Accurate_Tx and sent CacheLineID_Accurate_Rx	If the received CacheLineID_Accurate_Tx is False and the sent CacheLineID_Accurate_Rx is True, the Receiver on this interface needs to either include logic to correctly generate the CacheLineID field or downgrade its requirements for CacheLineID in a way that is compatible with the on-chip CHI protocol.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B14.4 C2C-specific properties

This section describes C2C specific properties, including:

- B14.4.1 *Uniform properties*
- B14.4.2 *Receiver and Transmitter properties*
- B14.4.3 *Expectations when exchanged C2C specific properties differ*

This section also describes the format of IwT entries of an individual address decoder, based on the C2C_AddrDecCtrlStatus.TgtType value.

B14.4.1 Uniform properties

Table B14.7 lists the Uniform properties that are C2C-specific.

Table B14.7: Uniform C2C-specific properties

Property	Width (bits)	Issue introduced	Description
Num_Properties_Msg	4	A	This property indicates the number of MiscU.Properties messages per Logical Link that can be sent and accepted by the interface. `0b0000` 1 MiscU.Properties message. `0b0001` 2 MiscU.Properties messages. Others Reserved.
Protocol	4	A	This property indicates which protocol is supported by the interface. `0b0000` CHI C2C. Others Reserved.
Version	4	A	This property indicates which specification version is supported by the interface. `0b0000` Issue A. `0b0001` Issue B. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.7 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
Container_Format	4	A	This property indicates which container formats are supported by the interface. The following bits are defined: **Bits[3]** Reserved. **Bit[2]** Format Z. `0b0` Not supported. `0b1` Supported. **Bit[1]** Format Y. `0b0` Not supported. `0b1` Supported. **Bit[0]** Format X. `0b0` Not supported. `0b1` Supported. For the C2C_PropSuppUniform1, C2C_PropAdvdUniform1 and C2C_PropInfdUniform1 registers, multiple bits can be asserted at the same time. For the C2C_PropNgtdUniform1 register, only a single bit can be asserted.
Deactivation_Support	4	A	This property indicates which deactivation modes are supported by the interface. The following bits are defined: **Bits[3:2]** Reserved. **Bit[1]** Protocol aware. `0b0` Not supported. `0b1` Supported. **Bit[0]** Protocol agnostic. `0b0` Not supported. `0b1` Supported.
PortID	8	B	This property indicates the PortID assigned to this interface within its chip. See B13.3 *PortID exchange* for more detail.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.7 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
IDE_G0_Reservation	2	B	This property indicates support for the reservation of granule message rollover rules. `0b00` False. The interface is not capable of using the reservation of granule G0 or using the associated message rollover rules. The G0IDECtrl container header bits must not be used and the modified message rollover rules defined in B11.2.2 *IDE message rollover rules* do not apply. The traffic for the associated Logical Link cannot traverse an IDE-enabled link. `0b01` True. The interface is capable of using the reservation of granule G0 for MAC insertion and using the associated message rollover rules. The Transmitter interface can use the G0IDECtrl container header bit to inform the Receiver about granule G0 content and granule position of any protocol message rollover. The Receiver must check the {G0IDECtrl, MsgStart[0]} container header bits to understand whether G0 has valid protocol content, and, if a rollover is expected, which is the starting granule for the rollover between G0 and G1. Others Reserved.

See B15.11.7 *Uniform property registers* for Uniform1 and Uniform2 property registers.

B14.4.2 Receiver and Transmitter properties

Table B14.8 lists the Receiver properties that are C2C-specific.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.8: Receiver C2C-specific properties

Property	Width (bits)	Issue introduced	Description
Num_RP_REQ_Rx	4	A	This property indicates the maximum number of Resource Planes the interface supports on the REQ channel. `0b0000` 1 `0b0001` 2 `0b0010` 3 `0b0011` 4 `0b0100` 5 `0b0101` 6 `0b0110` 7 `0b0111` 8 Others Reserved.
WritePush_Support_Rx	2	A	This property indicates whether an interface supports WritePush flows for write transactions. `0b00` False. WritePush is not supported for Immediate Writes and their combined CMO variants, or Atomic transactions. `0b01` True. WritePush is supported for Immediate Writes and their combined CMO variants, and Atomic transactions. Others Reserved.
DVMPush_Support_Rx	2	A	This property indicates whether an interface supports a WritePush flow for DVM transactions. This property is inapplicable when DVM_Support_Rx is False. `0b00` False. A WrReqDataL message is not supported for DVM transactions. `0b01` True. A WrReqDataL message can be used for DVM transactions. Others Reserved.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.8 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
DevAssign_Support_Rx	4	A	This property indicates support for RME-DA or RME-CDA. This property is not expected to be used in isolation to establish trust in the link. Additional attestation and software configuration is expected. The following bits are defined for the roles of the other chip to which this chip can connect: **Bit[3]** Reserved. **Bit[2]** Device_NoStreamID_NoSecSID1 `0b0` Must not connect to a device. Does not use SecSID1 and StreamID fields. `0b1` Can connect to a device. Does not use SecSID1 and StreamID fields. **Bit[1]** Device_StreamID_SecSID1 `0b0` Must not connect to a device. Does not use SecSID1 and StreamID fields. `0b1` Can connect a device. Can make use of SecSID1 and StreamID fields. **Bit[0]** Host `0b0` Must not connect to a host. `0b1` Can connect to a host.
Event_Support_Rx	2	B	This property indicates whether an interface supports the receipt of MiscU.Event messages. See B5.9.6 *Event message format* for more details. `0b00` False. The interface does not support the receipt of MiscU.Event requests. `0b01` True. The interface supports the receipt of MiscU.Event requests and the sending of required MiscU.Event acknowledgements. Others Reserved.

Table B14.9 lists the Transmitter properties that are C2C-specific.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.9: Transmitter C2C-specific properties

Property	Width (bits)	Issue introduced	Description
Num_RP_REQ_Tx	4	A	This property indicates the maximum number of Resource Planes the interface supports on the REQ channel. `0b0000` 1 `0b0001` 2 `0b0010` 3 `0b0011` 4 `0b0100` 5 `0b0101` 6 `0b0110` 7 `0b0111` 8 Others Reserved.
WritePush_Support_Tx	2	A	This property indicates whether an interface supports WritePush flows for write transactions. `0b00` False. Will not use WritePush for Immediate Writes and their combined CMO variants, or Atomic transactions. `0b01` True. Can use WritePush for Immediate Writes and their combined CMO variants, or Atomic transactions. Others Reserved.
DVMPush_Support_Tx	2	A	This property indicates whether an interface supports a WritePush flow for DVM transactions. This property is inapplicable when DVM_Support_Tx is False. `0b00` False. Will not use a WrReqDataL message for DVM transactions. `0b01` True. Can use a WrReqDataL message for DVM transactions. Others Reserved

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.9 – *Continued from previous page*

Property	Width (bits)	Issue introduced	Description
DevAssign_Support_Tx	4	A	This property indicates support for RME-DA or RME-CDA. This property is not expected to be used in isolation to establish trust in the link. Additional attestation and software configuration is expected. The following bits are defined for the roles of that this chip can present to the other chip: **Bit[3]** Reserved. **Bit[2]** Device_NoStreamID_NoSecSID1 `0b0` Cannot present as a device. Does not send SecSID1 and StreamID fields. `0b1` Can present as a device. Does not send SecSID1 and StreamID fields. **Bit[1]** Device_StreamID_SecSID1 `0b0` Cannot present as a device. Does not send SecSID1 and StreamID fields. `0b1` Can present as a device. Sends SecSID1 and StreamID fields. **Bit[0]** Host `0b0` Cannot present as a host. `0b1` Can present as a host.
Event_Support_Tx	2	B	This property indicates whether an interface supports sending MiscU.Event messages. See B5.9.6 *Event message format* for more details. `0b00` False. Will not send MiscU.Event requests. `0b01` True. Can send MiscU.Event requests and receive the required MiscU.Event acknowledgements. Others Reserved.

Table B14.9 lists the Transmitter properties that are C2C-specific.

B14.4.3 Expectations when exchanged C2C specific properties differ

Table B14.10 details the expected outcome when C2C-specific properties differ between two sides of a link.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.10: Expectations when exchanged C2C-specific properties differ

Property	Expectations
Protocol	If the remote interface supports a different protocol to the local interface, then there might be an incompatibility. Only one protocol is supported in this version of the specification.
Version	A newer version of the interface is expected to be backward compatible with older version. This property is not expected to have any effect on other individual properties. Optional features are controlled with individually-negotiated properties and, when a feature is only supported by one of the connected interfaces, the negotiation will result in the feature being disabled.
Num_Properties_Msg	If the received Num_Properties_Msg is less than the sent Num_Properties_Msg, the interface must only send, and expect to receive, a number of MiscU.Properties messages that match the received Num_Properties_Msg. Properties defined in messages beyond the received Num_Properties_Msg are assumed to be implicitly set to all zeroes. If the received Num_Properties_Msg is greater than the sent Num_Properties_Msg, no special action is required.
Container_Format	See B14.4.3.1 *Container_Format resolution*.
Deactivation_Support	If none of the property bit position assertions match, deactivation is not supported by the protocol. If only one of the property bit position assertions matches, the associated mode is the only one supported. If multiple property bit position assertions match, any of associated deactivation approaches can be used. However, to ensure both sides use consistent approaches, higher-level software coordination may be required.
PortID	The received PortID value is not expected to match the sent PortID value. This has no functional effect, as the required result of the negotiation is not defined for the PortID property. See B13.3 *PortID exchange* for more information.
Received Num_RP_REQ_Rx and sent Num_RP_REQ_Tx	If the received Num_RP_REQ_Rx value is greater than the sent Num_RP_REQ_Tx value, there should not be any functional issues. The Transmitter might receive credits for Resource Planes that it does not intend to use. If the received Num_RP_REQ_Rx value is less than the sent Num_RP_REQ_Tx value, there might be functional issues if the Transmitter chip cannot consolidate traffic onto the reduced number of Resource Planes. The Transmitter will not receive credits for Resource Planes that it potentially needs to use for traffic dependency isolation. This should be considered at chip or chiplet selection and build time.

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.10 – *Continued from previous page*

Property	Expectations
Received Num_RP_REQ_Tx and sent Num_RP_REQ_Rx	If the received Num_RP_REQ_Tx value is less than the sent Num_RP_REQ_Rx value, there should not be any functional issues. The Receiver is permitted, but not required, to redistribute the credits that it would have provided for the unused Resource Planes. If the received Num_RP_REQ_Tx value is greater than the sent Num_RP_REQ_Rx value, there is no expected change in behavior by the Receiver.
Received DevAssign_Support_Rx and sent DevAssign_Support_Tx	If none of the DevAssign_Support bits match, it may not be possible to establish the link as RME-trusted.
Received DevAssign_Support_Tx and sent DevAssign_Support_Rx	If none of the DevAssign_Support bits match, it may not be possible to establish the link as RME-trusted.

B14.4.3.1 Container_Format resolution

This section describes the two processes for determining the container format to use on a single-hop Logical link and on a Pass-through Multihop Logical link.

B14.4.3.2 Single-hop Logical Link

There are two steps for the determination of the container format to be used on the link:

1. The container formats that are supported by the Protocol layer are compared with the requirements of the FlitFormat needed by the Link layer.

   After polling on the C2C_LogLinkCtrlStatus1.InitState[0] bit, the software sets the Container_Format property in the C2C_PropAdvdUniform1 register according to the resolution described in Table B14.11.

   Note

   The C2C_PropAdvdUniform1.Container_Format register field is allowed to advertise multiple formats.

   Table B14.11 shows the container format resolution against the Link layer flit format.

   The following key is used:

   - Not compatible.

Table B14.11: Container format resolution against Link layer flit format

Container_Format from C2C_PropAdvdUniform1	C2C_PortCtrlStatus1.LinkLayerFlitFormat		
	6 bytes	20 bytes	16 bytes
Format X	Format X	-	-
Format Y	Format Y	Format Y	-
Format Z	Format Z	-	Format Z
Format X and Format Y	Format X and Format Y	Format Y	-
Format Y and Format Z	Format Y and Format Z	Format Y	Format Z

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.11 – *Continued from previous page*

Container_Format from C2C_PropAdvdUniform1	**C2C_PortCtrlStatus1.LinkLayerFlitFormat**		
	6 bytes	**20 bytes**	**16 bytes**
Format X and Format Z	Format X and Format Z	-	Format Z
Format X, Format Y, and Format Z	Format X, Format Y, and Format Z	Format Y	Format Z

2. If required, the two Protocol layers exchange properties, and the Container_Format property is negotiated in hardware to determine which format will be used on the link.

   To determine the result of the negotiation, the following criteria are defined:

   a. If both sides of the link have advertised support for Format X, Format X will be the result of the negotiation.

   b. If the condition in (a) is not met, but both sides have advertised support for Format Z, Format Z will be the result of the negotiation.

   c. If the conditions in (a) and (b) are not met, but both sides have advertised support for Format Y, Format Y will be the result of the negotiation.

   d. For all other instances, this does not result in a successful negotiation.

Note

The C2C_PropNgtdUniform1.Container_Format register field is required to encode, at most, one format. This indicates the container format to be used on the link.

B14.4.3.3 Multihop Logical Link

The process for container format resolution on the Pass-through Multihop Logical Links is part of the software bring-up sequence of a Multihop path.

The expected steps for container format resolution are:

1. The software identifies the supported container formats for the Protocol layers at the two ends of the Logical Link. That is, the formats supported by the packer and unpacker for the Logical Link. This can be done by reading the respective C2C_PropSuppUniform1 registers.
2. The software identifies the supported Link layer flit formats on each intermediate physical link that is part of the Multihop path, from the C2C_PortCtrlStatus1.LinkLayerFlitFormat register field at each port. An overall Link layer flit format across the entire Multihop path can be defined as follows:
   - If a 6-byte flit format is supported on all physical links on the path, the overall Link layer flit format requirement is 6-bytes.
   - If any physical link on the path does not support 6-byte flit format, but all physical links support either 6-byte or 16-byte flit formats, the overall Link layer flit format requirement is 16-byte.
   - If any physical link on the path does not support 6-byte flit format, but all physical links support either 6-byte or 20-byte flit formats, the overall Link layer flit format requirement is 20-byte.
   - For other instances, it is not possible to establish a Multihop path across the considered set of physical links.
3. The software sets the Container_Format property in the C2C_PropAdvdUniform1 register on each of the two ends of the Logical Link, combining the packer and unpacker supported container formats, identified at step (1), with the overall Link layer flit format requirement, identified at step (2). Table B14.11 can be used to determine the compatibility criteria.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

4. If required, the properties are exchanged between the two Protocol layers for the Logical Link.

Note

It is possible that other Logical Links originating from the same source chip and routed to a different target chip use different container formats.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B14.5 Property packing in MiscU.Properties messages

Table B14.12 shows the packing of properties in the payload of MiscU.Properties(1) message.

Table B14.12: MiscU.Properties(1) message format

MISC bit position	Field name	Width (bits)	Value	Property classification
3:0	MsgType	4	MiscU	Not applicable
7:4	MiscOp	4	Properties	
11:8	Num_Properties_Msg	4		Uniform properties
15:12	Protocol	4		
19:16	Version	4		
23:20	Container_Format	4		
27:24	Deactivation_Support	4		
29:28	IDE_G0_Reservation	2		
39:30	Reserved	10		
43:40	Num_RP_REQ_Rx	4		Receiver properties
47:44	Reserved	4		
49:48	WritePush_Support_Rx	2		
51:50	DVMPush_Support_Rx	2		
53:52	Snoop_Transactions_Rx	2		
55:54	DVM_Support_Rx	2		
57:56	Atomic_Transactions_Rx	2		
59:58	Cache_Stash_Transactions_Rx	2		
61:60	Persist_Transactions_Rx	2		
63:62	MTE_Support_Rx	2		
65:64	Deferable_Write_Rx	2		
67:66	RME_Support_Rx	2		
69:68	CMO_Transactions_Rx	2		
71:70	MPAM_Support_Rx	2		
73:72	PBHA_Support_Rx	2		
75:74	MEC_Support_Rx	2		
79:76	RSVDC_REQ_Rx	4		
83:80	RSVDC_DAT_Rx	4		
87:84	DevAssign_Support_Rx	4		
89:88	Cache_Stash_Requests_Rx	2		
91:90	Cache_Stash_Snoops_Rx	2		

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.12 – *Continued from previous page*

MISC bit position	Field name	Width (bits)	Value	Property classification
99:92	Reserved	8		
103:100	Num_RP_REQ_Tx	4		Transmitter properties
107:104	Reserved	4		
109:108	WritePush_Support_Tx	2		
111:110	DVMPush_Support_Tx	2		
113:112	Snoop_Transactions_Tx	2		
115:114	DVM_Support_Tx	2		
117:116	Atomic_Transactions_Tx	2		
119:118	Cache_Stash_Transactions_Tx	2		
121:120	Persist_Transactions_Tx	2		
123:122	MTE_Support_Tx	2		
125:124	Deferable_Write_Tx	2		
127:126	RME_Support_Tx	2		
129:128	CMO_Transactions_Tx	2		
131:130	MPAM_Support_Tx	2		
133:132	PBHA_Support_Tx	2		
135:134	MEC_Support_Tx	2		
139:136	RSVDC_REQ_Tx	4		
143:140	RSVDC_DAT_Tx	4		
147:144	DevAssign_Support_Tx	4		
149:148	Cache_Stash_Requests_Tx	2		
151:150	Cache_Stash_Snoops_Tx	2		
159:152	Reserved	8		

Table B14.13 shows the packing of properties in the payload of MiscU.Properties(2) message.

Table B14.13: MiscU.Properties(2) message format

MISC bit position	Field name	Width (bits)	Value	Property classification
3:0	MsgType	4	MiscU	Not applicable
7:4	MiscOp	4	Properties	
15:8	PortID	8		Uniform properties

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B14.13 – *Continued from previous page*

MISC bit position	Field name	Width (bits)	Value	Property classification
39:16	Reserved	24		
43:40	MultiReq_WritePull_Support_Rx	4		Receiver properties
47:44	MultiReq_WritePush_Support_Rx	4		
51:48	MultiReq_NonWrite_Support_Rx	4		
53:52	CacheLineID_Accurate_Rx	2		
57:54	Reserved	4		
59:58	Event_Support_Rx	2		
61:60	Physical_Storage_Transactions_Rx	2		
63:62	DataTarget_Support_Rx	2		
99:64	Reserved	36		
103:100	MultiReq_WritePull_Support_Tx	4		Transmitter properties
107:104	MultiReq_WritePush_Support_Tx	4		
111:108	MultiReq_NonWrite_Support_Tx	4		
113:112	CacheLineID_Accurate_Tx	2		
117:114	Reserved	4		
119:118	Event_Support_Tx	2		
121:120	Physical_Storage_Transactions_Tx	2		
123:122	DataTarget_Support_Tx	2		
159:124	Reserved	36		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B14.6 Property packing into registers

Each Logical Link has multiple sets of 64-bit registers that contain the properties for use during runtime. The defined sets of registers are detailed in Table B14.14.

Table B14.14: Property register sets

Register sets	Access permissions	Description
Supported	RO	Provides information on the local chip interface directly out of reset
Advertised	RW	A copy of the Supported register that can be software-modified to downgrade the properties that are communicated in the MiscU.Properties messages.
Informed	RO	A copy of the properties received in the MiscU.Properties messages sent by the remote side of the Logical Link.
Negotiated	RO	The outcome following the comparison of each property from both sides of the Logical Link. This can be used by the: – Transmitter to control what messages are sent. – Receiver to enable power saving on logic that will not be utilized.

The format used for Uniform property registers is described in B15.11.7 *Uniform property registers*.

The format used for Receiver property registers is described in B15.11.8 *Receiver property registers*.

The format used for Transmitter property registers is described in B15.11.9 *Transmitter property registers*.

Note

The Informed property registers are defined such that the Rx and Tx suffixes match the property direction indicated by the remote connected interface in the following way:

- The local C2C_PropInfd*Rx registers contain the value of the Receiver property of the remote interface.
- The local C2C_PropInfd*Tx registers contain the value of the Transmitter property of the remote interface.

The Negotiated property registers on the local chip interface are defined in the following way:

- The local C2C_PropNgtd*Tx registers contain the negotiation between the local C2C_PropAdv*Tx and the local C2C_PropInfd*Rx register contents.
- The local C2C_PropNgtd*Rx registers contain the negotiation between the local C2C_PropAdv*Rx and the local C2C_PropInfd*Tx register contents.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B15
Software-visible registers

This chapter describes the software-visible registers. It contains the following sections:

- B15.1 *Introduction*
- B15.2 *Chip offset register block*
- B15.3 *Chip properties register block*
- B15.4 *Chip address decoders register block*
- B15.5 *Chip Port Aggregation Group address decoders register block*
- B15.6 *Chip ID Route Table register block*
- B15.7 *Port register locator register block*
- B15.8 *Port offset register block*
- B15.9 *Port capabilities and control register block*
- B15.10 *Port common Logical Links register block*
- B15.11 *Port individual Logical Links register block*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.1 Introduction

The software-visible registers are grouped into the following categories:

1. Chip.
2. Port.

For the Chip and Port categories, registers are arranged in separate register blocks.

Multiple instances of each chip register block may be included. The included number of instances of each port register block is equal to the number of C2C ports present on the chip.

Figure B15.1 shows how the different chip and port register blocks are arranged in the memory map. It is beyond the scope of this specification to define how the base address of the chip offset register block is determined.

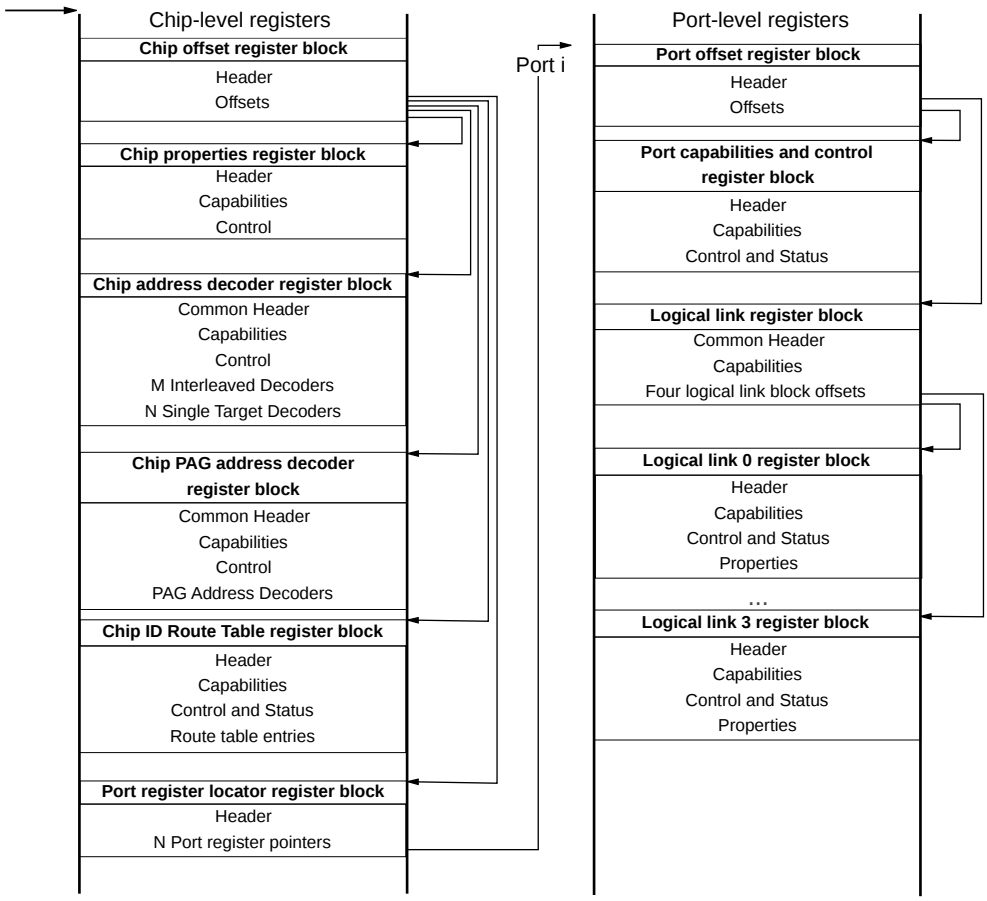


Figure B15.1: Configuration register memory map

Each register block has the following features:

- A 64-bit header. The header contains the version number and size of the block.
- A number of registers for capabilities. The capabilities register contains information regarding hardware capabilities related to the register block. For example, the Address decode block indicates the maximum number of decoders of each type that is supported by the chip.
- A number of registers for control and status. The control register sets the register block parameters to the values used by the system during operation, including status information.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- The control and status registers are followed by registers to control block-specific functionality.

Table B15.1 shows a view of the different chip and port register blocks, as illustrated in Figure B15.1, including information about how each register block is located.

Table B15.1: Configuration register blocks

Register block name	Contents	Pointed to by
Chip Offset	Offsets for other chip-level register blocks.	IMPLEMENTATION DEFINED
Chip Properties	Chip capabilities and control registers.	C2C_ChipOffset1.ChipPropOffset
Chip Address Decoders	Common address decoder capabilities and control registers and up to 64 individual address decoder register groups.	C2C_ChipOffset2.AddrDecOffset
Chip Port Aggregation Group Address Decoders	Common PAG address decoder capabilities and control registers and up to 64 individual PAG address decoder register groups.	C2C_ChipOffset2.PortAggrOffset
Chip ID Route Table	Common ID Route Table capabilities and control registers and ID Route Table entries registers.	C2C_ChipOffset3.IDRouteTableOffset
Port Register Locator	Set of pointers to port offset register blocks for the different ports on the chip.	C2C_ChipOffset3.PortRegLocatorOffset
Port Offset	Offsets for other port-level register blocks.	C2C_Port[i]RegPtr.PortAddrPtr[a]
Port Capabilities and Control	Port capabilities and control registers.	C2C_PortRegOffset1.PortCapCtrlOffset
Common Logical Link register block	Common Port Logical Link capabilities register, and offsets for up to four individual Logical Link register blocks.	C2C_PortRegOffset2.LogLinkOffset
Individual Logical Link register block	Individual Logical Link registers.	C2C_LogLinkOffset[j].LogLink[k]Offset[b]

[a] i is included in [0, C2C_ChipCap.NumPort - 1].

[b] j is included in [1, 2], and k is included in [0, 3].

B15.1.1 Registers access restrictions

Unless specified otherwise, all registers are 64 bits and use little-endian format. Each register description details register access attributes, namely if both Read and Write (RW) or Read Only (RO) accesses are allowed.

Bit positions reserved for future use are indicated as RES0. To preserve forward compatibility, software must not rely on the field reading as 0. When writing to the field, software must either write all zeroes or, when the field is being restored from a previously read state, write the previously read value.

An implementation must support aligned 64-bit accesses to the registers. Support for other access sizes and alignments is IMPLEMENTATION DEFINED. It is IMPLEMENTATION DEFINED whether 64-bit accesses to the registers are single-copy-atomic.

The entire memory map is contained in a single security state.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.1.2 Commit and lock-on-commit functionality

Register blocks that include registers with RW access control attributes include one set of the following fields:

- Commit. This field is used by software to request that the register updates performed for the entire register block are committed.
- CommitStatus. This field indicates the status of the requested commit operation, including the option of indicating an error condition.
  - The following bit positions are defined:
    CommitStatus[3:2] Reserved.

    CommitStatus[1] Error. The value of `0b1` indicates the commit failed to complete and the decoder is not active.

    CommitStatus[0] Committed. The value of `0b1` indicates the Commit is complete.
  - The Committed and Error bits must not be asserted at the same time. After a commit, the Committed or Error bit must remain set until the Commit bit is cleared. Clearing the Commit bit will result in all bits in this field being cleared.
- LockOnCommit. This field allows software to request that the register block content is locked upon committing. Once locked, all affected registers become read-only until reset is applied.

Once registers are updated for each individual register block, the software must set the Commit field associated with the register block indicating the completion of the update process.

Hardware must change the associated CommitStatus field to Committed after completing the required updates and encountering no errors. If errors are encountered by the hardware, it must set the associated CommitStatus field to Error.

Optionally, software can use the associated LockOnCommit field to inform hardware to change all registers for the associated register block to RO once the content is committed.

The granularity of the commit and lock-on-commit functionality is a register block, with the exception of the individual Logical Link register block. That is, there is a single set of Commit, CommitStatus, and LockOnCommit fields for an entire register block.

The individual Logical Link register block has two sets of Commit, CommitStatus, and LockOnCommit fields. The first set is located in the C2C_LogLinkCtrlStatus1 register and offers commit and lock-on-commit functionality only for the C2C_LogLinkCtrlStatus1 register itself. The second set is located in the C2C_LogLinkCtrlStatus2 register and offers commit and lock-on-commit functionality for the entire register block except the C2C_LogLinkCtrlStatus1 register.

B15.1.3 Error behavior

An ErrBehavior field is included in address decoders register blocks to indicate the supported error behaviors and to control which one to use.

- ErrBehavior.
  - The following bit positions are defined:
    ErrBehavior[7:5] Reserved.

    ErrBehavior[4] Data response with all ones.

    ErrBehavior[3] Data response with NDE.

    ErrBehavior[2] Data response with poison.

    ErrBehavior[1] None.

    ErrBehavior[0] Undefined.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.2 Chip offset register block

Table B15.2 shows the chip offset register block.

The offset values are with respect to the address of the base of the Chip offset register block.

Table B15.2: Chip offset register block

Offset	Register name
0x00	C2C_ChipHdr
0x08	C2C_ChipOffset1
0x10	C2C_ChipOffset2
0x18	C2C_ChipOffset3
0x20	Reserved
0x28	
0x30	
0x38	

Figure B15.2 shows the format of the offset fields within C2C_ChipOffset1, C2C_ChipOffset2, and C2C_ChipOffset3.

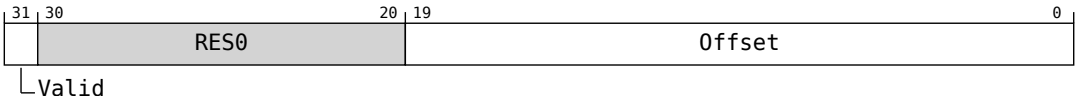


Figure B15.2: Offset field format

Table B15.3 shows the field descriptions for the offset format fields.

Table B15.3: Offset format field descriptions

Bits	Field	Description	Access type	Reset
[31]	Valid	This field indicates the offset field is valid.	RO	IMPLEMENTATION DEFINED.
[30:20]	RES0	Reserved.		
[19:0]	Offset	This field indicates the offset for the register block from the Chip offset register block base. The value must be 8-byte aligned. Offset[2:0] must be equal to 0.	RO	IMPLEMENTATION DEFINED.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.2.1 C2C_ChipHdr

The C2C_ChipHdr characteristics are:

Purpose

This register provides information on the Chip offset register block header.

Attributes

C2C_ChipHdr is a 64-bit register.

Field descriptions

The C2C_ChipHdr bit assignments are:

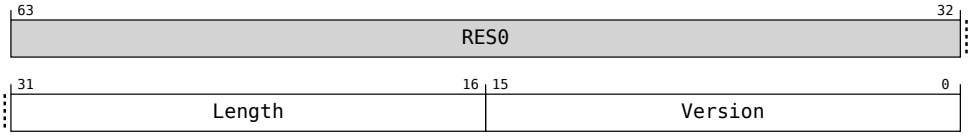


Table B15.4: C2C_ChipHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the Chip offset register block in bytes.	RO	`0x40`
[15:0]	Version	This field indicates the version of the Chip global offsets register block. The valid value is `0x00`.	RO	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.2.2 C2C_ChipOffset1

The C2C_ChipOffset1 characteristics are:

Purpose

This register provides the offset value for the Chip properties register block.

Attributes

C2C_ChipOffset1 is a 64-bit register.

Field descriptions

The C2C_ChipOffset1 bit assignments are:

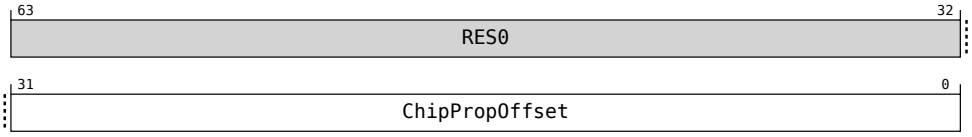


Table B15.5: C2C_ChipOffset1 field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:0]	ChipPropOffset	Offset to the Chip Properties register block. See Table B15.3 for the offset field descriptions.	RO	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.2.3 C2C_ChipOffset2

The C2C_ChipOffset2 characteristics are:

Purpose

This register provides offset values for the Chip address decoder and PAG address decoder register blocks.

Attributes

C2C_ChipOffset2 is a 64-bit register.

Field descriptions

The C2C_ChipOffset2 bit assignments are:

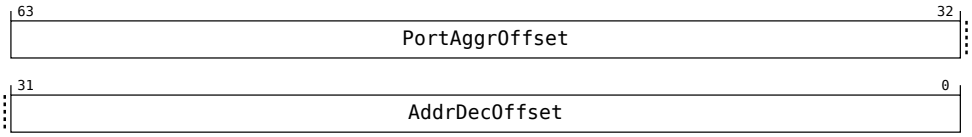


Table B15.6: C2C_ChipOffset2 field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	PortAggrOffset	Offset to the Chip Port Aggregation Group address decoders register block. See Table B15.3 for the offset field descriptions.	RO	
[31:0]	AddrDecOffset	Offset to the Chip Address Decoder register block. See Table B15.3 for the offset field descriptions.	RO	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.2.4 C2C_ChipOffset3

The C2C_ChipOffset3 characteristics are:

Purpose

This register provides offset values for the Chip ID Route Table and Port Register Locator register blocks.

Attributes

C2C_ChipOffset3 is a 64-bit register.

Field descriptions

The C2C_ChipOffset3 bit assignments are:

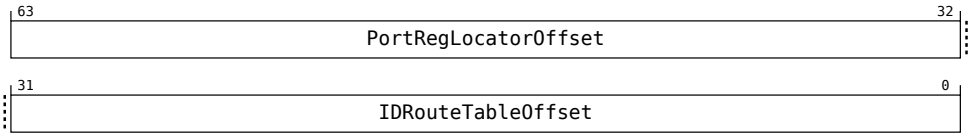


Table B15.7: C2C_ChipOffset3 field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	PortRegLocatorOffset	Offset to the Port Register Locator register block. See Table B15.3 for the offset field descriptions.	RO	
[31:0]	IDRouteTableOffset	Offset to the Chip ID Route Table register block. See Table B15.3 for the offset field descriptions.	RO	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.3 Chip properties register block

The chip properties register block includes capabilities that describe the number of nodes and ports in a chip, as well as registers to control the ChipID that is used.

Table B15.8 shows the chip properties register block.

Table B15.8: Chip properties register block

Offset	Register name
0x00	C2C_ChipPropHdr
0x08	C2C_ChipCap
0x10	Reserved
0x18	
0x20	C2C_ChipCtrlStatus
0x28	C2C_ChipIDCtrl
0x30	Reserved
0x38	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.3.1 C2C_ChipPropHdr

The C2C_ChipPropHdr characteristics are:

Purpose

This register provides information on the Chip properties register block header.

Attributes

C2C_ChipPropHdr is a 64-bit register.

Field descriptions

The C2C_ChipPropHdr bit assignments are:

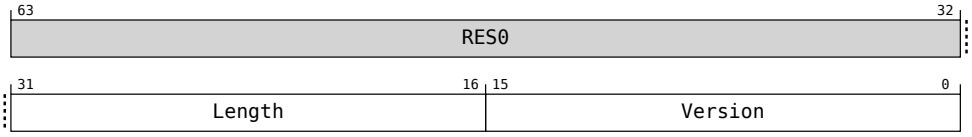


Table B15.9: C2C_ChipPropHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the Chip offset register block in bytes.	RO	`0x40`
[15:0]	Version	This field indicates the version of the Chip global offsets register block. The valid value is `0x00`.	RO	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.3.2 C2C_ChipCap

The C2C_ChipCap characteristics are:

Purpose

This register provides information on the Chip capabilities.

Attributes

C2C_ChipCap is a 64-bit register.

Field descriptions

The C2C_ChipCap bit assignments are:

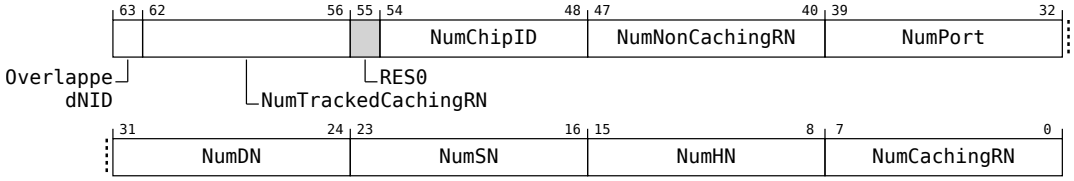


Table B15.10: C2C_ChipCap field descriptions

Bits	Field	Description	Access type	Reset
[63]	OverlappedNID	This field indicates whether the assigned AgentID values for caching and non-caching Request Nodes on this chip can be distinguished using a single bit of the AgentID. See B6.3.1 *Request Node enhanced NodeID assignment* for more information. `0b0` No distinguishing bit is used. Both caching and non-caching Request Node AgentID values must be compacted, as defined in B6.3.1 *Request Node enhanced NodeID assignment*. `0b1` The AgentID values for caching and non-caching Request Nodes overlap, except for 1 bit in the AgentID field. The AgentID bit that distinguishes the two types is defined in B6.3.1 *Request Node enhanced NodeID assignment*.	RO	IMPLEMENTATION DEFINED
[62:56]	NumTrackedCachingRN	This field indicates the maximum number of Caching Request Nodes from connected chips that can be individually tracked by Home Nodes on the chip. The maximum value allowed is 64.	RO	IMPLEMENTATION DEFINED
[55]	RES0	Reserved, RES0.		
[54:48]	NumChipID	This field indicates the number of ChipID values required by the chip. The valid values for this field are 0 to 64.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[47:40]	NumNonCachingRN	This field indicates the number of Non-caching Request Nodes exposed across the C2C link. A Non-caching RN is identified by a unique SrcID that is exposed across the C2C link, which represents one or more agents that do not include a hardware-coherent cache and do not issue snoopable allocating requests.	RO	IMPLEMENTATION DEFINED
[39:32]	NumPort	This field indicates the number of ports in the chip.	RO	IMPLEMENTATION DEFINED
[31:24]	NumDN	This field indicates the number of DVM Nodes exposed across the C2C link.	RO	IMPLEMENTATION DEFINED
[23:16]	NumSN	This field indicates the number of Subordinate Nodes exposed across the C2C link.	RO	IMPLEMENTATION DEFINED
[15:8]	NumHN	This field indicates the number of Home Nodes exposed across the C2C link.	RO	IMPLEMENTATION DEFINED
[7:0]	NumCachingRN	This field indicates the number of caching Request Nodes exposed across the C2C link. A Caching RN is identified by a unique SrcID that is exposed across the C2C link, which represents one or more agents that include a hardware-coherent cache and can issue snoopable allocating requests.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.3.3 C2C_ChipCtrlStatus

The C2C_ChipCtrlStatus characteristics are:

Purpose

This register provides information on the Chip control and status.

Attributes

C2C_ChipCtrlStatus is a 64-bit register.

Field descriptions

The C2C_ChipCtrlStatus bit assignments are:

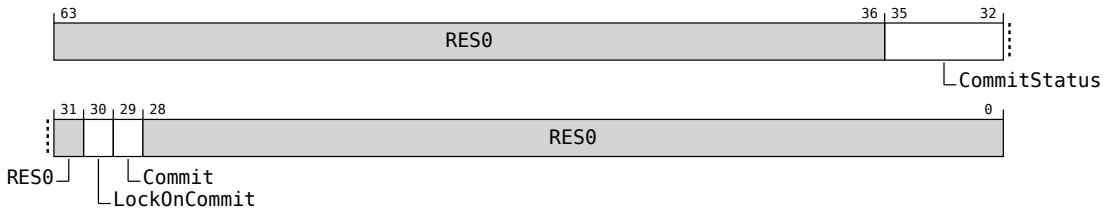


Table B15.11: C2C_ChipCtrlStatus field descriptions

Bits	Field	Description	Access type	Reset
[63:36]	RES0	Reserved, RES0.		
[35:32]	CommitStatus	This field indicates the commit status of the Chip properties register block. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RO	`0x0`
[31]	RES0	Reserved, RES0.		
[30]	LockOnCommit	This field requests that the programming of the Chip properties register block is locked upon committing. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, all registers in the Chip properties register block are Read-Only until reset.	RW	`0x00`
[29]	Commit	Setting this field requests that the updates are committed to the Chip properties register block. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[28:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.3.4 C2C_ChipIDCtrl

The C2C_ChipIDCtrl characteristics are:

Purpose

This register provides information on ChipID control.

Attributes

C2C_ChipIDCtrl is a 64-bit register.

Field descriptions

The C2C_ChipIDCtrl bit assignments are:

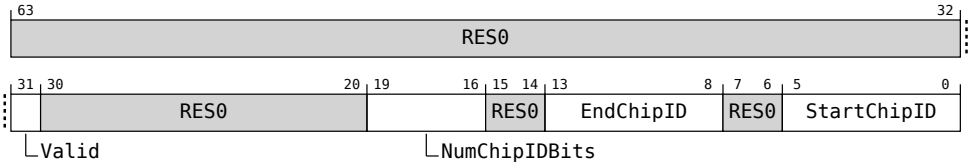


Table B15.12: C2C_ChipIDCtrl field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31]	Valid	This field indicates the register has been programmed and is valid.	RW	`0x0`
[30:20]	RES0	Reserved, RES0.		
[19:16]	NumChipIDBits	This field indicates the number of bits in the ChipID field. The valid values for this field are 0 to 6. A value of 0 specifies a single-chip topology.	RW	`0x0`
[15:14]	RES0	Reserved, RES0.		
[13:8]	EndChipID	This field indicates the highest ChipID range value assigned to the chip. The valid field values are 0 to ((2^NumChipIDBits) - 1). Typically, this field value is the same value as the StartChipID field. However, the field value may be larger if a single chip uses multiple ChipID values.	RW	`0x00`
[7:6]	RES0	Reserved, RES0.		
[5:0]	StartChipID	This field indicates the lowest ChipID range value assigned to this chip. The valid field values are 0 to ((2^NumChipIDBits) - 1).	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4 Chip address decoders register block

The chip address decoder register block includes common address decoder registers and individual address decoder register groups.

Table B15.13 shows the following:

- Common address decoder registers, which apply to all address decoders.
- Individual address decoder register groups. The layout of the individual address decoder register group is identical for interleaved address decoders and for single-target address decoders.

The offset value for each individual address decoder register is with respect to the address of the base of the individual address decoder register block.

Table B15.13: Chip address decoder register block

Offset	Register name
`0x00`	C2C_AddrDecHdr
`0x08`	C2C_AddrDecCommonCap
`0x10`	Reserved
`0x18`	
`0x20`	C2C_AddrDecCommonCtrl
`0x28`	Reserved
`0x30`	
`0x38`	
`0x40` to (M x `0x60`) + `0x38`	M interleaved address decoders.
`0x40` + (M x `0x60`) to (M x `0x60`) + (N x `0x60`) + `0x38`	N single-target address decoders.

Offset	Register name
`0x00`	C2C_AddrDecCap
`0x08`	C2C_AddrDecCtrlStatus
`0x10`	C2C_AddrDecBaseAddr
`0x18`	C2C_AddrDecEndAddr
`0x20`	C2C_AddrDecHashMask
`0x28`	C2C_AddrDecIwToTgtMap1
`0x30`	C2C_AddrDecIwToTgtMap2
`0x38`	C2C_AddrDecIwToTgtMap3
`0x40`	C2C_AddrDecIwToTgtMap4
`0x48`	Reserved
`0x50`	
`0x58`	

The following conditions apply for the number of address decoders, M and N in Table B15.13:

- A chip is permitted to include:
  - M = 0 to 32 interleaved address decoders.
  - N = 0 to 32 single-target address decoders.
- A chip must have sufficient address decoders, either interleaved or single-target, to enable routing of requests to all required targets on remote chips.

Figure B15.3 shows the IwT format when TgtType is `0b0`, indicating PAG.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

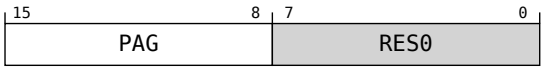


Figure B15.3: IwT entry format when TgtType = 0b0, indicating PAG

Table B15.14 shows the field descriptions for the IwT format when TgtType is `0b0`, indicating PAG.

Table B15.14: IwT entry field when TgtType = 0b0, indicating PAG

Bits	Field	Description	Access type	Reset
[15:8]	PAG	PAG for this interleave way.	RW	`0x00`
[7:2]	RES0	Reserved		

Figure B15.4 shows the IwT format when TgtType is `0b1`, indicating Port and Logical Link.

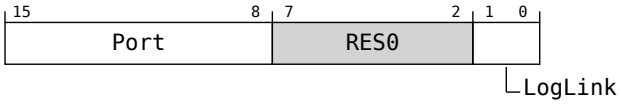


Figure B15.4: IwT entry format when TgtType = 0b1, indicating Port and Logical Link

Table B15.14 shows the field descriptions for the IwT format when TgtType is `0b1`, indicating Port and Logical Link.

Table B15.15: IwT entry field when TgtType = 0b1, indicating Port and Logical Link

Bits	Field	Description	Access type	Reset
[15:8]	Port	Port number for this interleave way. A value of `0xFF` specifies the target is a local chip.	RW	`0x00`
[7:2]	RES0	Reserved		
[1:0]	LogLink	Logical Link for this interleave way. Inapplicable and must be zero when Port == Local.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.1 C2C_AddrDecHdr

The C2C_AddrDecHdr characteristics are:

Purpose

This register provides information on the Chip Address Decoder register block header.

Attributes

C2C_AddrDecHdr is a 64-bit register.

Field descriptions

The C2C_AddrDecHdr bit assignments are:

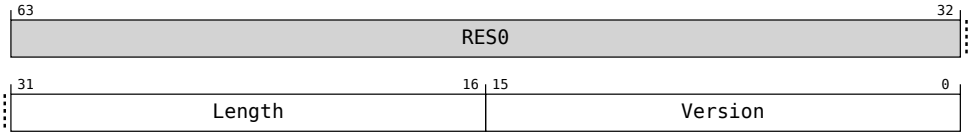


Table B15.16: C2C_AddrDecHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the Address Decoder register block in bytes.	RO	IMPLEMENTATION DEFINED
[15:0]	Version	This field indicates the version of the Address Decoder register block. The valid value is `0x0`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.2 C2C_AddrDecCommonCap

The C2C_AddrDecCommonCap characteristics are:

Purpose

This register provides information on the Address Decoder capabilities common to all address decoders.

Attributes

C2C_AddrDecCommonCap is a 64-bit register.

Field descriptions

The C2C_AddrDecCommonCap bit assignments are:

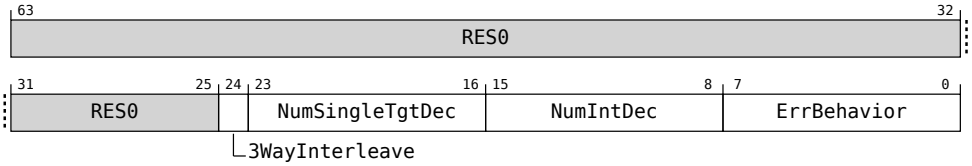


Table B15.17: C2C_AddrDecCommonCap field descriptions

Bits	Field	Description	Access type	Reset
[63:25]	RES0	Reserved, RES0.		
[24]	3WayInterleave	This field indicates support for optional three-way interleave. `0b0` Three-way interleave is not supported. `0b1` Three-way interleave is supported.	RO	IMPLEMENTATION DEFINED
[23:16]	NumSingleTgtDec	This field indicates the number of single-target address decoders that are available. The valid values are 0 to 32. In a multi-chip configuration, the combination of NumIntDec and NumSingleTgtDec must be such that all chips can be reached using the decoders.	RO	IMPLEMENTATION DEFINED
[15:8]	NumIntDec	This field indicates the number of interleaved address decoders that are available. The valid values are 0 to 32. In a multi-chip configuration, the combination of NumIntDec and NumSingleTgtDec must be such that all chips can be reached using the decoders.	RO	IMPLEMENTATION DEFINED
[7:0]	ErrBehavior	This field indicates the address decoder error behaviors that are supported. Multiple bits are permitted to be set to indicate the error behaviors that are supported by the decoders. No default capability is defined. See B15.1.3 *Error behavior* for more information.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.3 C2C_AddrDecCommonCtrl

The C2C_AddrDecCommonCtrl characteristics are:

Purpose

This register provides information on Address Decoder common control mechanisms.

There are no address decoder common control mechanisms defined in this issue of the specification.

Attributes

C2C_AddrDecCommonCtrl is a 64-bit register.

Field descriptions

The C2C_AddrDecCommonCtrl bit assignments are:

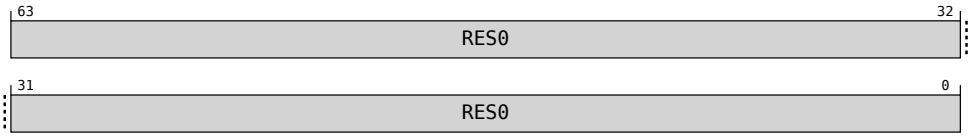


Table B15.18: C2C_AddrDecCommonCtrl field descriptions

Bits	Field	Description	Access type	Reset
[63:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.4 C2C_AddrDecCap

The C2C_AddrDecCap characteristics are:

Purpose

This register provides information on the individual Address Decoder capabilities.

There are no decoder-specific capabilities defined in this issue of the specification.

Attributes

C2C_AddrDecCap is a 64-bit register.

Field descriptions

The C2C_AddrDecCap bit assignments are:

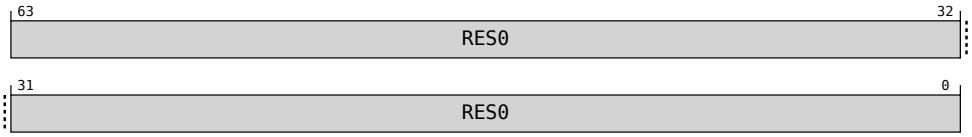


Table B15.19: C2C_AddrDecCap field descriptions

Bits	Field	Description	Access type	Reset
[63:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.5 C2C_AddrDecCtrlStatus

The C2C_AddrDecCtrlStatus characteristics are:

Purpose

This register provides information on the individual Address Decoder control and status.

Attributes

C2C_AddrDecCtrlStatus is a 64-bit register.

Field descriptions

The C2C_AddrDecCtrlStatus bit assignments are:

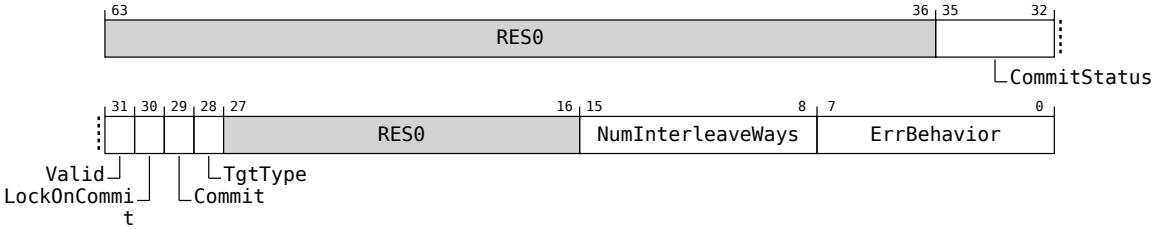


Table B15.20: C2C_AddrDecCtrlStatus field descriptions

Bits	Field	Description	Access type	Reset
[63:36]	RES0	Reserved, RES0.		
[35:32]	CommitStatus	This field indicates the commit status of the Address Decoder register block. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RO	`0x0`
[31]	Valid	This field indicates that all registers within the Address Decoder register block are programmed and valid.	RW	`0x0`
[30]	LockOnCommit	This field requests that the programming of the Address Decoder register block is locked upon committing. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, all registers in the Address Decoder register block are Read-Only until reset.	RW	`0x0`
[29]	Commit	Setting this field requests that the updates are committed to the Address Decoder register block. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[28]	TgtType	This field indicates the target type for the address decoder. See B15.4 *Chip address decoders register block* for more information. `0b0` PAG. `0b1` Port and Logical Link.	RW	`0x0`
[27:16]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[15:8]	NumInterleaveWays	This field indicates the number of interleave ways used by the address decoder. 1, 2, 3, 4, 8, and 16 values are valid. The value of 3 must not be used when C2C_AddrDecCommonCap.3WayInterleave is 0. This field does not apply to single-target decoders.	RW	`0x00`
[7:0]	ErrBehavior	This field indicates the address decoder error behavior that is selected for this decoder. Only 1 bit is allowed to be set at the same time. A behavior can be selected only when the respective capability bit is set in the C2C_AddrDecCommonCap register. See B15.1.3 *Error behavior* for more information.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.6 C2C_AddrDecBaseAddr

The C2C_AddrDecBaseAddr characteristics are:

Purpose

This register provides information on the base address of the individual Address Decoder.

Attributes

C2C_AddrDecBaseAddr is a 64-bit register.

Field descriptions

The C2C_AddrDecBaseAddr bit assignments are:

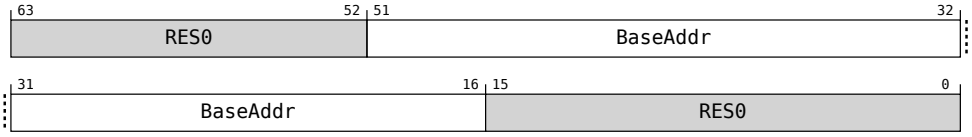


Table B15.21: C2C_AddrDecBaseAddr field descriptions

Bits	Field	Description	Access type	Reset
[63:52]	RES0	Reserved, RES0.		
[51:16]	BaseAddr	This field indicates the base address for this address decoder. The address must be 64KB aligned. BaseAddr[15:0] bits are implicitly all zeros.	RW	`0x0`
[15:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.7 C2C_AddrDecEndAddr

The C2C_AddrDecEndAddr characteristics are:

Purpose

This register provides information on the end address for the individual Address Decoder.

Attributes

C2C_AddrDecEndAddr is a 64-bit register.

Field descriptions

The C2C_AddrDecEndAddr bit assignments are:

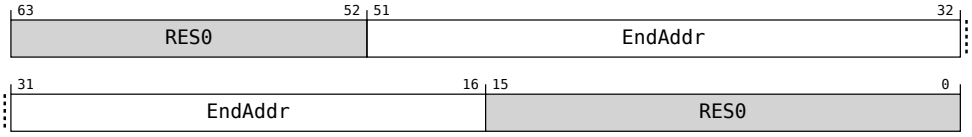


Table B15.22: C2C_AddrDecEndAddr field descriptions

Bits	Field	Description	Access type	Reset
[63:52]	RES0	Reserved, RES0.		
[51:16]	EndAddr	This field indicates the end address for this address decoder. This is the last address to which the decoder will respond. EndAddr[15:0] bits are implicitly all ones.	RW	`0x0`
[15:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.8 C2C_AddrDecHashMask

The C2C_AddrDecHashMask characteristics are:

Purpose

This register provides information on the Address Decoder Hash Mask.

Attributes

C2C_AddrDecHashMask is a 64-bit register.

Field descriptions

The C2C_AddrDecHashMask bit assignments are:

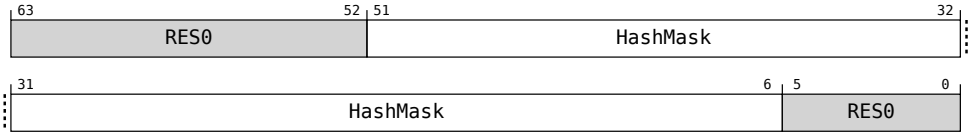


Table B15.23: C2C_AddrDecHashMask field descriptions

Bits	Field	Description	Access type	Reset
[63:52]	RES0	Reserved, RES0.		
[51:6]	HashMask	This field indicates the Hash Mask bits for the address decoder. Hash must be 64-byte aligned. HashMask[5:0] are implicitly all zeroes.	RW	`0x0`
[5:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.9 C2C_AddrDecIwToTgtMap1

The C2C_AddrDecIwToTgtMap1 characteristics are:

Purpose

This register provides entries 0 to 3 of the Address Decoder Interleave way to Target table.

Attributes

C2C_AddrDecIwToTgtMap1 is a 64-bit register.

Field descriptions

The C2C_AddrDecIwToTgtMap1 bit assignments are:

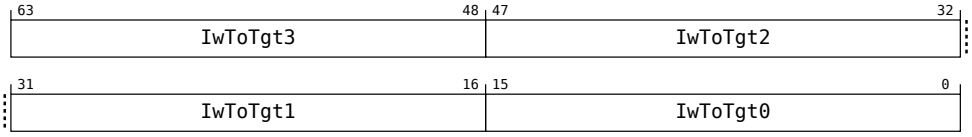


Table B15.24: C2C_AddrDecIwToTgtMap1 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToTgt3	IwT table entry 3. Reserved for single-target decoder.	RW	`0x0`
[47:32]	IwToTgt2	IwT table entry 2. Reserved for single-target decoder.	RW	`0x0`
[31:16]	IwToTgt1	IwT table entry 1. Reserved for single-target decoder.	RW	`0x0`
[15:0]	IwToTgt0	IwT table entry 0.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.10 C2C_AddrDecIwToTgtMap2

The C2C_AddrDecIwToTgtMap2 characteristics are:

Purpose

This register provides entries 4 to 7 of the Address Decoder Interleave way to Target table.

Attributes

C2C_AddrDecIwToTgtMap2 is a 64-bit register.

Field descriptions

The C2C_AddrDecIwToTgtMap2 bit assignments are:

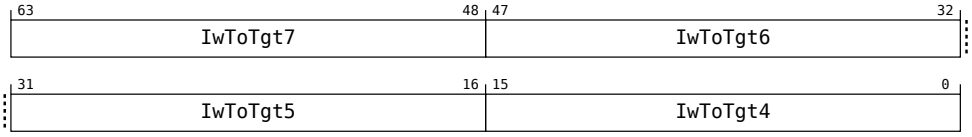


Table B15.25: C2C_AddrDecIwToTgtMap2 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToTgt7	IwT table entry 7. Reserved for single-target decoder.	RW	`0x0`
[47:32]	IwToTgt6	IwT table entry 6. Reserved for single-target decoder.	RW	`0x0`
[31:16]	IwToTgt5	IwT table entry 5. Reserved for single-target decoder.	RW	`0x0`
[15:0]	IwToTgt4	IwT table entry 4. Reserved for single-target decoder.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.11 C2C_AddrDecIwToTgtMap3

The C2C_AddrDecIwToTgtMap3 characteristics are:

Purpose

This register provides entries 8 to 11 of the Address Decoder Interleave way to Target table.

Attributes

C2C_AddrDecIwToTgtMap3 is a 64-bit register.

Field descriptions

The C2C_AddrDecIwToTgtMap3 bit assignments are:

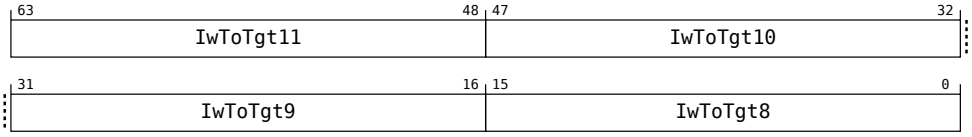


Table B15.26: C2C_AddrDecIwToTgtMap3 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToTgt11	IwT table entry 11. Reserved for single-target decoder.	RW	`0x0`
[47:32]	IwToTgt10	IwT table entry 10. Reserved for single-target decoder.	RW	`0x0`
[31:16]	IwToTgt9	IwT table entry 9. Reserved for single-target decoder.	RW	`0x0`
[15:0]	IwToTgt8	IwT table entry 8. Reserved for single-target decoder.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.4.12 C2C_AddrDecIwToTgtMap4

The C2C_AddrDecIwToTgtMap4 characteristics are:

Purpose

This register provides entries 12 to 15 of the Address Decoder Interleave way to Target table.

Attributes

C2C_AddrDecIwToTgtMap4 is a 64-bit register.

Field descriptions

The C2C_AddrDecIwToTgtMap4 bit assignments are:

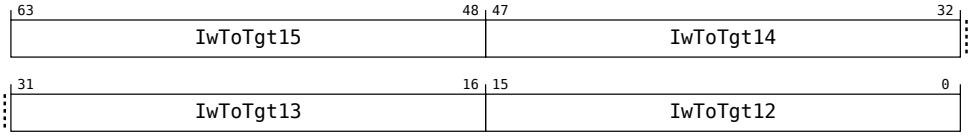


Table B15.27: C2C_AddrDecIwToTgtMap4 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToTgt15	IwT table entry 15. Reserved for single-target decoder.	RW	`0x0`
[47:32]	IwToTgt14	IwT table entry 14. Reserved for single-target decoder.	RW	`0x0`
[31:16]	IwToTgt13	IwT table entry 13. Reserved for single-target decoder.	RW	`0x0`
[15:0]	IwToTgt12	IwT table entry 12. Reserved for single-target decoder.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5 Chip Port Aggregation Group address decoders register block

Table B15.28 shows the following:

- Common PAG address decoder registers, which applies to all PAG address decoders.
- Individual PAG address decoder register groups

The numerical value of N is defined by the NumPAG field in C2C_PortAggrCommonCap.

The offset value for each individual PAG address decoder register is with respect to the address of the base of the individual PAG address decoder register block.

Table B15.28: PAG address decoder register block

Offset	Register name	
`0x00`	C2C_PortAggrHdr	
`0x08`	C2C_PortAggrCommonCap	
`0x10`	Reserved	
`0x18`		
`0x20`	C2C_PortAggrCommonCtrl	
`0x28`	Reserved	
`0x30`		
`0x38`		

`0x40` to (N x `0x60`) + `0x38`	PAG0 to PAG(N-1)	**Offset Register name**
		`0x00` C2C_PAGCap
		`0x08` C2C_PAGCtrlStatus
		`0x10` Reserved
		`0x18`
		`0x20` C2C_PAGHashMask
		`0x28` C2C_PAGIwToPortMap1
		`0x30` C2C_PAGIwToPortMap2
		`0x38` C2C_PAGIwToPortMap3
		`0x40` C2C_PAGIwToPortMap4
		`0x48` Reserved
		`0x50`
		`0x58`

A PAG register has the following characteristics:

- There is no address range. See B7.2.2 *Port Aggregation Group address decoder*.
- A 16-entry Interleave way to Port (IwP) mapping table is included. The output of the mapping is a Port and Logical Link number.

Figure B15.5 shows the format of the IwP entry fields within C2C_PAGIwToPortMap1, C2C_PAGIwToPortMap2, C2C_PAGIwToPortMap3, and C2C_PAGIwToPortMap4.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

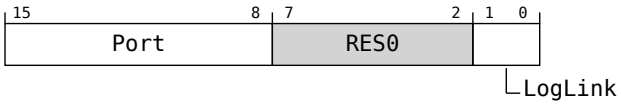


Figure B15.5: IwP entry field format

Table B15.29 shows the field descriptions for IwP entry field formats.

Table B15.29: IwP entry field descriptions

Bits	Field	Description	Access type	Reset
[15:8]	Port	Port number of this IwP entry. A value of `0xFF` specifies the target is a local chip.	RW	`0x00`
[7:2]	RES0	Reserved		
[1:0]	LogLink	Logical Link for this IwP entry. Inapplicable and must be zero when Port == Local.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.1 C2C_PortAggrHdr

The C2C_PortAggrHdr characteristics are:

Purpose

This register provides information on the PAG address decoder register block header.

Attributes

C2C_PortAggrHdr is a 64-bit register.

Field descriptions

The C2C_PortAggrHdr bit assignments are:

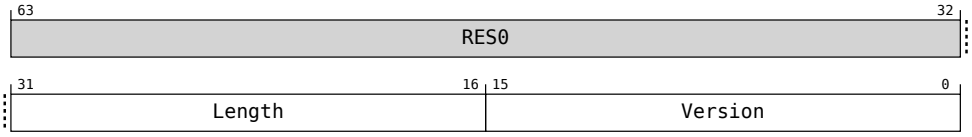


Table B15.30: C2C_PortAggrHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the PAG register block in bytes.	RO	IMPLEMENTATION DEFINED, based upon the number of Port Aggregation Groups
[15:0]	Version	This field indicates the version of the PAG register block. The valid value is `0x00`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.2 C2C_PortAggrCommonCap

The C2C_PortAggrCommonCap characteristics are:

Purpose

This register provides information on the capabilities common to all PAG address decoders.

Attributes

C2C_PortAggrCommonCap is a 64-bit register.

Field descriptions

The C2C_PortAggrCommonCap bit assignments are:

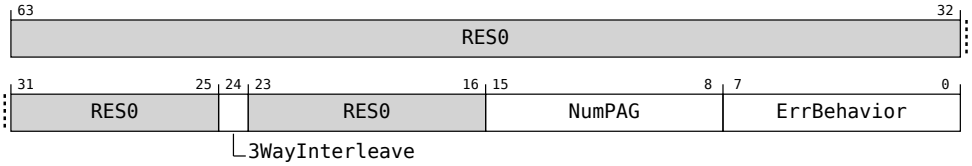


Table B15.31: C2C_PortAggrCommonCap field descriptions

Bits	Field	Description	Access type	Reset
[63:25]	RES0	Reserved, RES0.		
[24]	3WayInterleave	This field indicates support for optional three-way interleave. `0b0` Three-way interleave is not supported. `0b1` Three-way interleave is supported.	RO	IMPLEMENTATION DEFINED
[23:16]	RES0	Reserved, RES0.		
[15:8]	NumPAG	This field indicates the number of Port Aggregation Groups in the chip. The valid values for this field are 0 to 64.	RO	IMPLEMENTATION DEFINED
[7:0]	ErrBehavior	This field indicates the PAG error behaviors that are supported. Multiple bits are permitted to be set to indicate the error behaviors that are supported by the decoders. No default capability is defined. See B15.1.3 *Error behavior* for more information.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.3 C2C_PortAggrCommonCtrl

The C2C_PortAggrCommonCtrl characteristics are:

Purpose

This register provides information on the PAG common control.

There are no Port Aggregation Group-specific control mechanisms defined in this issue of the specification.

Attributes

C2C_PortAggrCommonCtrl is a 64-bit register.

Field descriptions

The C2C_PortAggrCommonCtrl bit assignments are:

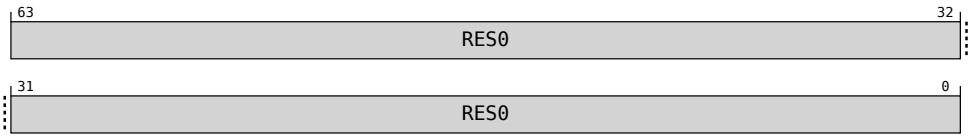


Table B15.32: C2C_PortAggrCommonCtrl field descriptions

Bits	Field	Description	Access type	Reset
[63:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.4 C2C_PAGCap

The C2C_PAGCap characteristics are:

Purpose

This register provides information on the individual PAG capabilities.

There are no Port Aggregation Group-specific capabilities defined in this issue of the specification.

Attributes

C2C_PAGCap is a 64-bit register.

Field descriptions

The C2C_PAGCap bit assignments are:

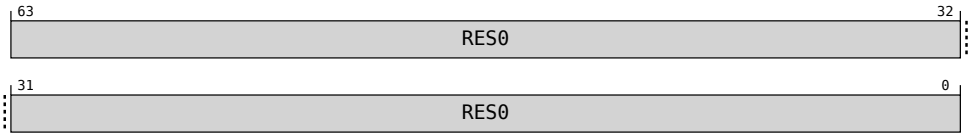


Table B15.33: C2C_PAGCap field descriptions

Bits	Field	Description	Access type	Reset
[63:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.5 C2C_PAGCtrlStatus

The C2C_PAGCtrlStatus characteristics are:

Purpose

This register provides information on the individual PAG control and status.

Attributes

C2C_PAGCtrlStatus is a 64-bit register.

Field descriptions

The C2C_PAGCtrlStatus bit assignments are:

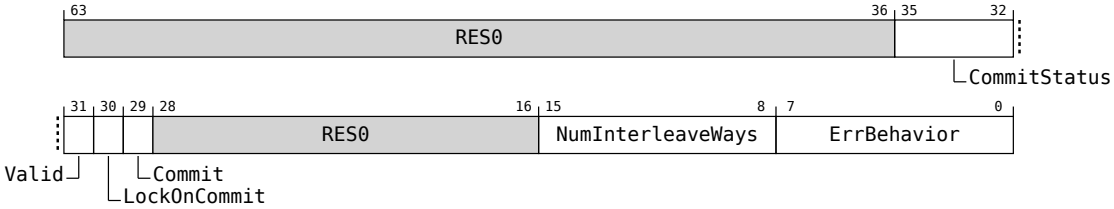


Table B15.34: C2C_PAGCtrlStatus field descriptions

Bits	Field	Description	Access type	Reset
[63:36]	RES0	Reserved, RES0.		
[35:32]	CommitStatus	This field indicates the commit status of the individual PAG register block. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[31]	Valid	This field indicates that all registers within the PAG register block are programmed and valid.	RW	`0x0`
[30]	LockOnCommit	This field requests that the programming of the PAG register block is locked upon committing. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, all registers in the PAG register block are Read-Only until reset.	RW	`0x0`
[29]	Commit	Setting this field requests that the updates are committed to the PAG register block. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[28:16]	RES0	Reserved, RES0.		
[15:8]	NumInterleaveWays	This field indicates the number of interleave ways used by the PAG address decoder. 1, 2, 3, 4, 8, and 16 values are valid. The value of 3 must not be used when C2C_PortAggrCommonCap.3WayInterleave is 0.	RW	`0x00`
[7:0]	ErrBehavior	This field indicates the PAG error behavior that is selected. Only 1 bit must be set. See B15.1.3 *Error behavior* for more information.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.6 C2C_PAGHashMask

The C2C_PAGHashMask characteristics are:

Purpose

This register provides information on the PAG Hash Mask.

Attributes

C2C_PAGHashMask is a 64-bit register.

Field descriptions

The C2C_PAGHashMask bit assignments are:

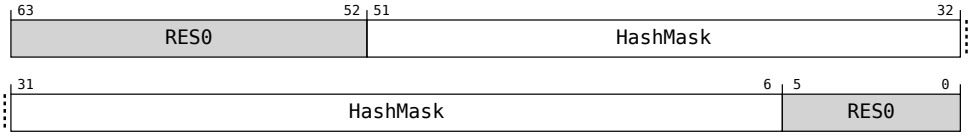


Table B15.35: C2C_PAGHashMask field descriptions

Bits	Field	Description	Access type	Reset
[63:52]	RES0	Reserved, RES0.		
[51:6]	HashMask	This field indicates the Hash Mask bits for the PAG. Hash must be 64-byte aligned. HashMask[5:0] are implicitly all zeroes.	RW	`0x0`
[5:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.7 C2C_PAGIwToPortMap1

The C2C_PAGIwToPortMap1 characteristics are:

Purpose

This register provides entries 0 to 3 of the PAG Interleave way to Port table.

Attributes

C2C_PAGIwToPortMap1 is a 64-bit register.

Field descriptions

The C2C_PAGIwToPortMap1 bit assignments are:

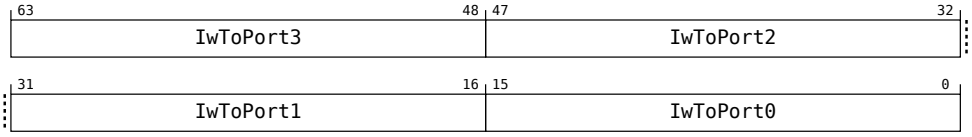


Table B15.36: C2C_PAGIwToPortMap1 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToPort3	IwP table entry 3.	RW	`0x0`
[47:32]	IwToPort2	IwP table entry 2.	RW	`0x0`
[31:16]	IwToPort1	IwP table entry 1.	RW	`0x0`
[15:0]	IwToPort0	IwP table entry 0.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.8 C2C_PAGIwToPortMap2

The C2C_PAGIwToPortMap2 characteristics are:

Purpose

This register provides entries 4 to 7 of the PAG Interleave way to Port table.

Attributes

C2C_PAGIwToPortMap2 is a 64-bit register.

Field descriptions

The C2C_PAGIwToPortMap2 bit assignments are:

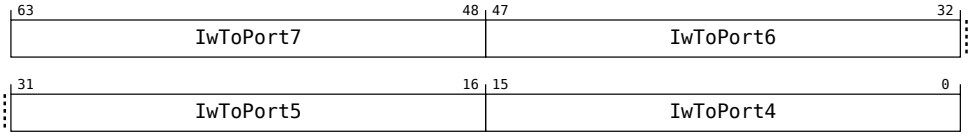


Table B15.37: C2C_PAGIwToPortMap2 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToPort7	IwP table entry 7.	RW	`0x0`
[47:32]	IwToPort6	IwP table entry 6.	RW	`0x0`
[31:16]	IwToPort5	IwP table entry 5.	RW	`0x0`
[15:0]	IwToPort4	IwP table entry 4.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.9 C2C_PAGIwToPortMap3

The C2C_PAGIwToPortMap3 characteristics are:

Purpose

This register provides entries 8 to 11 of the PAG Interleave way to Port table.

Attributes

C2C_PAGIwToPortMap3 is a 64-bit register.

Field descriptions

The C2C_PAGIwToPortMap3 bit assignments are:

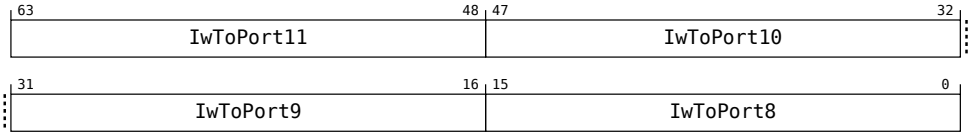


Table B15.38: C2C_PAGIwToPortMap3 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToPort11	IwP table entry 11.	RW	`0x0`
[47:32]	IwToPort10	IwP table entry 10.	RW	`0x0`
[31:16]	IwToPort9	IwP table entry 9.	RW	`0x0`
[15:0]	IwToPort8	IwP table entry 8.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.5.10 C2C_PAGIwToPortMap4

The C2C_PAGIwToPortMap4 characteristics are:

Purpose

This register provides entries 12 to 15 of the PAG Interleave way to Port table.

Attributes

C2C_PAGIwToPortMap4 is a 64-bit register.

Field descriptions

The C2C_PAGIwToPortMap4 bit assignments are:

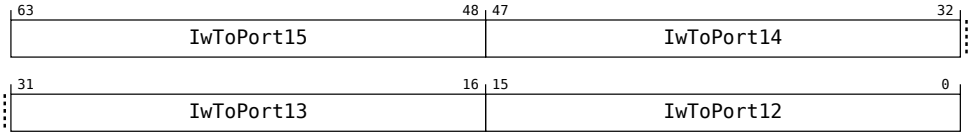


Table B15.39: C2C_PAGIwToPortMap4 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IwToPort15	IwP table entry 15.	RW	`0x0`
[47:32]	IwToPort14	IwP table entry 14.	RW	`0x0`
[31:16]	IwToPort13	IwP table entry 13.	RW	`0x0`
[15:0]	IwToPort12	IwP table entry 12.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.6 Chip ID Route Table register block

Table B15.40 shows the chip ID Route Table register block.

The chip ID Route Table register block has N ID Route Table registers each with four ID Route entries. For an M ID Route entries table, where M is defined by NumIDRouteEntries in C2C_ChipIDRouteTableCap, N = `ceil(M/4)`.

Table B15.40: Chip ID Route Table register block

Offset	Register name
`0x00`	C2C_ChipIDRouteTableHdr
`0x08`	C2C_ChipIDRouteTableCap
`0x10`	Reserved
`0x18`	
`0x20`	C2C_ChipIDRouteTableCtrlStatus
`0x28`	Reserved
`0x30`	
`0x38`	
`0x40` to (N x `0x8`) + `0x38`	C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1]

Figure B15.6 shows the layout of the IDRoute[0] to IDRoute[M-1] fields within the C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1] registers.

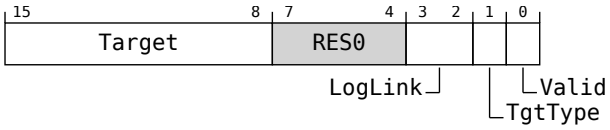


Figure B15.6: IDRoute[0] to IDRoute[M-1] layout

Table B15.41 shows the IDRoute[0] to IDRoute[M-1] field descriptions.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B15.41: IDRoute[0] to IDRoute[M-1] field descriptions

Bits	Field	Description	Access type	Reset
[15:8]	Target	This field indicates the target port or PAG for this ID Route entry. **When TgtType == Port**: Target[7:0] == Port number. The value of `0xFF` indicates the target is local. **When TgtType == PAG**: Target[7:6] == Reserved Target[5:0] == PAG.	RW	`0x00`
[7:4]	RES0	Reserved.		
[3:2]	LogLink	This field indicates the Logical Link for this ID Route entry. This field is inapplicable and must be 0 for PAG or a local target.	RW	`0x0`
[1]	TgtType	This field indicates the target type for this ID Route entry. `0b0` Target ID is a PAG. `0b1` Target ID is a port or local.	RW	`0x0`
[0]	Valid	This field indicates that this register has been programmed and is valid.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.6.1 C2C_ChipIDRouteTableHdr

The C2C_ChipIDRouteTableHdr characteristics are:

Purpose

This register provides information on the Chip ID Route Table register block header.

Attributes

C2C_ChipIDRouteTableHdr is a 64-bit register.

Field descriptions

The C2C_ChipIDRouteTableHdr bit assignments are:

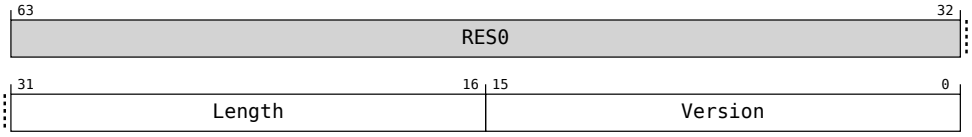


Table B15.42: C2C_ChipIDRouteTableHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the ID Route Table register block in bytes.	RO	IMPLEMENTATION DEFINED, based upon the number of ID routes
[15:0]	Version	This field indicates the version of the ID Route Table register block. The valid value is `0x00`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.6.2 C2C_ChipIDRouteTableCap

The C2C_ChipIDRouteTableCap characteristics are:

Purpose

This register provides information on the ID Route Table capabilities.

Attributes

C2C_ChipIDRouteTableCap is a 64-bit register.

Field descriptions

The C2C_ChipIDRouteTableCap bit assignments are:

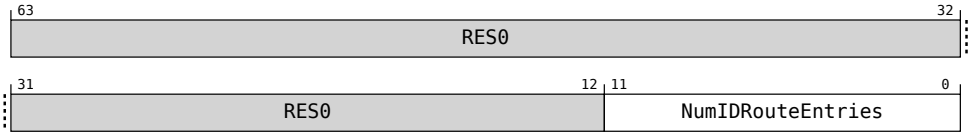


Table B15.43: C2C_ChipIDRouteTableCap field descriptions

Bits	Field	Description	Access type	Reset
[63:12]	RES0	Reserved, RES0.		
[11:0]	NumIDRouteEntries	This field indicates the number of ID Route Table entries supported. A value of 0 indicates that the ID Route Table is not implemented.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.6.3 C2C_ChipIDRouteTableCtrlStatus

The C2C_ChipIDRouteTableCtrlStatus characteristics are:

Purpose

This register provides information on the ID Route Table control and status.

Attributes

C2C_ChipIDRouteTableCtrlStatus is a 64-bit register.

Field descriptions

The C2C_ChipIDRouteTableCtrlStatus bit assignments are:

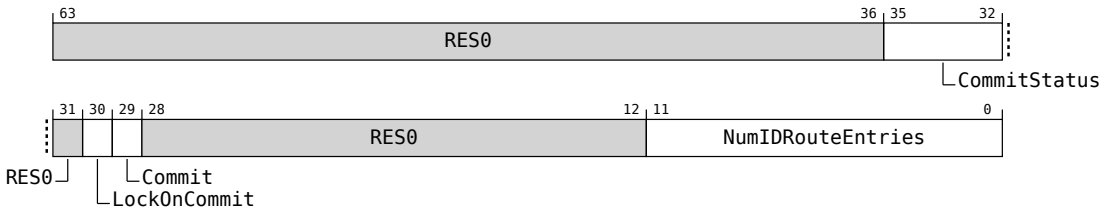


Table B15.44: C2C_ChipIDRouteTableCtrlStatus field descriptions

Bits	Field	Description	Access type	Reset
[63:36]	RES0	Reserved, RES0.		
[35:32]	CommitStatus	This field indicates the commit status of the ID Route Table register block. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[31]	RES0	Reserved, RES0.		
[30]	LockOnCommit	This field requests that the programming of the Chip ID Route Table register block is locked upon committing. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, all registers in the Chip ID Route Table register block are Read-Only until reset.	RW	`0x0`
[29]	Commit	Setting this field requests that the updates are committed to the Chip ID Route Table register block. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[28:12]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[11:0]	NumIDRouteEntries	This field indicates the number of ID Route Table entries in use. The valid values are 0 to the number of supported ID Route Table entries. The value must be greater than or equal to the highest route table entry enabled. **Note** The ID Route Table entries are numbered from 0.	RW	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.6.4 C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1]

The C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1] characteristics are:

Purpose

These registers provide the Chip ID Route Table entries.

Field descriptions

Each of the C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1] registers, includes four IDRoute[(4 x n) + 0] to IDRoute[(4 x n) + 3] entries with the following bit layout. The index n identifies each register and is comprised between 0 and (N-1):

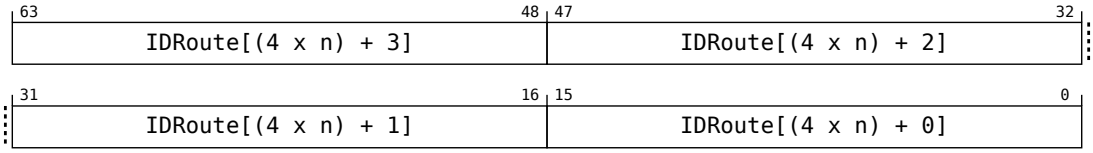


Figure B15.7: C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1] layout

Table B15.45: C2C_ChipIDRoute[0] to C2C_ChipIDRoute[N-1] field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	IDRoute[(4 x n) + 3]	See Table B15.41 for field description.	RW	
[47:32]	IDRoute[(4 x n) + 2]	See Table B15.41 for field description.	RW	
[31:16]	IDRoute[(4 x n) + 1]	See Table B15.41 for field description.	RW	
[15:0]	IDRoute[(4 x n) + 0]	See Table B15.41 for field description.	RW	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.7 Port register locator register block

Table B15.46 shows the Port register locator register block.

Table B15.46: Chip ID Route Table register block

Offset	Register name
`0x00`	C2C_PortRegLocHdr
`0x08`	C2C_PortRegLocCtrlStatus
`0x10` to (N x `0x8`) + `0x08`	C2C_Port[0]RegPtr to C2C_Port[N-1]RegPtr

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.7.1 C2C_PortRegLocHdr

The C2C_PortRegLocHdr characteristics are:

Purpose

This register provides information on the Port register locator block header.

Attributes

C2C_PortRegLocHdr is a 64-bit register.

Field descriptions

The C2C_PortRegLocHdr bit assignments are:

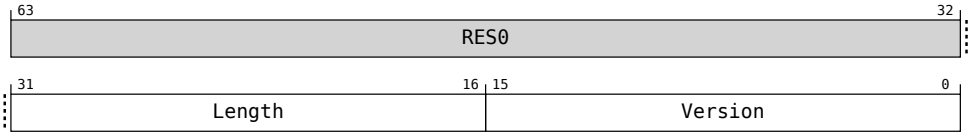


Table B15.47: C2C_PortRegLocHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the Port register locator block in bytes.	RO	IMPLEMENTATION DEFINED
[15:0]	Version	This field indicates the version of the Port register locator block. The valid value is `0x0`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.7.2 C2C_PortRegLocCtrlStatus

The C2C_PortRegLocCtrlStatus characteristics are:

Purpose

This register provides information on Port register locator control and status.

Attributes

C2C_PortRegLocCtrlStatus is a 64-bit register.

Field descriptions

The C2C_PortRegLocCtrlStatus bit assignments are:

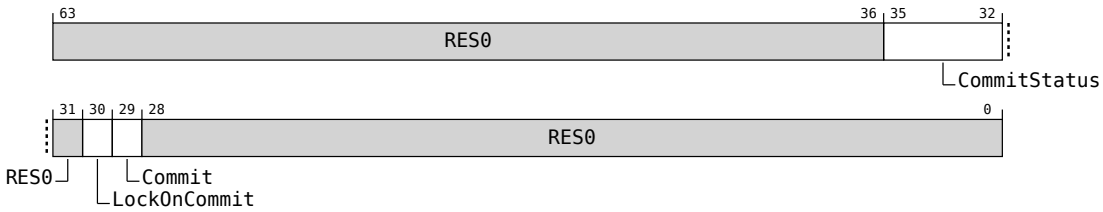


Table B15.48: C2C_PortRegLocCtrlStatus field descriptions

Bits	Field	Description	Access type	Reset
[63:36]	RES0	Reserved, RES0.		
[35:32]	CommitStatus	This field indicates the commit status of the Port register locator block. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RO	`0x0`
[31]	RES0	Reserved, RES0.		
[30]	LockOnCommit	This field requests that the programming of the Port register locator block is locked upon committing. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, all registers in the Port register locator block are Read-Only until reset.	RW	`0x0`
[29]	Commit	Setting this field requests that the updates are committed to the Port Register Locator register block. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[28:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.7.3 C2C_Port[0]RegPtr to C2C_Port[N-1]RegPtr

The C2C_Port[0]RegPtr to C2C_Port[N-1]RegPtr characteristics are:

Purpose

These registers provide the Port register pointers.

Field descriptions

The numerical value of N is defined by the NumPort field in C2C_ChipCap. The C2C_Port[0]RegPtr to C2C_Port[N-1]RegPtr bit assignments are:

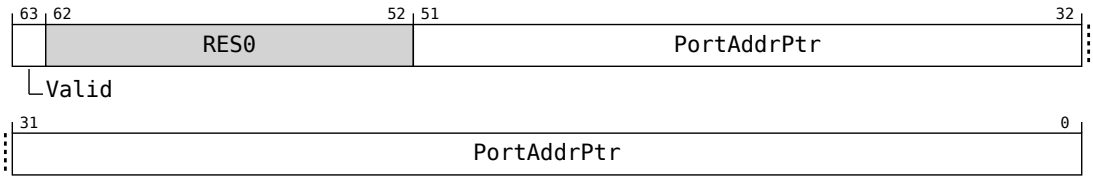


Figure B15.8: C2C_Port[0]RegPtr to C2C_Port[N-1]RegPtr layout

Table B15.49: C2C_Port[0]RegPtr to C2C_Port[N-1]RegPtr field descriptions

Bits	Field	Description	Access type	Reset
[63]	Valid	This field indicates this register has been programmed and is valid.	RW	`0x00` or `0x01` if the hardware is statically configured.
[62:52]	RES0	Reserved.		
[51:0]	PortAddrPtr	This field indicates a pointer to the registers for a port. The addresses must be 4K aligned. PortAddrPtr[11:0] must be 0.	RW	`0x00` or the hardware is statically configured

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.8 Port offset register block

Table B15.50 shows the port offset register block.

The offset value is with respect to the address of the base of the port register block.

Table B15.50: Port offset register block

Offset	Register name
0x00	C2C_PortRegOffsetHdr
0x08	C2C_PortRegOffset1
0x10	C2C_PortRegOffset2
0x18	Reserved

Figure B15.9 shows the format of the offset fields within C2C_PortRegOffset1 and C2C_PortRegOffset2.

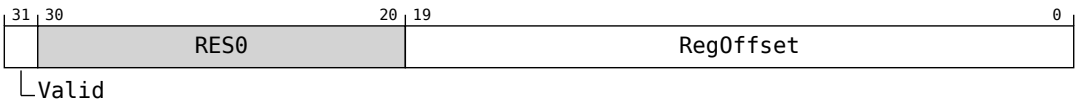


Figure B15.9: Port offset field format

Table B15.51 shows the field descriptions for the offset format fields.

Table B15.51: Offset format field descriptions

Bits	Field	Description	Access type	Reset
[31]	Valid	This field indicates the offset field is valid.	RO	IMPLEMENTATION DEFINED.
[30:20]	RES0	Reserved.		
[19:0]	RegOffset	This field indicates the offset from the Port Register Base for this register block. The value must be 8-byte aligned. Offset[2:0] must be equal to 0.	RO	IMPLEMENTATION DEFINED.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.8.1 C2C_PortRegOffsetHdr

The C2C_PortRegOffsetHdr characteristics are:

Purpose

This register provides information on the Port offset register block header.

Attributes

C2C_PortRegOffsetHdr is a 64-bit register.

Field descriptions

The C2C_PortRegOffsetHdr bit assignments are:

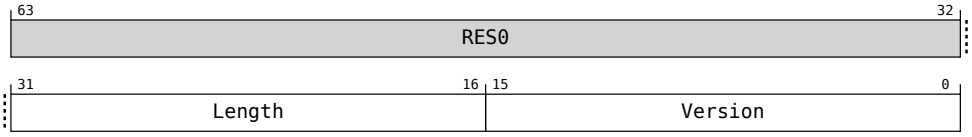


Table B15.52: C2C_PortRegOffsetHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the Port offset register block in bytes.	RO	`0x20`
[15:0]	Version	This field indicates the version of the Port offset register block. The valid value is `0x00`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.8.2 C2C_PortRegOffset1

The C2C_PortRegOffset1 characteristics are:

Purpose

This register provides the offset value for the Port capabilities and control register block.

Attributes

C2C_PortRegOffset1 is a 64-bit register.

Field descriptions

The C2C_PortRegOffset1 bit assignments are:

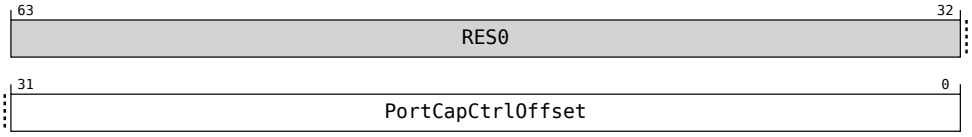


Table B15.53: C2C_PortRegOffset1 field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:0]	PortCapCtrlOffset	Offset to the Port Capabilities and Control register block. See Figure B15.9 for the offset field description.	RO	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.8.3 C2C_PortRegOffset2

The C2C_PortRegOffset2 characteristics are:

Purpose

This register provides the offset value for the Logical Link register block.

Attributes

C2C_PortRegOffset2 is a 64-bit register.

Field descriptions

The C2C_PortRegOffset2 bit assignments are:

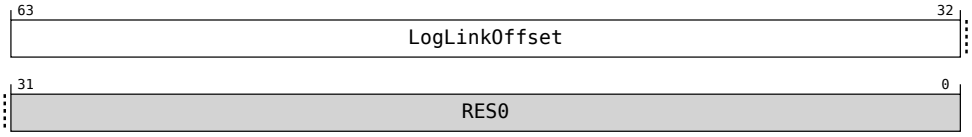


Table B15.54: C2C_PortRegOffset2 field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	LogLinkOffset	Offset to the Port common Logical Links register block. See Figure B15.9 for the offset field description.	RO	
[31:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.9 Port capabilities and control register block

Table B15.55 shows the port capabilities and control register block.

Table B15.55: Port capabilities and control register block

Offset	Register name
0x00	C2C_PortCapCtrlHdr
0x08	C2C_PortCap1
0x10	C2C_PortCap2
0x18	Reserved
0x20	C2C_PortCtrlStatus1
0x28	C2C_PortCtrlStatus2
0x30	Reserved
0x38	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.9.1 C2C_PortCapCtrlHdr

The C2C_PortCapCtrlHdr characteristics are:

Purpose

This register provides information on the Port capabilities and control header.

Attributes

C2C_PortCapCtrlHdr is a 64-bit register.

Field descriptions

The C2C_PortCapCtrlHdr bit assignments are:

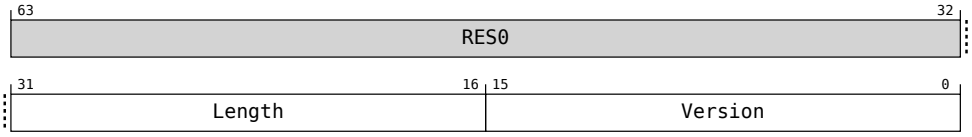


Table B15.56: C2C_PortCapCtrlHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the Port capabilities and control register block in bytes.	RO	`0x40`
[15:0]	Version	This field indicates the version of the Port capabilities and control register block. The valid value is `0x00`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.9.2 C2C_PortCap1

The C2C_PortCap1 characteristics are:

Purpose

This register provides information on the Port capabilities.

Attributes

C2C_PortCap1 is a 64-bit register.

Field descriptions

The C2C_PortCap1 bit assignments are:

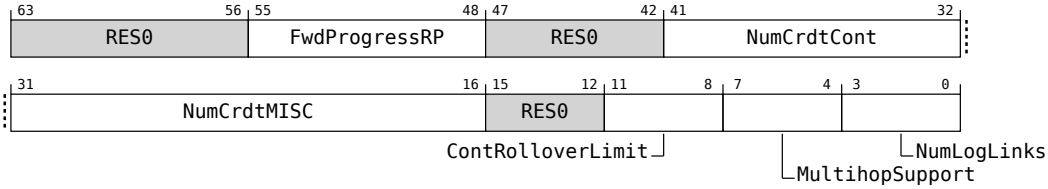


Table B15.57: C2C_PortCap1 field descriptions

Bits	Field	Description	Access type	Reset
[63:56]	RES0	Reserved, RES0.		
[55:48]	FwdProgressRP	This field indicates whether each internal RP can be considered free-flowing in the chip implementation. See B8.3.2 *Request resource plane attributes* for more information.	RO	IMPLEMENTATION DEFINED
[47:42]	RES0	Reserved, RES0.		
[41:32]	NumCrdtCont	This field indicates the number of container credits available.	RO	IMPLEMENTATION DEFINED
[31:16]	NumCrdtMISC	This field indicates the number of MISC credits available.	RO	IMPLEMENTATION DEFINED
[15:12]	RES0	Reserved, RES0.		
[11:8]	ContRolloverLimit	This field indicates the maximum number of containers following the first that can have messages rolling over from the previous container. This field is a Transmitter capability. If the port has multiple Logical Links, it is a common capability for all of them. The valid values for this field are 0 to 3.	RO	IMPLEMENTATION DEFINED
[7:4]	MultihopSupport	This field indicates the Multihop support capability of the port. See B7.5.4 *MultihopSupport capability* for more information. If the port has multiple Logical Links, it is a common capability for all of them. The following bit definitions are defined: **Bit[3:2]** Reserved. **Bit[1]** Pass-through. **Bit[0]** Unpack.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[3:0]	NumLogLinks	This field indicates the number of Logical Links supported by this Port. The valid values for this field are 0 to 3. When the port is disconnected, the value of this field is `0x0`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.9.3 C2C_PortCap2

The C2C_PortCap2 characteristics are:

Purpose

This register provides additional information on the Port capabilities.

Attributes

C2C_PortCap2 is a 64-bit register.

Field descriptions

The C2C_PortCap2 bit assignments are:

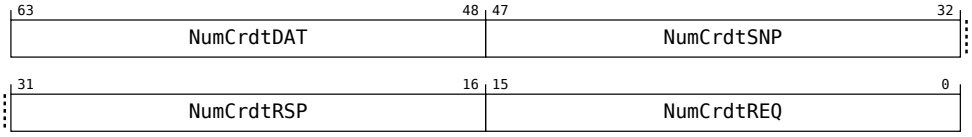


Table B15.58: C2C_PortCap2 field descriptions

Bits	Field	Description	Access type	Reset
[63:48]	NumCrdtDAT	This field indicates the number of Data credits available.	RO	IMPLEMENTATION DEFINED
[47:32]	NumCrdtSNP	This field indicates the number of Snoop credits available.	RO	IMPLEMENTATION DEFINED
[31:16]	NumCrdtRSP	This field indicates the number of Response credits available.	RO	IMPLEMENTATION DEFINED
[15:0]	NumCrdtREQ	This field indicates the number of Request credits available.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.9.4 C2C_PortCtrlStatus1

The C2C_PortCtrlStatus1 characteristics are:

Purpose

This register provides information on the Port control and status.

Attributes

C2C_PortCtrlStatus1 is a 64-bit register.

Field descriptions

The C2C_PortCtrlStatus1 bit assignments are:

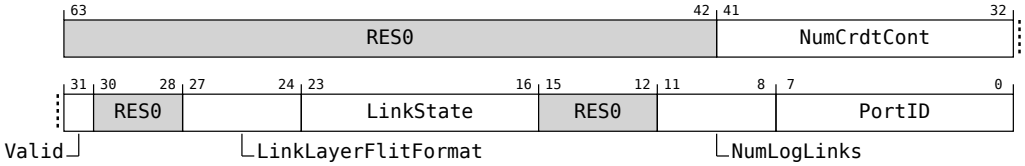


Table B15.59: C2C_PortCtrlStatus1 field descriptions

Bits	Field	Description	Access type	Reset
[63:42]	RES0	Reserved, RES0.		
[41:32]	NumCrdtCont	This field indicates the number of container credits allocated to this port. The maximum allowed value is indicated by the C2C_PortCap1.NumCrdtCont register field.	RW	`0x000`
[31]	Valid	This field indicates that this register is programmed and valid.	RW	`0x0`
[30:28]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[27:24]	LinkLayerFlitFormat	This field indicates the supported flit format by the physical link. `0b0000` MiscU.LinkStatus message is not received. `0b0001` 6 bytes required for the link header. Compatible with Format X, Format Y, and Format Z. `0b0010` 20 bytes required for the link header. Compatible with Format Y. `0b0011` 16 bytes required for the link header. Compatible with Format Z. `0b0100` Reserved. `0b0101` Reserved. `0b0110` Reserved. `0b0111` Reserved. `0b1000` Reserved. `0b1001` Reserved. `0b1010` Reserved. `0b1011` Reserved. `0b1100` Reserved. `0b1101` Reserved. `0b1110` Reserved. `0b1111` Reserved.	RO	`0x0`
[23:16]	LinkState	This field indicates the link state for the Physical link. The following bits are defined: **Bits[7:2]** Reserved. **Bit[1]** Disconnected. **Bit[0]** Active.	RO	`0x00`
[15:12]	RES0	Reserved, RES0.		
[11:8]	NumLogLinks	This field indicates the number of Logical Links used by this Port.	RW	`0x0`
[7:0]	PortID	This field indicates the port ID for this port.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.9.5 C2C_PortCtrlStatus2

The C2C_PortCtrlStatus2 characteristics are:

Purpose

This register provides additional information on the Port control and status.

Attributes

C2C_PortCtrlStatus2 is a 64-bit register.

Field descriptions

The C2C_PortCtrlStatus2 bit assignments are:

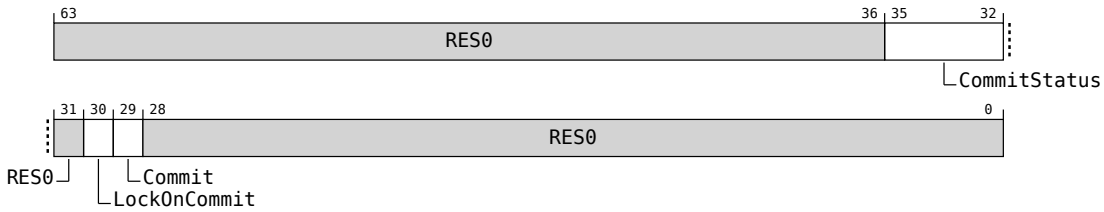


Table B15.60: C2C_PortCtrlStatus2 field descriptions

Bits	Field	Description	Access type	Reset
[63:36]	RES0	Reserved, RES0.		
[35:32]	CommitStatus	This field indicates the commit status of the Port capabilities and control register block. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RO	`0x0`
[31]	RES0	Reserved, RES0.		
[30]	LockOnCommit	This field requests that the programming of the Port capabilities and control register block is locked upon committing. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, all registers in the Port capabilities and control register block are Read-Only until reset.	RW	`0x0`
[29]	Commit	Setting this field requests that the updates are committed to the Port capabilities and control register block. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[28:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.10 Port common Logical Links register block

Table B15.61 shows the port common Logical Links registers. These registers provide offset values to locate the register blocks for each of the individual Logical Links.

Table B15.61: Common Logical Links register block

Offset	Register name
0x00	C2C_LogLinkCommonHdr
0x08	C2C_LogLinkCommonCap
0x10	Reserved
0x18	
0x20	C2C_LogLinkOffset1. See Table B15.67 for the individual Logical Link register block description.
0x28	C2C_LogLinkOffset2. See Table B15.67 for the individual Logical Link register block description.
0x30	Reserved
0x38	

Figure B15.10 shows the format of the offset fields within C2C_LogLinkOffset1 and C2C_LogLinkOffset2.

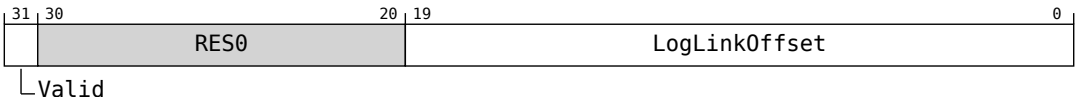


Figure B15.10: Logical Links offset field format

Table B15.62 shows the field descriptions for the offset format fields.

Table B15.62: Logical Links format field descriptions

Bit position	Field	Description	Attributes	Reset value
[31]	Valid	This field indicates the offset field is valid.	RO	IMPLEMENTATION DEFINED.
[30:20]	RES0	Reserved.		
[19:0]	LogLinkOffset	This field indicates the offset from the Port Register Base for this individual Logical Link register block. The value must be 8-byte aligned. Offset[2:0] must be equal to 0.	RO	IMPLEMENTATION DEFINED.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.10.1 C2C_LogLinkCommonHdr

The C2C_LogLinkCommonHdr characteristics are:

Purpose

This register provides information on the Logical Link register block common header.

Attributes

C2C_LogLinkCommonHdr is a 64-bit register.

Field descriptions

The C2C_LogLinkCommonHdr bit assignments are:

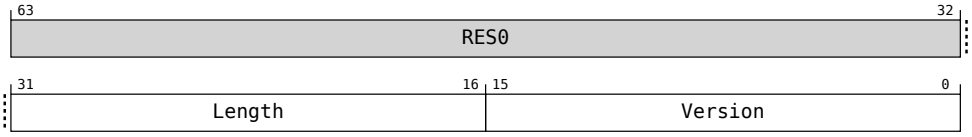


Table B15.63: C2C_LogLinkCommonHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the Port Logical Link register block in bytes.	RO	`0x40`
[15:0]	Version	This field indicates the version of the Port Logical Link register block. The valid value is `0x00`.	RO	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.10.2 C2C_LogLinkCommonCap

The C2C_LogLinkCommonCap characteristics are:

Purpose

This register provides information on the Logical Link common capabilities.

There are no common Logical Link capabilities defined in this issue of the specification.

Attributes

C2C_LogLinkCommonCap is a 64-bit register.

Field descriptions

The C2C_LogLinkCommonCap bit assignments are:

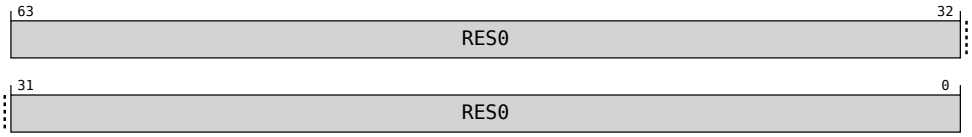


Table B15.64: C2C_LogLinkCommonCap field descriptions

Bits	Field	Description	Access type	Reset
[63:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.10.3 C2C_LogLinkOffset1

The C2C_LogLinkOffset1 characteristics are:

Purpose

This register provides the offset values to the individual register blocks for Logical Links 0 and 1.

Attributes

C2C_LogLinkOffset1 is a 64-bit register.

Field descriptions

The C2C_LogLinkOffset1 bit assignments are:

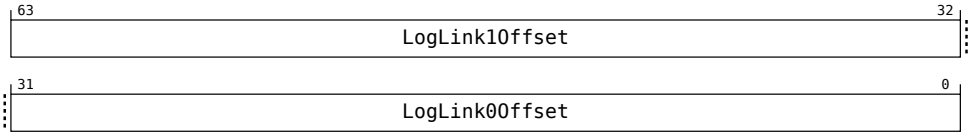


Table B15.65: C2C_LogLinkOffset1 field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	LogLink1Offset	Offset to the Port individual Logical Link 1 register block. See Table B15.62 for the offset field description.	RO	
[31:0]	LogLink0Offset	Offset to the Port individual Logical Link 0 register block. See Table B15.62 for the offset field description.	RO	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.10.4 C2C_LogLinkOffset2

The C2C_LogLinkOffset2 characteristics are:

Purpose

This register provides the offset values to the individual register blocks for Logical Links 2 and 3.

Attributes

C2C_LogLinkOffset2 is a 64-bit register.

Field descriptions

The C2C_LogLinkOffset2 bit assignments are:

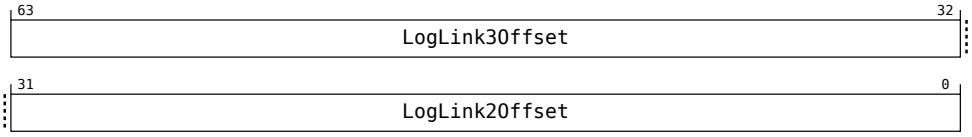


Table B15.66: C2C_LogLinkOffset2 field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	LogLink3Offset	Offset to the Port individual Logical Link 3 register block. See Table B15.62 for the offset field description.	RO	
[31:0]	LogLink2Offset	Offset to the Port individual Logical Link 2 register block. See Table B15.62 for the offset field description.	RO	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11 Port individual Logical Links register block

Table B15.67 shows the individual Logical Link register block. Each of the supported individual Logical Link register blocks has the format described in this section.

The offset value is with respect to the address of the base of the individual Logical Link register block.

Table B15.67: Individual Logical Link register block

Offset	Register name
0x000	C2C_LogLinkHdr
0x008	C2C_LogLinkCap
0x010	Reserved
0x018	C2C_LogLinkCtrlStatus1
0x020	C2C_LogLinkCtrlStatus2
0x028	C2C_LogLinkRPMapTable
0x030	C2C_LogLinkRPAttrTable
0x038	Reserved
0x040	C2C_PropSuppUniform1
0x048	C2C_PropSuppUniform2
0x050	C2C_PropSuppRx1
0x058	C2C_PropSuppRx2
0x060	C2C_PropSuppTx1
0x068	C2C_PropSuppTx2
0x070	Reserved
0x078	
0x080	C2C_PropAdvdUniform1
0x088	C2C_PropAdvdUniform2
0x090	C2C_PropAdvdRx1
0x098	C2C_PropAdvdRx2
0x0A0	C2C_PropAdvdTx1
0x0A8	C2C_PropAdvdTx2
0x0B0	Reserved
0x0B8	
0x0C0	C2C_PropInfdUniform1
0x0C8	C2C_PropInfdUniform2
0x0D0	C2C_PropInfdRx1
0x0D8	C2C_PropInfdRx2

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B15.67 – *Continued from previous page*

Offset	Register name
0x0E0	C2C_PropInfdTx1
0x0E8	C2C_PropInfdTx2
0x0F0	Reserved
0x0F8	
0x100	C2C_PropNgtdUniform1
0x108	C2C_PropNgtdUniform2
0x110	C2C_PropNgtdRx1
0x118	C2C_PropNgtdRx2
0x120	C2C_PropNgtdTx1
0x128	C2C_PropNgtdTx2
0x130	Reserved
0x138	
0x140	C2C_LogLinkCrdt1
0x148	C2C_LogLinkCrdt2
0x150	C2C_LogLinkCrdt3
0x158	C2C_LogLinkCrdt4
0x160	C2C_LogLinkCrdt5
0x168	Reserved
0x170	
0x178	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.1 C2C_LogLinkHdr

The C2C_LogLinkHdr characteristics are:

Purpose

This register provides information on the individual Logical Link register block header.

Attributes

C2C_LogLinkHdr is a 64-bit register.

Field descriptions

The C2C_LogLinkHdr bit assignments are:

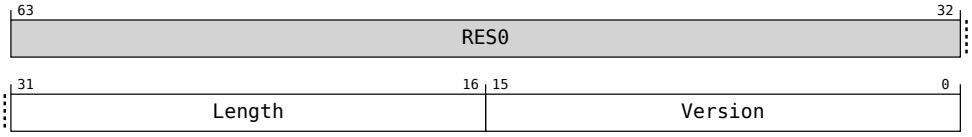


Table B15.68: C2C_LogLinkHdr field descriptions

Bits	Field	Description	Access type	Reset
[63:32]	RES0	Reserved, RES0.		
[31:16]	Length	This field indicates the length of the individual Logical Link register block in bytes.	RO	`0x180`
[15:0]	Version	This field indicates the version of the individual Logical Link register block. The valid value is `0x0`.	RO	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.2 C2C_LogLinkCap

The C2C_LogLinkCap characteristics are:

Purpose

This register provides information on the individual Logical Link capabilities.

There are no Logical Link-specific capabilities defined in this issue of the specification.

Attributes

C2C_LogLinkCap is a 64-bit register.

Field descriptions

The C2C_LogLinkCap bit assignments are:

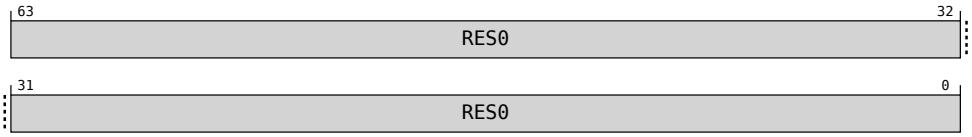


Table B15.69: C2C_LogLinkCap field descriptions

Bits	Field	Description	Access type	Reset
[63:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.3 C2C_LogLinkCtrlStatus1

The C2C_LogLinkCtrlStatus1 characteristics are:

Purpose

This register provides information on the individual Logical Link control and status.

Attributes

C2C_LogLinkCtrlStatus1 is a 64-bit register.

Field descriptions

The C2C_LogLinkCtrlStatus1 bit assignments are:

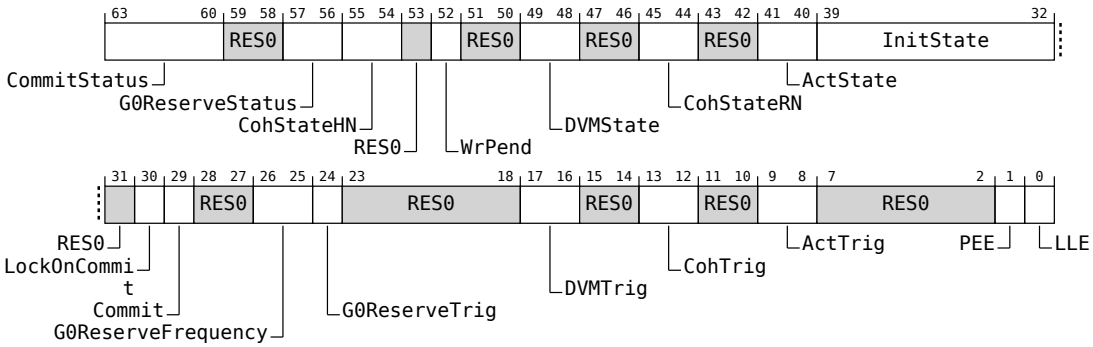


Table B15.70: C2C_LogLinkCtrlStatus1 field descriptions

Bits	Field	Description	Access type	Reset
[63:60]	CommitStatus	This field indicates the commit status of the C2C_LogLinkCtrlStatus1 register only. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RO	`0x0`
[59:58]	RES0	Reserved, RES0.		
[57:56]	G0ReserveStatus	This field indicates the status of the request for the packer to start reserving granule G0 for MAC insertion. The following bits are defined: **Bit[1]** Error. When set to `0b1`, this bit indicates that the request failed to complete. **Bit[0]** G0Reserved. When set to `0b1`, this bit indicates the packer is guaranteed to be reserving granule G0 at least every G0ReserveFrequency containers. When set to `0b0`, this bit indicates the packer is not guaranteed to be reserving granule G0.	RO	`0x00`
[55:54]	CohStateHN	This field indicates the Coherency state of remote requesters connected to the coherency domain managed from local home nodes. `0b00` CohDisabled `0b01` CohConnect `0b10` CohEnabled `0b11` CohDisconnect	RO	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[53]	RES0	Reserved, RES0.		
[52]	WrPend	This field indicates whether any write transactions are pending at the egress gateway. `0b0` No write transactions are pending. `0b1` Write transactions are pending.	RO	`0x0`
[51:50]	RES0	Reserved, RES0.		
[49:48]	DVMState	This field indicates the DVM state for the Logical Link. `0b00` DVMDisabled `0b01` DVMConnect `0b10` DVMEnabled `0b11` DVMDisconnect	RO	`0x0`
[47:46]	RES0	Reserved, RES0.		
[45:44]	CohStateRN	This field indicates the Coherency state of local requesters connected to the coherency domain extending across the Logical Link. `0b00` CohDisabled `0b01` CohConnect `0b10` CohEnabled `0b11` CohDisconnect	RO	`0x0`
[43:42]	RES0	Reserved, RES0.		
[41:40]	ActState	This field indicates the Activation state for the Logical Link. `0b00` STOP `0b01` ACTIVATE `0b10` RUN `0b11` DEACTIVATE	RO	`0x00`
[39:32]	InitState	This field indicates the initialization state for the Logical Link. The following bits are defined: **Bits[7:3]** Reserved **Bit[2]** PropNegComplete **Bit[1]** RemoteActive **Bit[0]** LinkStatusRcvd. The value is Reserved for Logical Links, except Logical Link 0. An implementation must reset Bits[2:1] when the Logical Link is deactivated.	RO	`0x00`
[31]	RES0	Reserved, RES0.		
[30]	LockOnCommit	This field requests that the programming of the C2C_LogLinkCtrlStatus1 register is locked upon committing. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, the C2C_LogLinkCtrlStatus1 register is Read-Only until reset.	RW	`0x0`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[29]	Commit	Setting this field requests that the updates are committed to the C2C_LogLinkCtrlStatus1 register. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RW	`0x0`
[28:27]	RES0	Reserved, RES0.		
[26:25]	G0ReserveFrequency	This field defines the minimum frequency, expressed in number of containers, with which granule G0 is requested to be reserved for MAC insertion. `0b00` Reserve granule G0 every container. `0b01` Reserve granule G0 every 2 containers. `0b10` Reserved. `0b11` Reserved.	RW	`0x0`
[24]	G0ReserveTrig	This field enables software to control the reservation of granule G0 by the packer for MAC insertion. When G0ReserveStatus[0] = 0, writing to this field requests that the packer starts reserving a granule G0 at least once every G0ReserveFrequency number of containers, and following the message rollover rules in B11.2.2 *IDE message rollover rules*. A packer is permitted to over-reserve granule G0. When G0ReserveStatus[0] = 1, writing to this field requests that the packer stops reserving granule G0. A packer is permitted to ignore this request and continue reserving granule G0 in packed containers. This field can only be written to `0b1`. Writing this field to `0b0` results in UNPREDICTABLE behavior. This field is automatically cleared by the hardware when the toggle request has been processed and the value of G0ReserveStatus has been updated. When the IDE_G0_Reservation property has negotiated to False, writing to this bit results in UNPREDICTABLE behavior.	RW	`0x0`
[23:18]	RES0	Reserved, RES0.		
[17:16]	DVMTrig	This field enables software to request connection or disconnection to DVM activities across the C2C link. See B12.4.4 *DVM Trigger, DVMTrig* for more information. `0b00` No action. `0b01` Send DVMConnectReq. `0b10` Send DVMDisconnectReq. `0b11` Reserved.	RW	`0x0`
[15:14]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[13:12]	CohTrig	This field enables software to request connection or disconnection of local requesters to a coherency domain that is hardware-managed from across the link. See B12.3.3 *Coherency Trigger, CohTrig* for more information. `0b00` No action. `0b01` Send CohConnectReq. `0b10` Send CohDisconnectReq. `0b11` Reserved.	RW	`0x0`
[11:10]	RES0	Reserved, RES0.		
[9:8]	ActTrig	This field enables software to request activation or deactivation of the interface. When Deactivation_Support[1:0] has negotiated to `0b00`, setting this field to `0b10` results in UNPREDICTABLE behavior. For more information on the usage of this field, see B12.2 *Interface activation and deactivation*. `0b00` No action. `0b01` Send ActivateReq. `0b10` Send DeactivateReq. `0b11` Reserved.	RW	`0x0`
[7:2]	RES0	Reserved, RES0.		
[1]	PEE	This field indicates whether the Property Exchange is requested by this interface during the initialization. `0b0` Property exchange is not requested. `0b1` Property exchange is requested.	RW	IMPLEMENTATION DEFINED
[0]	LLE	This field indicates whether the Logical Link is enabled. `0b0` Logical link is disabled or not valid. `0b1` Logical link is enabled.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.4 C2C_LogLinkCtrlStatus2

The C2C_LogLinkCtrlStatus2 characteristics are:

Purpose

This register provides additional information on the individual Logical Link control and status.

Attributes

C2C_LogLinkCtrlStatus2 is a 64-bit register.

Field descriptions

The C2C_LogLinkCtrlStatus2 bit assignments are:

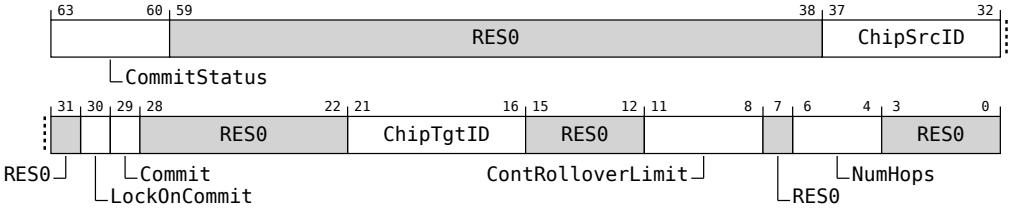


Table B15.71: C2C_LogLinkCtrlStatus2 field descriptions

Bits	Field	Description	Access type	Reset
[63:60]	CommitStatus	This field indicates the commit status of the Logical Link register block, except B15.11.3 *C2C_LogLinkCtrlStatus1*. See B15.1.2 *Commit and lock-on-commit functionality* for more information.	RO	`0x00`
[59:38]	RES0	Reserved, RES0.		
[37:32]	ChipSrcID	This field determines the ChipSrcID container header field to be used by containers transmitted on this Logical Link. The valid values for this field are 0 to 63.	RW	`0x00`
[31]	RES0	Reserved, RES0.		
[30]	LockOnCommit	This field requests that the programming of the Logical Link register block is locked upon committing, except the B15.11.3 *C2C_LogLinkCtrlStatus1* register. See B15.1.2 *Commit and lock-on-commit functionality* for more information. `0b0` No lock on commit. `0b1` Lock on commit. Once committed, all registers in the Logical Link register block are Read-Only until reset, except the B15.11.3 *C2C_LogLinkCtrlStatus1* register.	RW	

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Bits	Field	Description	Access type	Reset
[29]	Commit	Setting this field requests that the updates are committed to the Logical Link register block, except the B15.11.3 *C2C_LogLinkCtrlStatus1* register. This field will remain set until cleared by the software. This field must be cleared to initiate a subsequent commit. See B15.1.2 *Commit and lock-on-commit functionality* for more information.		`0x0`
[28:22]	RES0	Reserved, RES0.		
[21:16]	ChipTgtID	This field determines the ChipTgtID container header field to be used by containers transmitted on this Logical Link. The valid values for this field are 0 to 63.	RW	`0x00`
[15:12]	RES0	Reserved, RES0.		
[11:8]	ContRolloverLimit	This field indicates the container rollover limit for this Logical Link. The valid values for this field are 0 to 3.	RW	`0x0`
[7]	RES0	Reserved, RES0.		
[6:4]	NumHops	This field indicates the number of hops to the target chip for this Logical Link. `0b000` 1 hop. `0b001` 2 hops. `0b010` 3 hops. `0b011` 4 hops. `0b100` 5 hops. `0b101` 6 hops. `0b110` 7 hops. `0b111` 8 hops.	RW	`0x00`
[3:0]	RES0	Reserved, RES0.		

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.5 C2C_LogLinkRPMapTable

The C2C_LogLinkRPMapTable characteristics are:

Purpose

This register provides information on the Resource Plane map table for the Logical Link.

Attributes

C2C_LogLinkRPMapTable is a 64-bit register.

Field descriptions

The C2C_LogLinkRPMapTable bit assignments are:

63:60	59:56	55:52	51:48	47:44	43:40	39:36	35:32
RxRPMT7	RxRPMT6	RxRPMT5	RxRPMT4	RxRPMT3	RxRPMT2	RxRPMT1	RxRPMT0

31:28	27:24	23:20	19:16	15:12	11:8	7:4	3:0
TxRPMT7	TxRPMT6	TxRPMT5	TxRPMT4	TxRPMT3	TxRPMT2	TxRPMT1	TxRPMT0

Table B15.72: C2C_LogLinkRPMapTable field descriptions

Bits	Field	Description	Access type	Reset
[63:60]	RxRPMT7	This entry contains the on-chip RP number to which C2C RP 7 is mapped.	RW	0x0
[59:56]	RxRPMT6	This entry contains the on-chip RP number to which C2C RP 6 is mapped.	RW	0x0
[55:52]	RxRPMT5	This entry contains the on-chip RP number to which C2C RP 5 is mapped.	RW	0x0
[51:48]	RxRPMT4	This entry contains the on-chip RP number to which C2C RP 4 is mapped.	RW	0x0
[47:44]	RxRPMT3	This entry contains the on-chip RP number to which C2C RP 3 is mapped.	RW	0x0
[43:40]	RxRPMT2	This entry contains the on-chip RP number to which C2C RP 2 is mapped.	RW	0x0
[39:36]	RxRPMT1	This entry contains the on-chip RP number to which C2C RP 1 is mapped.	RW	0x0
[35:32]	RxRPMT0	This entry contains the on-chip RP number to which C2C RP 0 is mapped.	RW	0x0
[31:28]	TxRPMT7	This entry contains the C2C RP number to which on-chip RP 7 is mapped.	RW	0x0
[27:24]	TxRPMT6	This entry contains the C2C RP number to which on-chip RP 6 is mapped.	RW	0x0
[23:20]	TxRPMT5	This entry contains the C2C RP number to which on-chip RP 5 is mapped.	RW	0x0
[19:16]	TxRPMT4	This entry contains the C2C RP number to which on-chip RP 4 is mapped.	RW	0x0
[15:12]	TxRPMT3	This entry contains the C2C RP number to which on-chip RP 3 is mapped.	RW	0x0
[11:8]	TxRPMT2	This entry contains the C2C RP number to which on-chip RP 2 is mapped.	RW	0x0
[7:4]	TxRPMT1	This entry contains the C2C RP number to which on-chip RP 1 is mapped.	RW	0x0
[3:0]	TxRPMT0	This entry contains the C2C RP number to which on-chip RP 0 is mapped.	RW	0x0

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.6 C2C_LogLinkRPAttrTable

The C2C_LogLinkRPAttrTable characteristics are:

Purpose

This register provides information on the traffic attributes for the Transmitter to use on an RP.

Attributes

C2C_LogLinkRPAttrTable is a 64-bit register.

Field descriptions

The C2C_LogLinkRPAttrTable bit assignments are:

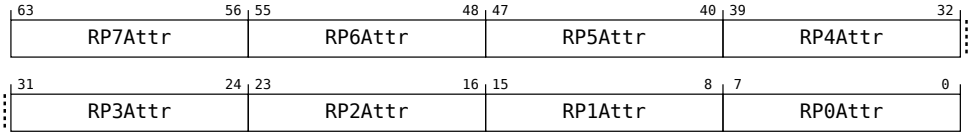


Table B15.73: C2C_LogLinkRPAttrTable field descriptions

Bits	Field	Description	Access type	Reset
[63:56]	RP7Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP7.	RW	0x00
[55:48]	RP6Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP6.	RW	0x00
[47:40]	RP5Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP5.	RW	0x00
[39:32]	RP4Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP4.	RW	0x00
[31:24]	RP3Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP3.	RW	0x00
[23:16]	RP2Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP2.	RW	0x00
[15:8]	RP1Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP1.	RW	0x00
[7:0]	RP0Attr	This field defines the traffic attributes for the Transmitter to use on the C2C Request RP0.	RW	0x00

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Figure B15.11 shows the layout of the RP*Attr field formats in C2C_LogLinkRPAttrTable

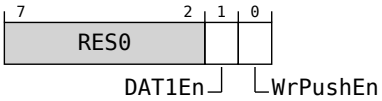


Figure B15.11: RP attributes

Table B15.74 shows the encodings for each RP*Attr attributes entry format.

Table B15.74: RPxAttr attributes entry encoding descriptions

Bits	Field	Description	Access type	Reset
[7:2]	RES0	Reserved.		
[1]	DAT1En	This field indicates whether traffic on this RP is allowed to use DAT1Credit credit type for WrReqData* messages. This bit can only be set when WrPushEn is also set.	RW	IMPLEMENTATION DEFINED
[0]	WrPushEn	This field indicates whether traffic on this RP is allowed to use WrReqData* messages.	RW	IMPLEMENTATION DEFINED

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.7 Uniform property registers

Table B15.75 shows the attribute and reset values for each Uniform property register set.

Table B15.75: Uniform property set attribute and reset values

Register set	Access type	Reset values
Supported	RO	IMPLEMENTATION DEFINED.
Advertised	RW	IMPLEMENTATION DEFINED.
Informed	RO	`0x0`
Negotiated	RO	`0x0`

Figure B15.12 shows the layout of the fields within the C2C_Prop*Uniform1 property registers.

C2C_Prop*Uniform1 represents the following registers:

- C2C_PropSuppUniform1
- C2C_PropAdvdUniform1
- C2C_PropInfdUniform1
- C2C_PropNgtdUniform1

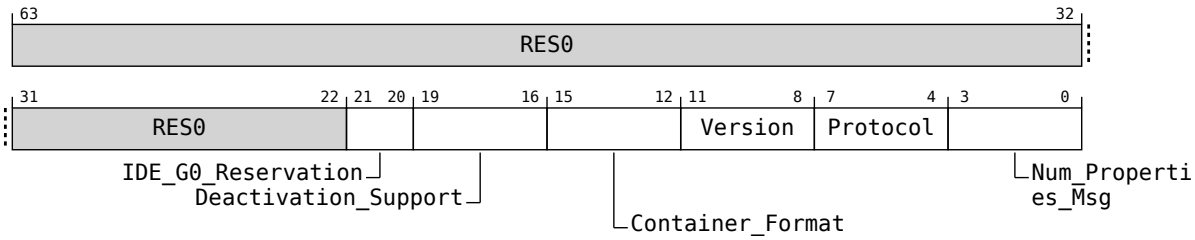


Figure B15.12: C2C_Prop*Uniform1 register layout

Table B15.76 lists the fields for C2C_Prop*Uniform1.

Table B15.76: C2C_Prop*Uniform1 register field positions

Bits	Field	Description
[63:22]	RES0	Reserved.
[21:20]	IDE_G0_Reservation	See Table B14.7 for more information.
[19:16]	Deactivation_Support	See Table B14.7 for more information.
[15:12]	Container_Format	See Table B14.7 for more information.
[11:8]	Version	See Table B14.7 for more information.
[7:4]	Protocol	See Table B14.7 for more information.
[3:0]	Num_Properties_Msg	See Table B14.7 for more information.

Figure B15.13 shows the layout of the fields within the C2C_Prop*Uniform2 property registers.

C2C_Prop*Uniform2 represents the following registers:

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- C2C_PropSuppUniform2
- C2C_PropAdvdUniform2
- C2C_PropInfdUniform2
- C2C_PropNgtdUniform2

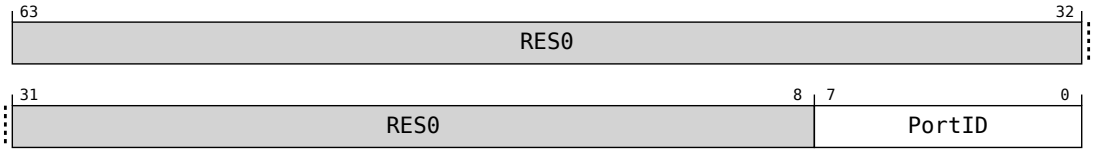


Figure B15.13: C2C_Prop*Uniform2 register layout

Table B15.77 shows the encodings for C2C_Prop*Uniform2.

Table B15.77: C2C_Prop*Uniform2 register field descriptions

Bits	Field	Description
[63:8]	RES0	Reserved.
[7:0]	PortID	See Table B14.7 for more information.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.8 Receiver property registers

Table B15.78 shows the attribute and reset values for each Receiver property register set.

Table B15.78: Receiver property set attribute and reset values

Register set	Access type	Reset values
Supported	RO	IMPLEMENTATION DEFINED.
Advertised	RW	IMPLEMENTATION DEFINED.
Informed	RO	`0x0`
Negotiated	RO	`0x0`

Figure B15.14 shows the layout of the fields within the C2C_Prop*Rx1 property registers.

C2C_Prop*Rx1 represents the following registers:

- C2C_PropSuppRx1
- C2C_PropAdvdRx1
- C2C_PropInfdRx1
- C2C_PropNgtdRx1

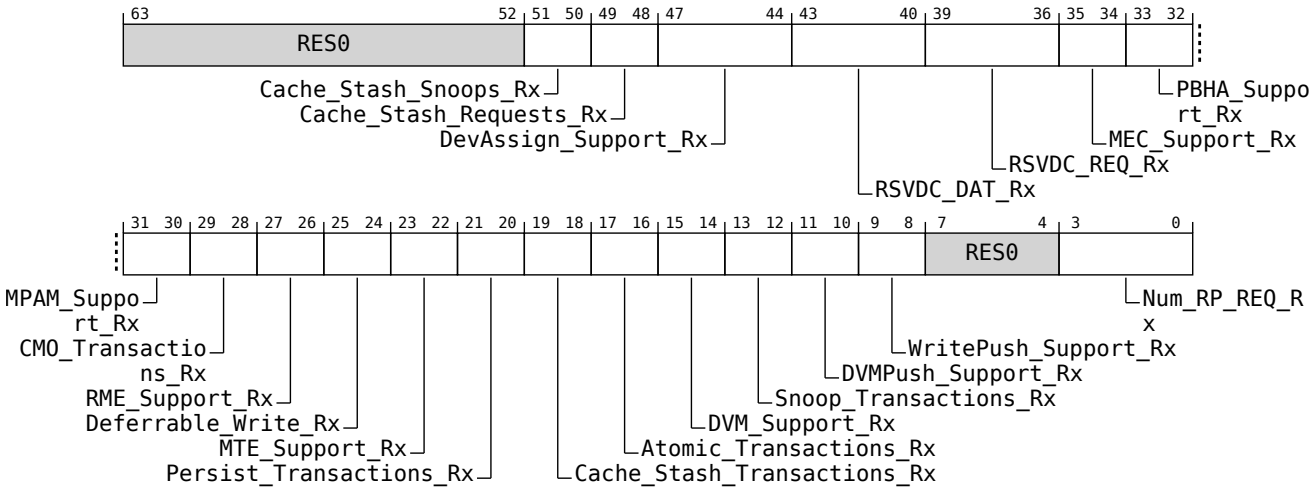


Figure B15.14: C2C_Prop*Rx1 layout

Table B15.79 lists the fields for C2C_Prop*Rx1.

Table B15.79: C2C_Prop*Rx1 field positions

Bits	Field	Description
[63:52]	RES0	Reserved

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B15.79 – *Continued from previous page*

Bits	Field	Description
[51:50]	Cache_Stash_Snoops_Rx	See Table B14.4 for more information.
[49:48]	Cache_Stash_Requests_Rx	See Table B14.4 for more information.
[47:44]	DevAssign_Support_Rx	See Table B14.8 for more information.
[43:40]	RSVDC_DAT_Rx	See Table B14.1 for more information.
[39:36]	RSVDC_REQ_Rx	See Table B14.1 for more information.
[35:34]	MEC_Support_Rx	See Table B14.1 for more information.
[33:32]	PBHA_Support_Rx	See Table B14.1 for more information.
[31:30]	MPAM_Support_Rx	See Table B14.1 for more information.
[29:28]	CMO_Transactions_Rx	See Table B14.4 for more information.
[27:26]	RME_Support_Rx	See Table B14.4 for more information.
[25:24]	Deferable_Write_Rx	See Table B14.4 for more information.
[23:22]	MTE_Support_Rx	See Table B14.4 for more information.
[21:20]	Persist_Transactions_Rx	See Table B14.4 for more information.
[19:18]	Cache_Stash_Transactions_Rx	See Table B14.4 for more information.
[17:16]	Atomic_Transactions_Rx	See Table B14.4 for more information.
[15:14]	DVM_Support_Rx	See Table B14.4 for more information.
[13:12]	Snoop_Transactions_Rx	See Table B14.4 for more information.
[11:10]	DVMPush_Support_Rx	See Table B14.8 for more information.
[9:8]	WritePush_Support_Rx	See Table B14.8 for more information.
[7:4]	RES0	Reserved.
[3:0]	Num_RP_REQ_Rx	See Table B14.8 for more information.

Figure B15.15 shows the layout of the fields within the C2C_Prop*Rx2 property registers.

C2C_Prop*Rx2 represents the following registers:

- C2C_PropSuppRx2
- C2C_PropAdvdRx2
- C2C_PropInfdRx2
- C2C_PropNgtdRx2

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

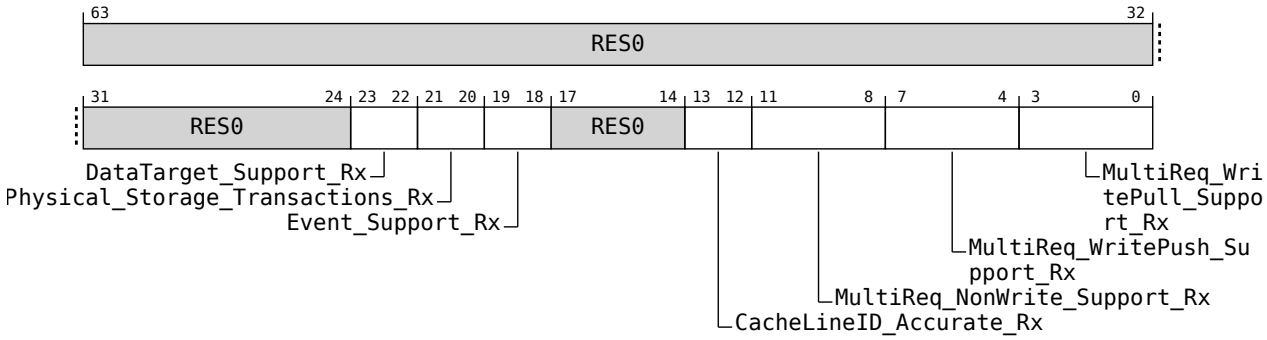


Figure B15.15: C2C_Prop*Rx2 layout

Table B15.80 lists the fields for C2C_Prop*Rx2.

Table B15.80: C2C_Prop*Rx2 register field positions

Bits	Field	Description
[63:24]	RES0	Reserved
[23:22]	DataTarget_Support_Rx	See Table B14.1 for more information.
[21:20]	Physical_Storage_Transactions_Rx	See Table B14.4 for more information.
[19:18]	Event_Support_Rx	See Table B14.8 for more information.
[17:14]	RES0	Reserved
[13:12]	CacheLineID_Accurate_Rx	See Table B14.4 for more information.
[11:8]	MultiReq_NonWrite_Support_Rx	See Table B14.4 for more information.
[7:4]	MultiReq_WritePush_Support_Rx	See Table B14.4 for more information.
[3:0]	MultiReq_WritePull_Support_Rx	See Table B14.4 for more information.

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.9 Transmitter property registers

Table B15.81 shows the attribute and reset values for each Transmitter property register set.

Table B15.81: Transmitter property set attribute and reset values

Register set	Access type	Reset values
Supported	RO	IMPLEMENTATION DEFINED.
Advertised	RW	IMPLEMENTATION DEFINED.
Informed	RO	`0x0`
Negotiated	RO	`0x0`

Figure B15.16 shows the layout of the fields within the C2C_Prop*Tx1 property registers.

C2C_Prop*Tx1 represents the following registers:

- C2C_PropSuppTx1
- C2C_PropAdvdTx1
- C2C_PropInfdTx1
- C2C_PropNgtdTx1

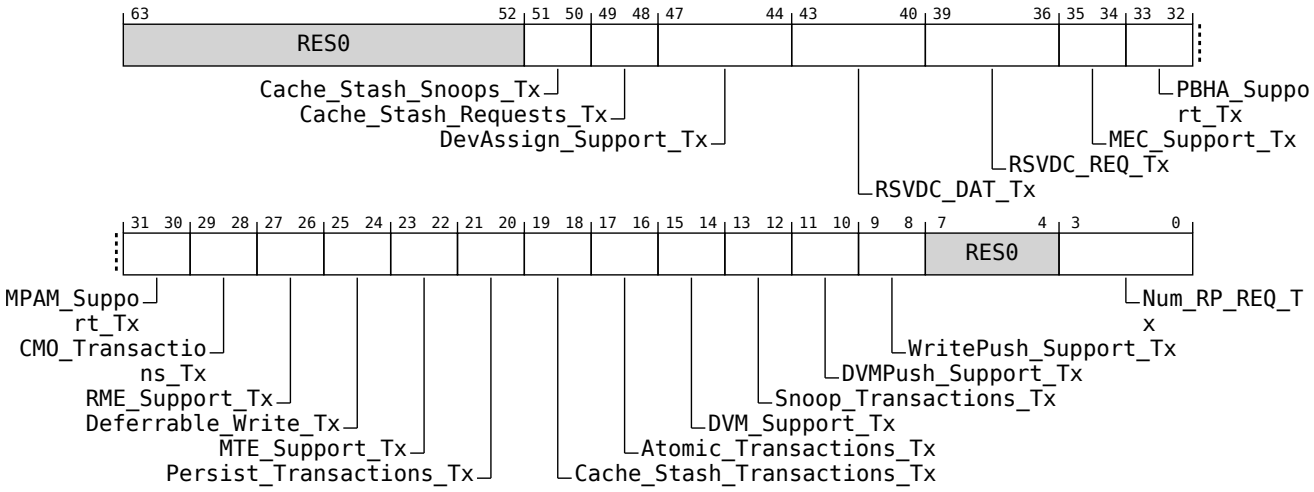


Figure B15.16: C2C_Prop*Tx1 layout

Table B15.82 lists the fields for C2C_Prop*Tx1.

Table B15.82: C2C_Prop*Tx1 register field positions

Bits	Field	Description
[63:52]	RES0	Reserved

Continued on next page

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B15.82 – *Continued from previous page*

Bits	Field	Description
[51:50]	Cache_Stash_Snoops_Tx	See Table B14.5 for more information.
[49:48]	Cache_Stash_Requests_Tx	See Table B14.5 for more information.
[47:44]	DevAssign_Support_Tx	See Table B14.9 for more information.
[43:40]	RSVDC_DAT_Tx	See Table B14.2 for more information.
[39:36]	RSVDC_REQ_Tx	See Table B14.2 for more information.
[35:34]	MEC_Support_Tx	See Table B14.2 for more information.
[33:32]	PBHA_Support_Tx	See Table B14.2 for more information.
[31:30]	MPAM_Support_Tx	See Table B14.2 for more information.
[29:28]	CMO_Transactions_Tx	See Table B14.5 for more information.
[27:26]	RME_Support_Tx	See Table B14.5 for more information.
[25:24]	Deferable_Write_Tx	See Table B14.5 for more information.
[23:22]	MTE_Support_Tx	See Table B14.5 for more information.
[21:20]	Persist_Transactions_Tx	See Table B14.5 for more information.
[19:18]	Cache_Stash_Transactions_Tx	See Table B14.5 for more information.
[17:16]	Atomic_Transactions_Tx	See Table B14.5 for more information.
[15:14]	DVM_Support_Tx	See Table B14.5 for more information.
[13:12]	Snoop_Transactions_Tx	See Table B14.5 for more information.
[11:10]	DVMPush_Support_Tx	See Table B14.9 for more information.
[9:8]	WritePush_Support_Tx	See Table B14.9 for more information.
[7:4]	RES0	Reserved
[3:0]	Num_RP_REQ_Tx	See Table B14.9 for more information.

Figure B15.17 shows the layout of the fields within the C2C_Prop*Tx2 property registers.

C2C_Prop*Tx2 represents the following registers:

- C2C_PropSuppTx2
- C2C_PropAdvdTx2
- C2C_PropInfdTx2
- C2C_PropNgtdTx2

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

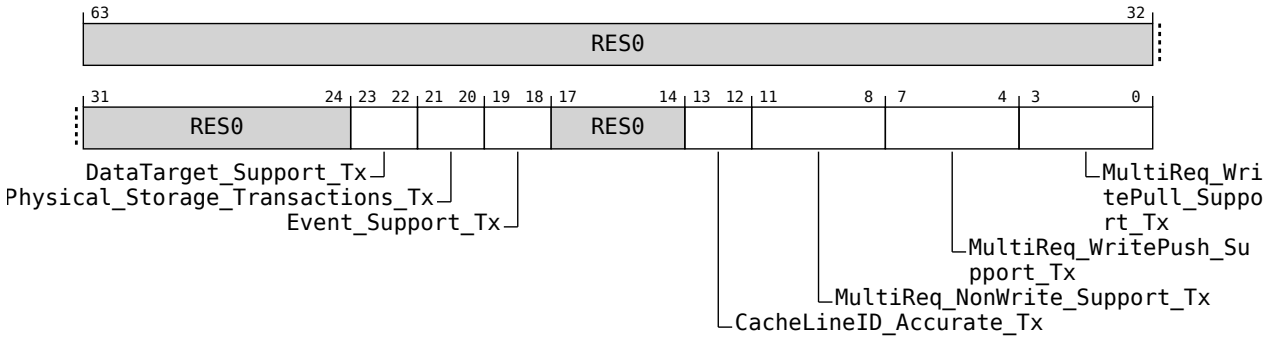


Figure B15.17: C2C_Prop*Tx2 layout

Table B15.83 lists the fields for C2C_Prop*Tx2.

Table B15.83: C2C_Prop*Tx2 register field positions

Bits	Field	Description
[63:24]	RES0	Reserved
[23:22]	DataTarget_Support_Tx	See Table B14.2 for more information.
[21:20]	Physical_Storage_Transactions_Tx	See Table B14.5 for more information.
[19:18]	Event_Support_Tx	See Table B14.9 for more information.
[17:14]	RES0	Reserved
[13:12]	CacheLineID_Accurate_Tx	See Table B14.5 for more information.
[11:8]	MultiReq_NonWrite_Support_Tx	See Table B14.5 for more information.
[7:4]	MultiReq_WritePush_Support_Tx	See Table B14.5 for more information.
[3:0]	MultiReq_WritePull_Support_Tx	See Table B14.5 for more information.

B15.11.10 C2C_LogLinkCrdt1

The C2C_LogLinkCrdt1 characteristics are:

Purpose

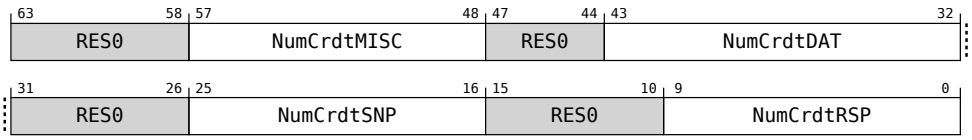
This register provides information on the Logical Link message class credits for RSP, SNP, DAT, and MISC channels.

Attributes

C2C_LogLinkCrdt1 is a 64-bit register.

Field descriptions

The C2C_LogLinkCrdt1 bit assignments are:



Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B15.84: C2C_LogLinkCrdt1 field descriptions

Bits	Field	Description	Access type	Reset
[63:58]	RES0	Reserved, RES0.		
[57:48]	NumCrdtMISC	This field indicates the maximum number of MISC credits allocated to this Logical Link.	RW	`0x00`
[47:44]	RES0	Reserved, RES0.		
[43:32]	NumCrdtDAT	This field indicates the maximum number of DAT credits allocated to this Logical Link.	RW	`0x00`
[31:26]	RES0	Reserved, RES0.		
[25:16]	NumCrdtSNP	This field indicates the maximum number of SNP credits allocated to this Logical Link.	RW	`0x00`
[15:10]	RES0	Reserved, RES0.		
[9:0]	NumCrdtRSP	This field indicates the maximum number of RSP credits allocated to this Logical Link.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.11 C2C_LogLinkCrdt2

The C2C_LogLinkCrdt2 characteristics are:

Purpose

This register provides information on the Logical Link message class credits for the REQ channel.

Attributes

C2C_LogLinkCrdt2 is a 64-bit register.

Field descriptions

The C2C_LogLinkCrdt2 bit assignments are:

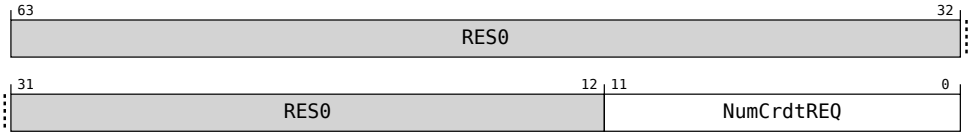


Table B15.85: C2C_LogLinkCrdt2 field descriptions

Bits	Field	Description	Access type	Reset
[63:12]	RES0	Reserved, RES0.		
[11:0]	NumCrdtREQ	This field indicates the number of REQ credits allocated to this Logical Link, including all per-RP credits and shared credits.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.12 C2C_LogLinkCrdt3

The C2C_LogLinkCrdt3 characteristics are:

Purpose

This register provides information on the Logical Link message class REQ credits for Resource Planes 0, 1, 2, and 3.

Attributes

C2C_LogLinkCrdt3 is a 64-bit register.

Field descriptions

The C2C_LogLinkCrdt3 bit assignments are:

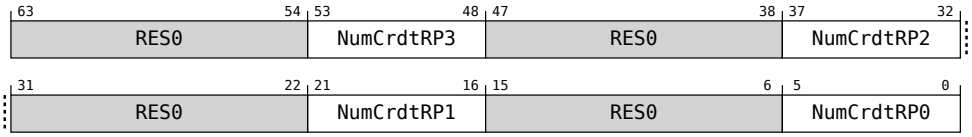


Table B15.86: C2C_LogLinkCrdt3 field descriptions

Bits	Field	Description	Access type	Reset
[63:54]	RES0	Reserved, RES0.		
[53:48]	NumCrdtRP3	This field indicates the number of REQ credits allocated to RP 3.	RW	`0x00`
[47:38]	RES0	Reserved, RES0.		
[37:32]	NumCrdtRP2	This field indicates the number of REQ credits allocated to RP 2.	RW	`0x00`
[31:22]	RES0	Reserved, RES0.		
[21:16]	NumCrdtRP1	This field indicates the number of REQ credits allocated to RP 1.	RW	`0x00`
[15:6]	RES0	Reserved, RES0.		
[5:0]	NumCrdtRP0	This field indicates the number of REQ credits allocated to RP 0.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.13 C2C_LogLinkCrdt4

The C2C_LogLinkCrdt4 characteristics are:

Purpose

This register provides information on the Logical Link message class REQ credits for Resource Planes 4, 5, 6, and 7.

Attributes

C2C_LogLinkCrdt4 is a 64-bit register.

Field descriptions

The C2C_LogLinkCrdt4 bit assignments are:

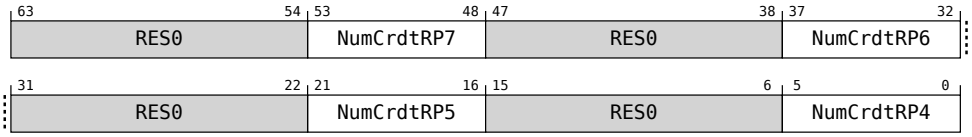


Table B15.87: C2C_LogLinkCrdt4 field descriptions

Bits	Field	Description	Access type	Reset
[63:54]	RES0	Reserved, RES0.		
[53:48]	NumCrdtRP7	This field indicates the number of REQ credits allocated to RP 7.	RW	`0x00`
[47:38]	RES0	Reserved, RES0.		
[37:32]	NumCrdtRP6	This field indicates the number of REQ credits allocated to RP 6.	RW	`0x00`
[31:22]	RES0	Reserved, RES0.		
[21:16]	NumCrdtRP5	This field indicates the number of REQ credits allocated to RP 5.	RW	`0x00`
[15:6]	RES0	Reserved, RES0.		
[5:0]	NumCrdtRP4	This field indicates the number of REQ credits allocated to RP 4.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B15.11.14 C2C_LogLinkCrdt5

The C2C_LogLinkCrdt5 characteristics are:

Purpose

This register provides information on the Logical Link message class shared credits for REQ, and for the different DAT credit types.

Attributes

C2C_LogLinkCrdt5 is a 64-bit register.

Field descriptions

The C2C_LogLinkCrdt5 bit assignments are:

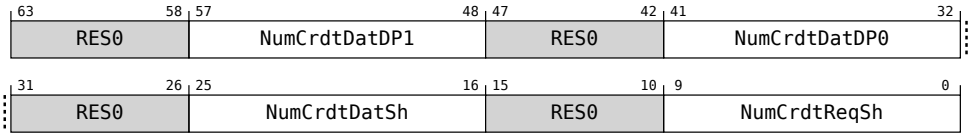


Table B15.88: C2C_LogLinkCrdt5 field descriptions

Bits	Field	Description	Access type	Reset
[63:58]	RES0	Reserved, RES0.		
[57:48]	NumCrdtDatDP1	This field indicates the number of DAT credits allocated to DP 1.	RW	`0x00`
[47:42]	RES0	Reserved, RES0.		
[41:32]	NumCrdtDatDP0	This field indicates the number of DAT credits allocated to DP 0.	RW	`0x00`
[31:26]	RES0	Reserved, RES0.		
[25:16]	NumCrdtDatSh	This field indicates the number of DAT credits allocated to Shared.	RW	`0x00`
[15:10]	RES0	Reserved, RES0.		
[9:0]	NumCrdtReqSh	This field indicates the number of REQ credits allocated to Shared.	RW	`0x00`

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Part C
Appendicies

Chapter C1
Revisions

This appendix describes the technical changes between released issues of this specification.

- C1.1 *Changes between Issue A and Issue A.b*
- C1.2 *Changes between Issue A.b and Issue B*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

C1.1 Changes between Issue A and Issue A.b

Table C1.1: Changes between Issue A and Issue A.b

Change	Location
Clarification: Additional information around property exchange on either side of the interface	B10.1 *Introduction* B5.9.1.4 *PropertyReq*
Clarification: When credits can be sent during interface Activation flow	B12.2.4 *Activation and Deactivation flow* B13.2 *Initialization flow*
Update: Maximum number of credits to track.	B8.2 *Message credits*
Addition: Use of shared credits and dedicated credits	B8.3 *Resource Planes*
Addition: When the Completer is permitted to send Comp for WritePush	B5.6.1 *Transaction flow*
Update: Coherency states for Request Nodes and Home Nodes	B12.3.1 *Coherency connect states*
Correction: Use of SnpDVMOp is not permitted	B10.1 *Introduction*
Clarification: MPAM widths and packing	Table B5.6 Table B5.10
Correction: FlitFormat value of `001` is compatible with Format X and Format Y	Table B5.23
Clarification: Number of credits for a Resp2 message	Table B8.1
Update: C10 field in WrReqData messages is OWO	Table B5.10
Addition: MiscU.Connect cannot be sent when the associated property is False	B12.3.2 *Messages* B12.4.2 *Messages*
Addition: MiscU.Connect can only be sent during RUN state	B5.9.2 *Connect message format*
Clarification: ChunkValid must be consistent with Size	B5.2.4 *Chunk valid, ChunkValid*
Correction: Which transaction types can use WritePush semantics	B5.6 *WritePush message*
Relaxation: Message-to-interface rules between transactions	B2.4 *Multiple interfaces*
Clarification: Applicability of various CMO properties to different CMO transactions	Table B14.4 Table B14.5

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential

C1.2 Changes between Issue A.b and Issue B

Table C1.2: Changes between Issue A and Issue B

Change	Location
New feature: Format Z container	B3.1 *Containers*
New feature: Request Resource Plane mapping	B8.3.1 *Request Resource Plane mapping*
New feature: Software-visible registers	Chapter B15 *Software-visible registers*
New feature: NodeID management	Chapter B6 *Node ID management*
New feature: CohTrig field	B12.3.3 *Coherency Trigger, CohTrig*
New feature: DVMTrig field	B12.4.4 *DVM Trigger, DVMTrig*
New feature: PortID hardware exchange	B13.3 *PortID exchange*
New feature: Properties added to MiscU.Properties messages	B14.5 *Property packing in MiscU.Properties messages*
New feature: Event message	B5.9.6 *Event message format*
New feature: Multihop routing	B1.1.3 *Hub-and-spoke topology* B7.5 *Multihop routing* B8.8 *Multihop flow control* B9.5 *DVM with Multihop routing*
New feature: Multi-request	Chapter B4 *Multi-request* B5.6.2 *Message fields* B8.8 *Multihop flow control* B4.2 *Container granule savings for Multi-request*
Update: Software can write to the ActTrig field	B12.2.5 *Activation trigger, ActTrig*
Update: Support for multiple MiscU.Properties messages in the interface initialization flow	B13.2 *Initialization flow*
Clarification: Resolution of Container_Format on physical and Logical Links	B14.4.3.1 *Container_Format resolution*
Update: Protocol header fields	B3.2 *Protocol header*
New feature: IDE	Chapter B11 *Integrity and Data Encryption*

Copyright © 2024-2026 Arm Limited or its affiliates. All rights reserved.
Non-confidential